

FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

Simon Coudenys

Student number: 02101804

Supervisors: Prof. dr. ir. Peter Ossieur, Dr. ir. Gertjan Coudyzer

Counsellors: Jonas De Busscher, Jakob Declercq, Axl Bomhals

Master's dissertation submitted in order to obtain the degree of Master of Science in Electronics and ICT Engineering Technology -
Embedded Systems

Academic year 2025-2026

This page is intentionally left blank.

This page is intentionally left blank.

FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

Simon Coudenys

Student number: 02101804

Supervisors: Prof. dr. ir. Peter Ossieur, Dr. ir. Gertjan Coudyzer

Counsellors: Jonas De Busscher, Jakob Declercq, Axl Bomhals

Master's dissertation submitted in order to obtain the degree of Master of Science in Electronics and ICT Engineering Technology -
Embedded Systems

Academic year 2025-2026

This page is intentionally left blank.

Acknowledgements

I would like to take this opportunity to give a special thank you to everyone who made this thesis possible. Writing a thesis is never done alone, just as engineering is never done alone.

My heartfelt salutations go out to ir. Jonas De Busscher. His proposal of this novel architecture was the reason this thesis became possible. His enthusiastic and insightful guidance throughout the course of my research has been invaluable and deserves the highest praise.

Another special thank you goes to ir. Axl Bomhals. His technical expertise throughout the year (especially regarding Vivado and VHDL programming) helped me tremendously, particularly during the moments when Vivado software seemed to become incomprehensible to me.

Not to forget are my friends: from the friends who went climbing with me as a sometimes much-needed break from writing, to the friends who are also writing their master's dissertations and experienced the same emotions I felt throughout this year. Therefore, I would like to thank Lucas Handekyn, Lies Kindt, Carlo De Kock and Emma Van Lierde (and many more) for their fantastic support. Without them, this thesis would also not have been possible.

Finally, I would also like to take this opportunity to thank my mom and dad, who not only supported me in countless ways this year, but throughout my entire academic career. Without them, I would not even be here. Thanks, Mom & Dad.

This page is intentionally left blank.

Admission to loan

“The author(s) gives (give) permission to make this master’s dissertation available for consultation and to copy parts of this master’s dissertation for personal use. In the case of any other use, the copyright terms have to be respected, in particular with regard to the obligation to state expressly the source when quoting results from this master’s dissertation.”

Toegang tot bruikleen

“De auteur(s) geeft (geven) de toelating dit afstudeerwerk voor consultatie beschikbaar te stellen en delen van het afstudeerwerk te kopiëren voor persoonlijk gebruik. Elk ander gebruik valt onder de beperkingen van het auteursrecht, in het bijzonder met betrekking tot de verplichting de bron uitdrukkelijk te vermelden bij het aanhalen van resultaten uit dit afstudeerwerk.”

This page is intentionally left blank.

Thesis as part of an exam

“This master’s dissertation is part of an exam. Any comments formulated by the assessment committee during the oral presentation of the master’s dissertation are not included in this text.”

Simon Coudenys, Mei 2026

This page is intentionally left blank.

FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

by
SIMON COUDENYS

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics and ICT Engineering Technology - Embedded Systems

Supervisor: prof. dr. ir. Peter Ossieur, dr.ir. Gertjan Coudyzer
Counsellor: ir. Jonas De Busscher, ir. Axl Bomhals, dr. ir. Jakob Declercq

Faculty of Engineering and Architecture
Ghent University

Department of Information Technology
Chair: prof. dr. ir. Peter Ossieur

Abstract - Coherent optical links are increasingly considered for high-capacity intra-datacenter communication, but their receiver digital signal processing (DSP) requires efficient carrier recovery to compensate residual carrier frequency offset and laser phase noise. Conventional blind phase search provides robust phase estimation for square-QAM constellations, but its hardware cost grows with the number of tested phase candidates. This thesis presents a fixed-point field programmable gate array (FPGA) implementation of a low-complexity carrier-recovery backend for baudrate-sampled 16-state quadrature amplitude modulation (16QAM) coherent links. The architecture combines serialised binary phase search, which estimates the wrapped centre phase of a processing block, with spread estimation, which estimates the signed intra-block phase evolution. Phase unwrapping and section-based phase compensation then convert these estimates into per-section correction phases for the delayed received symbol block.

The implemented register-transfer level (RTL) uses packed Q1.15 I/Q samples, trigonometric phase coefficients in the serialised binary phase search (SBPS) path and Q4.28 phase words in the spread-estimation, unwrapping and compensation paths. The spread-estimation branch includes candidate-symbol selection, fourth-power processing, normalized dot-product evaluation, sign estimation, filtering and look-up table (LUT)-based angle recovery. A top-level wrapper aligns the SBPS centre-phase estimate, the signed spread estimate and the delayed raw I/Q block before final compensation.

The design is verified using Python reference models, generated fixed-point vectors, VHSIC Hardware Description Language (VHDL) testbenches and post-processing scripts. Top-level simulations show no persistent phase drift between the Python model and the RTL; the largest measured mean model difference over the evaluated cases is 2.251 mrad. For the default residual-carrier frequency offset (CFO) test case, the impaired signal has an error vector magnitude (EVM) of 142%. Blockwise SBPS correction reduces this to 12.69%, while combined SBPS and spread-estimation compensation reduces it further to 7.41%. Implementation on an XCU200 target shows that the evaluated $B = 32$, six-stage, eight-section configuration uses about 33.3k LUTs, 36.0k flip-flops, 229 DSP blocks and four BRAM tiles, and closes timing at an 8 ns clock period after a local pipeline register is added in the spread-estimation path.

Keywords - DSP, Baudrate sampling, 16QAM, Intra datacenter, Coherent optical links

This page is intentionally left blank.

FPGA implementatie van vereenvoudigde draaggolfreconstructie voor 16QAM coherente optische intradatacenterverbindingen van de volgende generatie

Simon Coudenys

Promotoren: prof. dr. ir. Peter Ossieur, dr. ir. Gertjan Coudyzer
Begeleiders: ir. Jonas De Busscher, ir. Axl Bomhals, dr. ir. Jakob Declercq

Abstract □ Coherente optische verbindingen worden steeds relevanter voor intra-datacentercommunicatie, maar vereisen een efficiënte digitale carrier recovery. Deze thesis implementeert een laag-complexe backend voor baudrate-gesampled 16QAM-signalen op FPGA. De architectuur combineert geserialiseerde binaire blind phase search met spread estimation, zodat zowel de centrale blokfase als de intra-blok fase-evolutie kunnen worden gecompenseerd. De vaste-punt RTL wordt gevalideerd met Python-referentiemodellen, VHDL-testbanken en systeemniveau-resultaten

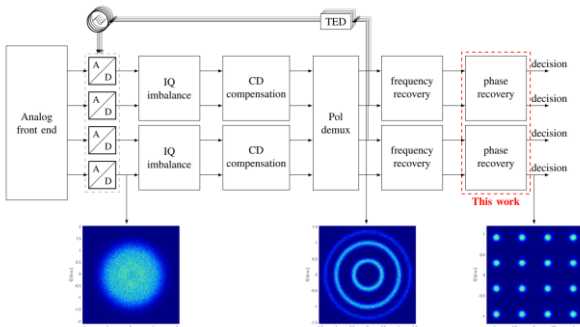
Keywords □ Coherente optische links, 16QAM, carrier recovery, FPGA, SBPS, spread estimation

I. INTRODUCTION

De groei van datacenterverkeer, onder meer door cloudtoepassingen en AI-workloads, verhoogt de druk op optische intra-datacenterlinks [1]. Coherente communicatie maakt hogere spectrale efficiëntie mogelijk dan klassieke IMDD-systemen, maar verschuift een deel van de complexiteit naar de digitale ontvanger [2]. Carrier recovery is daarbij een kritische stap. Restfrequentieoffset en laserfase-ruis veroorzaken een roterende constellatie. Blind phase search is robuust, maar evalueert traditioneel veel testfasen. Voor een hardware-implementatie met hoge paralleliteit wordt die kost snel dominant [3].

II. VOORGESTELDE ARCHITECTUUR

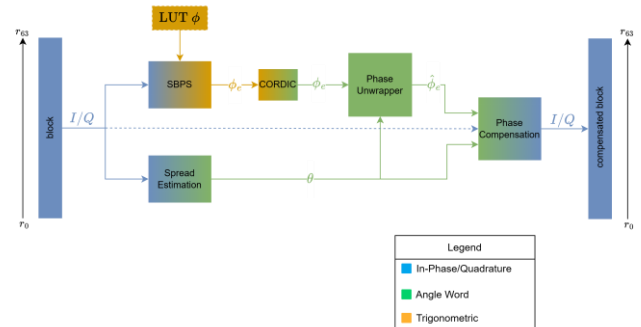
De onderzochte architectuur splitst de taak op in twee schattingen. De SBPS-keten bepaalt de gewrapte middenfase van een blok. De SE-keten schat de getekende fasespreiding over datzelfde blok. Nadien bewaart de fase-unwrapper de juiste kwadranttak tussen opeenvolgende blokken en genereert de fasecompensator sectiefasen voor de uiteindelijke rotatie.



Figuur 1. Positie van de carrier-recoverybackend in de volledige DSP-keten (thesisfiguur 1.2)

III. VASTE-PUNT REPRESENTATIE

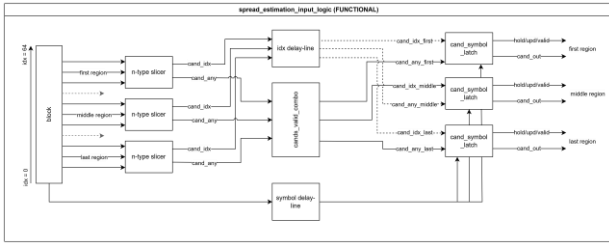
De implementatie gebruikt verschillende representaties op plaatsen waar die functioneel het meest efficiënt zijn. Ontvangen I/Q-symbolen worden als gepakte Q1.15-waarden verwerkt. De SBPS-stages gebruiken sinus- en cosinuscoëfficiënten omdat hun hoofdoperatie een Cartesiaanse rotatie met slicer-metrick is. Na de SBPS-keten wordt de fase-informatie naar een fasewoord domein gebracht. De SE-uitgang, de phase unwrapper en de compensatieblokken werken met getekende Q4.28-fasewoorden. Deze keuze vereenvoudigt optellingen, spreidingscorrectie en sectiefasegeneratie.



Figuur 2. Overzicht van de datarepresentaties tussen de hoofdblokken (thesisfiguur 4.3)

IV. HARDWARE-IMPLEMENTATIE

De top-level wrapper vertrekt van één ontvangen blokinterface. Het ruwe I/Q-blok wordt parallel aangeboden aan de SBPS-keten, de SE-keten en een expliciete I/Q-vertragsingslijn. De vertragingen zorgen ervoor dat de middenfase, de fasespreiding en de bijhorende samples op hetzelfde klokmoment in de PUPC-compensatieketen samenkomen. De spread-estimationketen bevat een inputselector, vierde-machtsberekening, genormaliseerd dotproduct, tekenbepaling, filters en een LUT-gebaseerde arccos-evaluatie. De inputselector zoekt representatieve symbolen nabij het begin, midden en einde van een blok. Middenringsymbolen van 16QAM worden vermeden wanneer ze onvoldoende betrouwbare informatie over de fasespreiding geven.

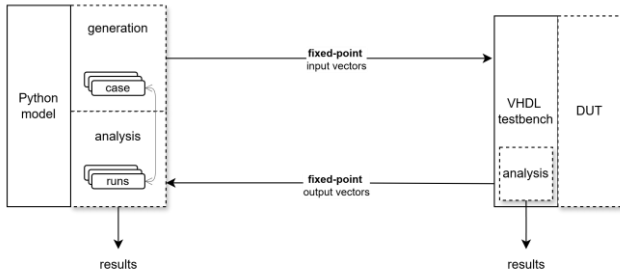


Figuur 3. Functionele opbouw van de inputselectie voor spread estimation (thesisfiguur 4.13)

De uiteindelijke compensatie gebruikt secties. Voor elke sectie wordt een representatieve fase berekend uit de ongewrapte middenfase en de getekende spreiding. Deze fase wordt vervolgens uitgebreid naar de symbolen in de sectie en aangeboden aan een parallelle CORDIC-rotatiebank.

V. VERIFICATIE

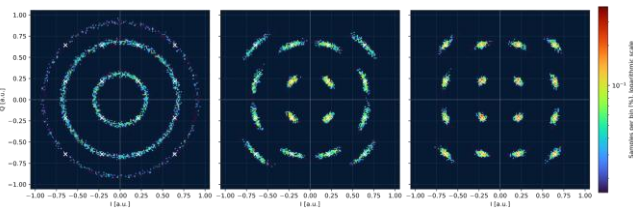
De verificatieketen combineert Python-referentiemodellen met vaste-punt vectoren en VHDL-testbanken. Blokniveau-testen controleren onder meer SBPS-fase-uitgangen, spread-estimation-rekenpaden, fase-unwrapping en CORDIC-rotatie. Op systeemniveau wordt de volledige wrapper gevoed met verstoorde 16QAM-blokken en wordt de gecorrigeerde uitgang opnieuw geanalyseerd in Python. Deze structuur maakt het mogelijk om numerieke fouten en timingfouten afzonderlijk te lokaliseren. Vooral bij de SE- en compensatiepaden is geldigheidstiming belangrijk, omdat data- en controlepaden door verschillende vaste vertragingen lopen voor ze opnieuw worden samengevoegd.



Figuur 4. Gebruikte verificatieflow met Python-vectoren, VHDL-simulatie en post-processing (thesisfiguur 5.1)

VI. RESULTATEN

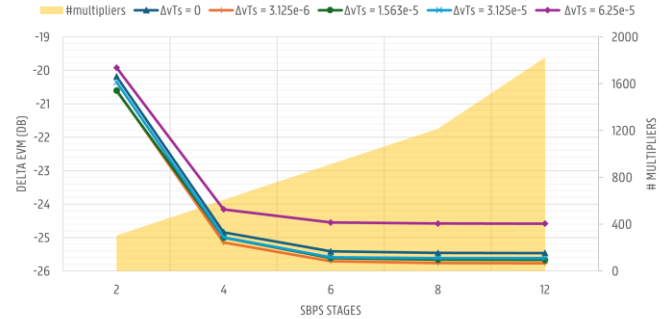
De systeemresultaten tonen het verschil tussen blokgewijze correctie en correctie met fasespreiding. In de standaardcase heeft het verstoorde signaal een EVM van 142%. SBPS-only-correctie reduceert dit tot 12.69%, terwijl SBPS+SE dit verder reduceert tot 7.41%.



Figuur 5. Constellaties voor, na SBPS-only en na SBPS+SE correctie (thesisfiguur 6.5)

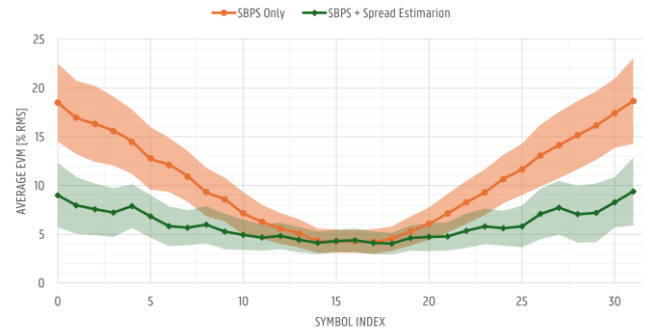
VII. PARAMETERAFWEGING

Het aantal SBPS-stages bepaalt de resolutie van de middenfaseschatting. De resultaten tonen dat de grootste verbetering optreedt bij de overgang van weinig stages naar ongeveer zes stages. Daarna vlakkt de winst af, omdat de resterende fout vooral door ruis, spread-estimationfouten en compensatiegranulariteit wordt bepaald.



Figuur 6. Invloed van het aantal SBPS-stages op de relatieve EVM-reductie (thesisfiguur 6.6)

Ook het aantal compensatiesecties vertoont een duidelijke trade-off. Eén sectie komt overeen met zuiver blokgewijze fasecorrectie. Meerdere secties reduceren de randfout van het blok, maar de winst wordt kleiner naarmate de secties finer worden.



Figuur 7. Lokalisatie van de rest-EVM binnen het blok voor SBPS-only en SBPS+SE (thesisfiguur 6.8)

VIII. IMPLEMENTATIERESULTATEN

Voor de besproken configuratie met blokgrrootte 32, zes SBPS-stages en acht compensatiesecties werd de implementatie geëvalueerd op een XCU200-doel. De gereviseerde timingversie sluit timing bij een klokperiode van 8 ns nadat een lokaal pipeline-register werd toegevoegd in het reciprocal-multiplicationpad van de spread-estimationketen.

Tabel 1. Implementatiepunt gebruikt voor de belangrijkste resultaatbespreking.

Parameter	Waarde
Blokgrrootte	32
SBPS-stages	6
Compensatiesecties	8
Resourcegebruik	33.3k LUT, 36.0k FF, 229 DSP, 4 BRAM
Timing	8 ns klokperiode

IX. BESLUIT

De thesis toont aan dat de voorgestelde SBPS+SE-architectuur als vaste-punt FPGA-datapad kan worden geïmplementeerd. De SE-uitbreiding levert een meetbare verbetering ten opzichte van zuiver blokgewijze SBPS-correctie, vooral aan de randen van het verwerkingsblok. De belangrijkste resterende beperkingen zijn de lokale lineariteitsaannname van de fasespreiding, de afhankelijkheid van geschikte 16QAM-kandidaatsymbolen en de nood aan verdere vaste-punt en streaming-integratieverificatie.

REFERENCES

- [1] D. J. Blumenthal, H. Ballani, R. O. Behunin, J. E. Bowers, P. Costa, D. Lenoski, P. A. Morton, S. B. Papp, and P. T. Rakich, "Frequency-stabilized links for coherent wdm fiber interconnects in the datacenter," *Journal of Lightwave Technology*, vol. 38, no. 13, pp. 3376–3386, 2020.
- [2] X. Zhou, R. Urata, and H. Liu, "Beyond 1 tb/s intra-data center interconnect technology: Im-dd or coherent?," *Journal of Lightwave Technology*, 2020.
- [3] M. Kuschnerov, F. N. Hauske, K. Piyawanno, B. Spinnler, M. S. Alfiad, A. Napoli, and B. Lankl, "Dsp for coherent single-carrier receivers," *Journal of Lightwave Technology*, vol. 27, no. 16, pp. 3614–3622, 2009.

This page is intentionally left blank.

Table of Contents

Acknowledgements	vii
Admission to loan	ix
Thesis as part of an exam	xi
Abstract	xiii
Extended Abstract	xv
Table of Contents	xix
List of Figures	xxi
List of Tables	xxii
List of Acronyms	xxiii
1 Introduction	1
1.1 Context and motivation	1
1.2 Problem statement	2
1.3 Objectives of this thesis	2
1.4 Contributions	3
1.5 Thesis outline	4
2 Background	5
2.1 Coherent optical communication systems	5
2.2 Phase noise and frequency offset	6
2.3 Carrier recovery in DSP chains	7
2.4 Blind Phase Search	7
2.5 CORDIC	8
3 Proposed phase recovery architecture	9
3.1 System overview	9
3.2 Serialized binary phase search	10
3.2.1 Pattern effect	12
3.2.2 Block size consideration	12
3.3 Spread estimation algorithm	13
3.3.1 Input symbol consideration	15
3.4 Phase unwrapping	16
3.5 Phase compensation	18
4 Hardware implementation	21
4.1 Data representation and scaling	21
4.2 Serialized binary phase search block	25
4.2.1 Functional operation	26
4.2.2 Data flow	26
4.2.3 Timing flow	28
4.2.4 Radius-directed stages	29
4.3 Spread estimation block	30
4.3.1 Input selector	30
4.3.2 Fourth-power I/Q calculators	32

4.3.3	Spread estimation	34
4.3.4	Sign estimation	36
4.3.5	Filters	38
4.3.6	Magnitude and Sign integration	39
4.4	Phase unwrapper	40
4.4.1	Functional operation	40
4.4.2	Data flow	40
4.4.3	Timing flow	41
4.5	Phase compensation block	41
4.5.1	Functional operation	41
4.5.2	Data flow	42
4.5.3	Timing flow	42
4.6	Top-design overview	43
4.6.1	Functional dataflow	43
4.6.2	Timing and alignment	44
5	Verification and simulation model	45
5.1	Overview	45
5.2	Test vector generation	46
5.2.1	Golden model structure	46
5.2.2	Block-level vectors	46
5.2.3	System-level cases	47
5.3	VHDL testbench setup	47
5.3.1	Block-level testbenches	47
5.3.2	Top-level wrapper testbench	48
5.4	Error analysis	48
5.4.1	Block-level analysis	48
5.4.2	System-level analysis	48
6	Results	51
6.1	Evaluation scope	51
6.2	Agreement with the reference model	52
6.2.1	SBPS	52
6.2.2	SE	53
6.2.3	Top-level agreement	54
6.3	Correction behaviour	56
6.4	Performance sweeps	57
6.4.1	Influence of SBPS refinement stages	57
6.4.2	Influence of compensation sections	58
6.4.3	Error localisation	58
6.4.4	Resource usage	60
6.4.5	Latency	61
7	Discussion	63
7.1	Limitations	63
7.2	Implementation Trade-Offs	64
7.3	Possible Improvements	64
7.4	Societal Responsibility	65
8	Conclusion	67
8.1	Summary	67
8.2	Achieved Objectives	67
8.3	Main Contributions	68
8.4	Future Work	68
	Bibliography	69

List of Figures

1.1	Qualitative transition from direct-detection to coherent techniques.	1
1.2	Position of the proposed carrier-recovery architecture within the full receiver DSP chain. .	3
2.1	Near-linear approximation of the total noise process	6
2.2	FFT-binsize leading to residual phase offset	7
2.3	Working BPS principle	8
3.1	Top architecture	9
3.2	Conceptual representation of staged SBPS refinement	11
3.3	Phase representation of staged SBPS refinement	11
3.4	Residual phase correction with SBPS only and with combined SBPS+SE correction . . .	13
3.5	16QAM symbols after fourth-power transformation	15
3.6	Hit probability versus #candidate symbols	16
3.7	Example of a phase unwrapping	17
3.8	Section-based phase compensation	18
4.1	Clipping causing false minima for SBPS metric	22
4.2	Clipping of 16QAM symbols under different SNR conditions	23
4.3	Data-representation throughout the top design	25
4.4	Functional diagram of the full SBPS architecture	25
4.5	Functional diagram of the SBPS stage	26
4.6	Functional diagram of the <i>rotate metric slice</i> unit	26
4.7	Datapath diagram of the full SBPS architecture	27
4.8	Datapath diagram of the SBPS stage	27
4.9	Datapath diagram of the <i>rotate metric slice</i> unit	28
4.10	Timing diagram of the SBPS stage	28
4.11	Timing diagram of the <i>rotate metric slice</i> unit	29
4.12	Top lay-out of SE block	30
4.13	Functional diagram of the SE input selector	30
4.14	Timing flow diagram of the SE input selector	31
4.15	Calculation-flow of the fourth-power module	33
4.16	Top overview of the spread estimation module	34
4.17	$\arccos(x)/4$ LUT symmetry	35
4.18	Top overview of the sign estimation module	36
4.19	Sign rolling-majority filter with circular buffer	39
4.20	Top overview of phase unwrapper block	40
4.21	Top overview of phase compensation block	41
4.22	Top overview implemented algorithm	43
5.1	Verification pipeline	45
5.2	Case lay-out	47
5.3	Result lay-out	49
6.1	Relative RTL-cosine error for phase slope	52
6.2	Relative RTL-sine error for phase slope	53
6.3	Relative RTL error for intra-block phase slope	53
6.4	Signed phase-error difference between Python reference model and RTL output over block index	55
6.5	Constellation diagram throughout the recovery pipeline	56
6.6	Effect of SBPS refinement depth on recovery performance	57
6.7	Effect of phase-compensation granularity on the recovery performance	58

6.8	Residual EVM after SBPS correction and SBPS + SE correction	59
6.9	Residual EVM after SBPS correction and SBPS + SE correction, 96 MHz CFO	60

List of Tables

4.1	Pipeline latency of the absolute spread-estimation calculation.	36
4.2	Pipeline latency of the sign-estimation calculation.	38
4.3	Pipeline latency before the CORDIC rotation bank in the PUPC backend.	42
4.4	Main data streams at the top-design boundary.	44
6.1	Overview of the simulation defaults used for the system-level results	51
6.2	Python-to-RTL phase-error differences for the top-level receiver, $n = 99$ blocks	55
6.3	FPGA resource usage for the evaluated receiver configurations	60
6.4	Post-implementation timing-closure summary for the clock sweep	61
6.5	Pipeline-latency summary for the implemented receiver	62

List of Acronyms

IMDD	intensity modulation and direct detection
PAM4	pulse amplitude modulation 4-level
QPSK	quadrature phase shift keying
QAM	quadrature amplitude modulation
16QAM	16-state quadrature amplitude modulation
ASIC	application specific integrated circuit
DSP	digital signal processing
BPS	blind phase search
SBPS	serialised binary phase search
SE	spread estimation
PU	phase unwrapper
PC	phase compensator
FPGA	field programmable gate array
CORDIC	coordinate rotation digital computer
IP	intellectual property
VHDL	VHSIC Hardware Description Language
RTL	register-transfer level
SNR	signal-to-noise ratio
BER	bit error rate
SER	symbol error rate
EVM	error vector magnitude
LSB	least significant bit
LUT	look-up table
CFO	carrier frequency offset
VHSIC	very high speed integrated circuit
LO	local oscillator
ASE	amplified spontaneous emission
FFT	fast fourier-transform
MUX	multiplexer
I/Q	in-phase/quadrature
AWGN	additive white Gaussian noise
DUT	design under test
LSB	least significant bit

This page is intentionally left blank.

Introduction

1.1 Context and motivation

The rapid growth of cloud computing, large-scale streaming platforms and AI services has sharply increased the amount of data exchanged inside modern datacenters. In particular, AI training and inference workloads rely on dense east-west traffic between accelerators, servers and switching equipment, which places stringent demands on the internal interconnect fabric [1, 2]. As a result, the optical links between servers, racks and switches are becoming a critical bottleneck for both throughput and energy efficiency.

To support this growth, datacenter optical interconnects are evolving toward 800 Gbit/s and 1.6 Tbit/s transceivers [3, 4]. The challenge is not only to increase the raw data rate, but to do so while keeping power consumption, latency and implementation cost within practical limits.

Today, most short-reach intra-datacenter optical links rely on intensity modulation and direct detection (IMDD), often using pulse amplitude modulation 4-level (PAM4) signalling, because these links offer relatively low implementation complexity and good power efficiency over short reaches [4, 2]. However, as symbol rates and transmission distances continue to increase, IMDD systems approach their limits in terms of chromatic-dispersion tolerance, achievable spectral efficiency and receiver sensitivity. Figure 1.1 qualitatively illustrates how the transition from direct-detection to coherent techniques moves toward shorter reaches as the target data rate increases [4, 5].

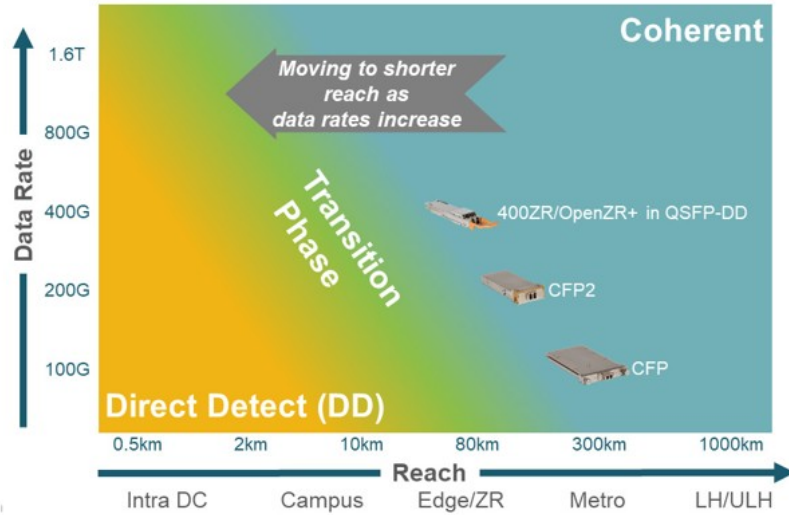


Figure 1.1: Qualitative transition from direct-detection to coherent techniques.

Coherent optical communication offers an attractive alternative because it modulates and detects the complex optical field rather than only the optical intensity. In IMDD, square-law photodetection makes the effective channel inherently nonlinear, whereas coherent reception preserves a linear field-domain model for the dominant propagation effects. As a result, linear impairments such as chromatic dispersion and polarization-related distortions can be compensated efficiently in the digital domain [6, 7,

8]. Coherent systems also enable higher-order modulation formats such as 16QAM, improving spectral efficiency [9, 4]. Experimental transceiver-level comparisons for intra-datacenter links have shown that coherent solutions can provide a larger optical power budget while maintaining comparable application specific integrated circuit (ASIC) power consumption, although that comparison does not isolate the contribution of individual DSP blocks such as carrier recovery [10].

1.2 Problem statement

Although coherent optical communication provides important advantages in terms of spectral efficiency and receiver sensitivity, these benefits come at the cost of increased receiver complexity. In particular, the DSP chain of a coherent receiver contains computationally expensive blocks for frequency recovery and carrier phase recovery.

Carrier recovery compensates the residual frequency offset and phase noise caused by the finite linewidth of the transmitter and local-oscillator lasers. In conventional coherent receivers, this is often split into a coarse frequency-estimation stage, for example using fourth-power spectral estimation, followed by a carrier-phase estimator such as blind phase search (BPS) [11, 12, 13]. The fourth-power stage exposes a carrier-related spectral tone that enables a coarse estimate of the residual frequency offset, after which the phase-recovery stage compensates the remaining constellation rotation and phase noise.

At first sight, a higher baud rate could appear to simplify phase recovery, since the symbol period becomes shorter and the laser-induced phase-noise increment between adjacent symbols decreases for a fixed linewidth. In practice, however, the implementation challenge remains severe. High-baudrate coherent receivers must process more symbols per second and typically rely on strong parallelism, while coarse frequency recovery still requires sufficient FFT resolution and BPS still requires multiple candidate rotations and metric evaluations per block. Consequently, carrier recovery remains a significant contributor to coherent DSP complexity and power consumption in feed-forward hardware implementations [8, 13].

These complexity challenges are particularly important in intra-datacenter links, where power consumption and latency are critical constraints and where hardware-efficient, feed-forward carrier recovery is preferred [13, 5]. Moreover, block-based phase recovery introduces an additional trade-off: larger blocks improve noise averaging and expose more hardware parallelism, but they also suffer more from intra-block phase drift and from decision errors in square-QAM constellations. It is therefore not sufficient for a carrier-recovery algorithm to be accurate alone; it must also be implementable efficiently in hardware.

This thesis therefore investigates a low-complexity FPGA-based carrier recovery architecture for baudrate-sampled 16QAM coherent intra-datacenter links. The proposed approach reduces the cost of conventional blind phase search by replacing a wide parallel test-phase evaluation with a serialized binary refinement. Because a single blockwise phase estimate is not sufficient when residual frequency offset causes noticeable phase evolution across the block, this serialized blind phase search is augmented with a phase spread estimation block. Together, these blocks aim to improve the effective frequency-tracking range of the receiver while keeping the overall hardware cost low.

1.3 Objectives of this thesis

A mathematical model of the proposed low-complexity carrier phase-recovery system has already been developed in Simulink and will be discussed in more detail in Chapter 3. In this thesis, the implementation of this phase-recovery architecture on an FPGA is presented in detail.

The proposed architecture forms the final stage of the receiver DSP chain, directly before the symbol decision stage, as shown in Figure 1.2.

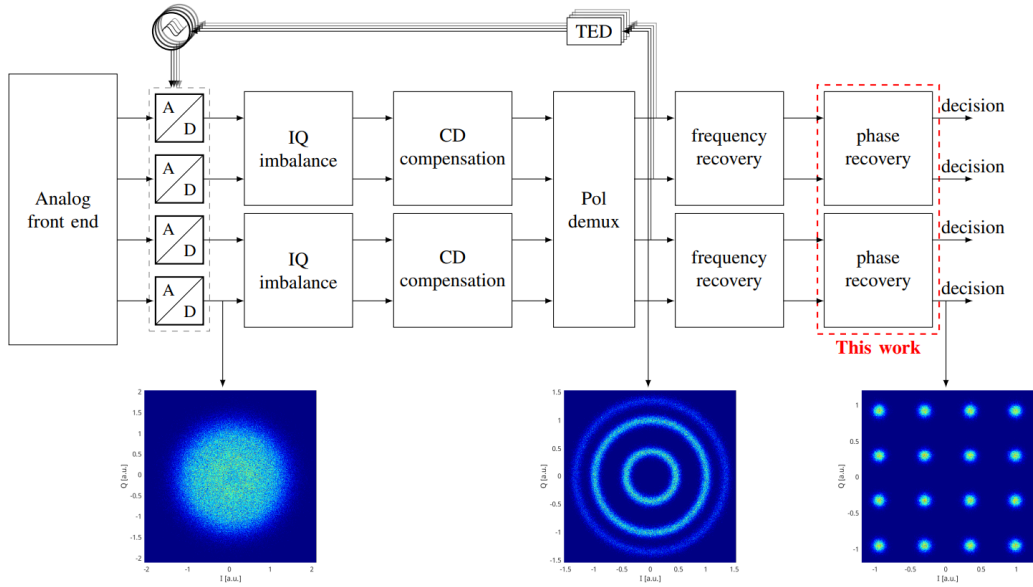


Figure 1.2: Position of the proposed carrier-recovery architecture within the full receiver DSP chain.

Because carrier recovery is the last impairment-compensation stage before slicing, residual phase errors directly translate into symbol-decision errors. The implementation therefore focuses not only on functional correctness, but also on efficient hardware mapping, pipelining and memory usage.

Where multiple implementation strategies are possible, the motivation behind the chosen design decisions is discussed. Alternative approaches are also considered, together with the reasons why they were not selected in the final design.

Finally, the performance of the implemented architecture is evaluated. The proposed carrier-recovery system contains several configurable parameters, such as the number of blind-phase-search stages, block size and phase-correction segments. Increasing these parameters can improve phase-estimation accuracy and cycle-slip tolerance, but also increases the required hardware resources, latency and power consumption.

The objective of this evaluation is therefore not only to determine the absolute performance of the design, but also to investigate the trade-off between hardware cost and receiver performance under different operating conditions. These conditions include different signal-to-noise ratios, residual frequency offsets and laser linewidths, as these are the dominant impairments affecting coherent carrier recovery.

1.4 Contributions

This thesis contributes a complete fixed-point RTL implementation of the proposed SBPS+spread estimation (SE) carrier-recovery backend for baudrate-sampled 16QAM coherent links. The work goes beyond modelling the algorithm by translating the centre-phase estimator, spread estimator, phase unwrapper and phase compensator into a hardware-oriented processing chain with explicit numerical formats, pipeline stages and valid-signal alignment.

A central contribution is the hardware realization of the spread-estimation path. This includes candidate-symbol selection, fourth-power processing, normalized dot-product evaluation, sign estimation, filtering and LUT-based angle recovery. The thesis also contributes the top-level integration of this spread estimate with the SBPS centre-phase estimate and the delayed raw I/Q block, so that the final phase-compensation stage operates on mutually aligned phase and sample data.

Finally, the work contributes a reproducible verification and evaluation flow. Python reference models, generated fixed-point vectors, VHDL testbenches and post-processing scripts are used to validate the fixed-point implementation and to compare it with the intended algorithmic behaviour. The resulting performance and implementation analysis identifies useful operating regions for the SBPS stage count and compensation-section count, and relates these choices to latency, timing closure and hardware-resource

usage.

1.5 Thesis outline

The remainder of this thesis is structured as follows:

Chapter 2 introduces the theoretical background required for this work. First, coherent optical communication systems are discussed, with a focus on how phase noise and residual frequency offset arise in these systems, how they distort the received signal, and how they translate into carrier-recovery requirements. Next, existing carrier-recovery techniques are reviewed together with their limitations and with the motivation for the proposed approach. The emphasis is placed on blind, feed-forward methods, since these are the most relevant candidates for the targeted hardware implementation. The digital representation of 16QAM signals is then discussed, because the proposed architecture strongly depends on fixed-point symbol scaling, numerical precision and signal-to-noise ratio (SNR)-related considerations. Finally, a section is dedicated to the coordinate rotation digital computer (CORDIC) algorithm and the existing intellectual property (IP) core used in this work [14].

Chapter 3 presents the proposed phase-recovery algorithm from a theoretical perspective. Building on the concepts introduced in the previous chapter, the complete architecture is divided into its main functional blocks, including the SBPS, SE, phase unwrapper (PU) and phase compensator (PC) modules. For each subblock, the operating principle, mathematical model and expected contribution to the overall system are discussed. The theoretical performance of the proposed method is also analysed based on the work of ir. J. De Busscher and co-authors [15].

Chapter 4 describes how the theoretical carrier-recovery system is translated into an FPGA implementation. This chapter discusses the practical challenges associated with such an implementation, including fixed-point numerical representations, timing closure and hierarchical design. In addition, it explains how the individual subblocks are combined into a complete processing chain and how dataflow, valid signals and latency alignment are handled throughout the design.

Chapter 5 explains the simulation and verification framework used for the implemented design. The testbenches of the different subblocks are discussed together with their corresponding results. The purpose of this chapter is to show that the hardware implementation behaves in accordance with the theoretical model and to explain any deviations between both. The focus is therefore on functional correctness rather than on overall system performance under different operating conditions. Comparisons between the FPGA implementation and the Python or Simulink reference models are also included to demonstrate that the implemented fixed-point design accurately reproduces the expected behaviour.

Chapter 6 evaluates the overall performance of the implemented carrier-recovery system. Separate sections discuss the performance of the SBPS and SE blocks and identify the conditions under which their performance begins to degrade. The influence of different system parameters, such as block size, number of search stages and datapath bit lengths, is also investigated. In addition, this chapter discusses the hardware resource usage and latency of the final design. The relation between implementation complexity and system performance is analysed in order to determine which parameters offer the most favourable trade-off.

Chapter 7 discusses the limitations of the current implementation and identifies possible improvements for future work. This includes both algorithmic improvements and hardware-related optimizations. Potential extensions such as more advanced LUT implementations, adaptive parameter selection, improved symbol-selection strategies and support for higher-order modulation formats are considered. In addition, the discussion reflects on the assumptions made throughout the work, such as the assumption that the residual frequency offset changes only slowly between neighbouring blocks. The chapter also provides a broader overview of the main design trade-offs and explains how these trade-offs influenced the final architecture.

Finally, Chapter 8 summarizes the most important aspects of the work. It revisits the original objectives of the thesis, discusses the extent to which these objectives were achieved and reflects on the main findings of the implementation and evaluation. The final chapter also outlines possible directions for future work, including integration into a complete coherent receiver chain and further optimization of the architecture for practical deployment.

Background

This chapter summarises the background required to contextualise the proposed coherent receiver carrier-recovery architecture. It focuses on (i) short-reach IM/DD links versus coherent detection, (ii) phase-related impairments (carrier frequency offset and laser phase noise), and (iii) representative DSP-based carrier recovery methods used in coherent receivers

2.1 Coherent optical communication systems

Traditional short-reach optical interconnects in data centres have often favoured intensity-modulation/direct-detection (IM/DD) transceivers because they enable simple and power-efficient receiver front-ends. In IM/DD systems, information is conveyed by modulating the received optical intensity (or optical power). At the receiver, a photodiode converts incident optical power into an electrical photocurrent (square-law photodetection), so the receiver directly observes intensity but does not directly access the optical carrier phase [16, 6].

IM/DD links are attractive for short distances and moderate data rates; however, as symbol rates increase, fibre chromatic dispersion becomes increasingly detrimental because group delay varies with optical wavelength, causing waveform distortion and frequency-selective fading for intensity-modulated signals. In addition, the achievable spectral efficiency (information rate per unit bandwidth, in b/s/Hz) is fundamentally constrained when only intensity is detected, because direct detection effectively exploits fewer degrees of freedom than coherent detection [6].

Coherent optical communication systems overcome these limitations by detecting both the amplitude and phase of the optical field and enabling advanced modulation formats such as QPSK and square M-QAM. Coherent detection also supports digital compensation of linear channel impairments (e.g., chromatic dispersion and polarization effects) in the electrical domain [6, 7, 8]. Recent work on data-centre optics evaluates when coherent detection becomes favourable relative to IM/DD as per-lane rates scale, highlighting the trade-off between coherent DSP power/complexity and improved spectral efficiency and impairment tolerance [4].

A typical intradyne coherent receiver combines the received optical field with a local oscillator (LO) laser in a 90° optical hybrid. The hybrid generates orthogonal interference terms that are detected by balanced photodiode pairs, yielding electrical in-phase (I) and quadrature (Q) components. Balanced detection improves sensitivity to the desired beating term and suppresses common-mode intensity noise [7, 9].

After analog-to-digital conversion, the coherent receiver typically performs several DSP operations, including chromatic dispersion compensation, timing recovery, adaptive equalization, carrier frequency recovery and carrier phase recovery [7, 8]. In many DSP chains, dispersion compensation and polarization demultiplexing/equalization are performed prior to carrier phase estimation because commonly used blind equalizers can converge without an explicit carrier-phase reference; in practice, frequency-offset estimation may be placed either before or after equalization depending on acquisition range and implementation constraints [7, 8].

Historically, optical coherent receivers often relied on analogue phase-locked loops (PLLs) for carrier synchronisation. In modern DSP-based coherent receivers, carrier recovery is commonly implemented digitally using feed-forward and/or decision-directed algorithms, which avoids tight analogue loop dynamics and integrates naturally with polarization-diverse digital processing [7, 12, 13].

2.2 Phase noise and frequency offset

The main phase-related impairments in coherent optical communication systems are carrier frequency offset (CFO) and phase noise. Both arise because the transmitter and local oscillator lasers are independent devices and therefore do not share an identical optical frequency or phase reference [9].

Carrier frequency offset occurs because the instantaneous optical frequency of the transmitter laser differs from that of the LO laser. After coherent detection and digitisation, CFO appears as an effective baseband frequency offset and produces a deterministic phase ramp across samples [9]. Mathematically, this phase rotation can be expressed as

$$y[k] = s[k]e^{j(\theta[k] + 2\pi\Delta f T_s k)} + \eta[k] \quad (2.1)$$

where $s[k]$ is the transmitted symbol, $\theta[k]$ is the phase noise, Δf is the carrier frequency offset, T_s is the symbol period and $\eta[k]$ is additive noise [9, 17].

The term $2\pi\Delta f T_s k$ corresponds to a linear phase ramp. A larger frequency offset or a larger symbol duration leads to faster constellation rotation. This is particularly relevant for block-based carrier recovery systems, because the total accumulated phase change within one block influences the probability of phase ambiguities and cycle slips [13].

Besides frequency offset, coherent optical systems are also affected by phase noise. Laser phase noise is commonly modelled as a discrete-time Wiener process (random walk) [17]:

$$\theta[k] = \theta[k-1] + \Delta\theta[k] \quad (2.2)$$

where $\Delta\theta[k]$ is a zero-mean Gaussian increment whose variance is proportional to the combined (beat) linewidth and the sampling interval [17, 13].

For the systems considered in this thesis, the residual CFO after coarse frequency recovery is especially important. While a separate frequency recovery stage can compensate most of the CFO, a small leftover frequency error remains due to the finite resolution of practical estimators [11, 18]. This residual offset causes a nearly linear phase drift across each DSP block.

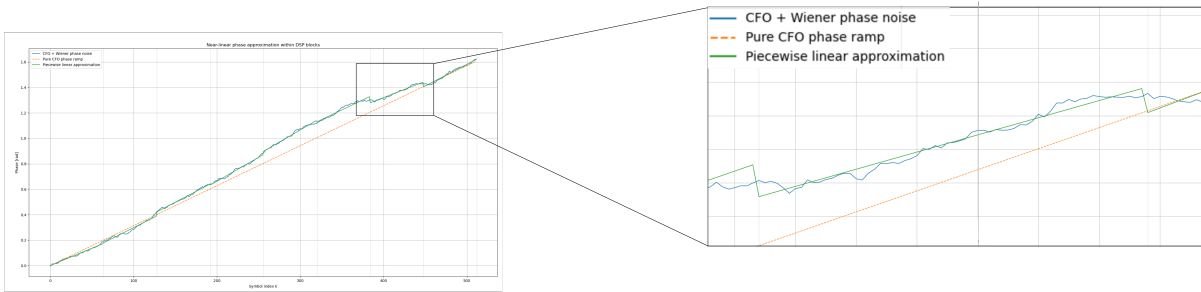


Figure 2.1: Near-linear approximation of the total noise process

The phase recovery system must therefore not only compensate random phase noise caused by laser linewidth, but also account for remaining within-block phase slope. This motivates the channel model used throughout this thesis, in which the received symbols are impaired by:

- additive white Gaussian noise,
- a residual carrier frequency offset,
- Wiener phase noise caused by laser linewidth.

A coarse FFT-based frequency recovery algorithm is often used before phase recovery. FFT-based CFO estimation provides wide acquisition range at modest complexity [11, 18]. For M-PSK constellations, an M-th power nonlinearity can suppress the data modulation and expose a strong carrier-related spectral line that enables peak-based CFO estimation; related techniques are also adapted for square QAM via partitioning, pilots, or data-aided processing [19].

The estimated frequency offset can then be compensated digitally. However, the accuracy of this estimate depends on the FFT size. A finite FFT resolution leads to a remaining frequency estimation

error. Since the FFT bin spacing equals F_s/N_{FFT} , a simple bin-centred peak estimate implies an error on the order of half a frequency bin:

$$\Delta f_e \leq \frac{F_s}{2N_{\text{FFT}}} \quad (2.3)$$

[20, 21].

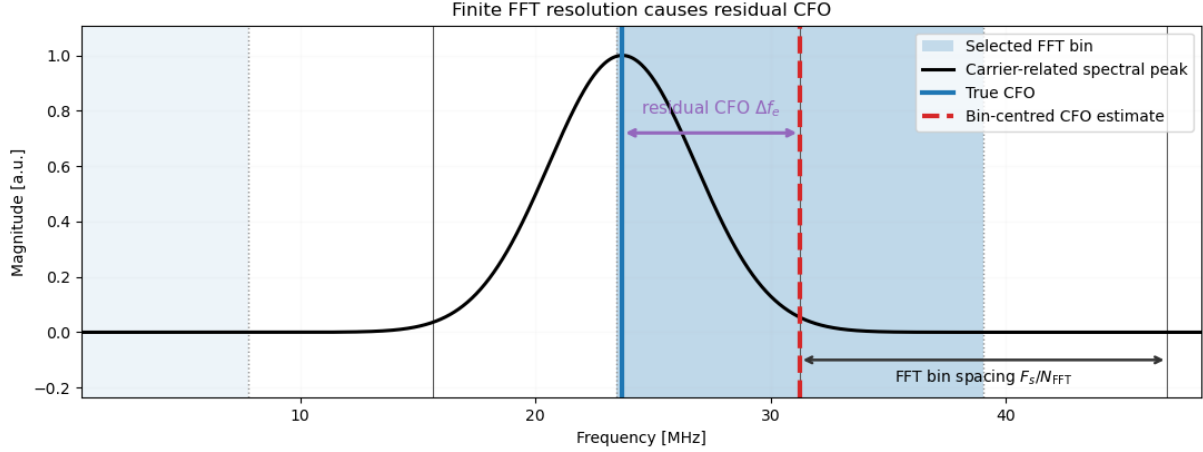


Figure 2.2: FFT-binsize leading to residual phase offset

This remaining frequency error is then passed on to the phase recovery stage. If the residual offset is too large, the phase recovery block may no longer be able to track the constellation rotation correctly, which increases the probability of cycle slips [13, 8].

2.3 Carrier recovery in DSP chains

Carrier recovery is the DSP stage responsible for compensating the phase rotations introduced by residual CFO and phase noise. Since these impairments have different characteristics, coherent receivers commonly separate carrier recovery into (i) frequency offset estimation/compensation and (ii) carrier phase estimation [7, 8].

Frequency recovery estimates a single CFO parameter over an observation interval and therefore benefits from aggregating many samples to improve estimator variance; it is not typically meaningful to produce an independent CFO estimate per symbol. By contrast, carrier phase recovery must track a time-varying phase process (Wiener phase noise plus residual CFO), motivating symbol-rate or block-rate phase estimation [11, 13].

A major challenge is that modern coherent DSP systems process many symbols in parallel in order to reduce clock speed requirements. As a result, carrier phase is often estimated only once per processing block instead of once per symbol [8]. Although this significantly reduces computational complexity, it also increases sensitivity to phase drift within the block. If the total phase change over a block becomes too large, the phase recovery algorithm may unwrap to an incorrect $\pi/2$ -shifted hypothesis (cycle slip) for square QAM constellations [13].

If a block contains B symbols and the residual frequency offset is Δf_e , the total phase drift across the block can be approximated by

$$\Delta\phi_{\text{block}} \approx 2\pi\Delta f_e B T_s \quad (2.4)$$

To reduce the probability of cycle slips, a practical design rule is to keep the intra-block phase drift well below the midpoint between neighbouring $\pi/2$ -ambiguous hypotheses, often expressed as $|\Delta\phi_{\text{block}}| \lesssim \pi/4$ [13].

2.4 Blind Phase Search

Blind Phase Search (BPS) is a widely used feed-forward carrier phase estimation method for coherent square M-QAM. Its advantages include pilot-free operation, modulation-format compatibility, and strong

tolerance to laser phase noise when sufficient test phases and averaging are used [13, 22].

The principle of BPS is straightforward. A block of received symbols is rotated over a set of candidate test angles. For each candidate angle, the rotated symbols are sliced to the nearest ideal constellation points and an error metric is computed. The candidate angle that minimises the total error is selected as the estimated carrier phase [13].

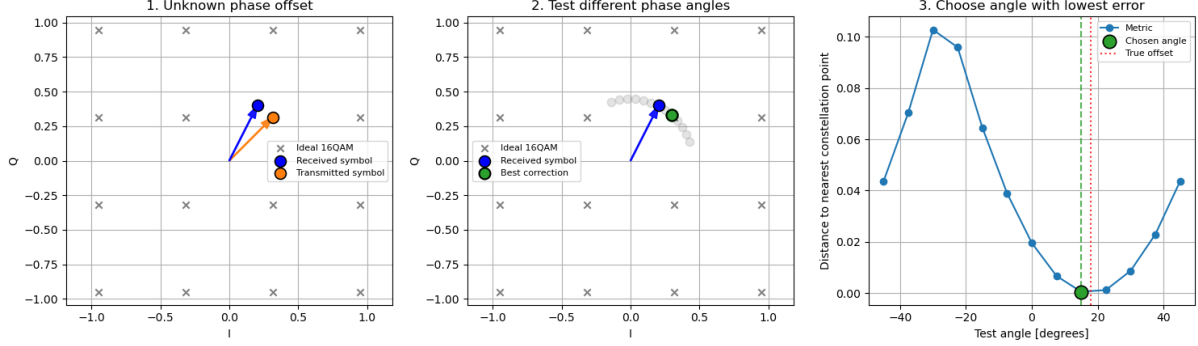


Figure 2.3: Working BPS principle

For a received block $z[k]$, the BPS metric for a test phase ϕ can be written as

$$e(\phi) = \sum_{k=0}^{B-1} |D(z[k]e^{-j\phi}) - z[k]e^{-j\phi}|^2 \quad (2.5)$$

where $D(\cdot)$ denotes the decision operation that maps a rotated symbol to its nearest constellation point. The phase estimate is then given by the test angle that minimises $e(\phi)$.

For square QAM constellations, the objective function in BPS is periodic with period $\pi/2$ due to constellation rotational symmetry. Therefore, it suffices to search any interval of length $\pi/2$; a common centred choice is $[-\pi/4, \pi/4]$ [13]. However, a large number of test phases is usually required to obtain sufficient estimation accuracy, making BPS computationally expensive in high-throughput FPGA implementations [13, 22].

Another limitation of blockwise BPS is that it estimates one average phase per block. When residual CFO induces significant intra-block phase drift, a single blockwise estimate can be insufficient and may increase cycle-slip probability [13]. This thesis therefore extends the traditional blockwise BPS approach by also estimating the phase spread across the block, as explained in Chapter 3.

2.5 CORDIC

The Coordinate Rotation Digital Computer (CORDIC) algorithm implements vector rotations and elementary functions using iterative shift-add operations. Its attractiveness in FPGA/ASIC implementations stems from replacing general multipliers with pipelinable add/subtract and bit-shift datapaths [23, 24].

In the rotation mode of CORDIC, a vector is iteratively rotated by a sequence of predefined micro-rotations. The iterative equations can be written as [23, 24]:

$$x_{i+1} = x_i - d_i y_i 2^{-i} \quad (2.6)$$

$$y_{i+1} = y_i + d_i x_i 2^{-i} \quad (2.7)$$

$$z_{i+1} = z_i - d_i \arctan(2^{-i}) \quad (2.8)$$

where $d_i \in \{-1, +1\}$ determines the direction of rotation at iteration i . After a sufficient number of iterations, the vector is rotated by the desired angle with good precision [24].

In this thesis, CORDIC is used to rotate received symbols by the estimated carrier phase. The implementation leverages an FPGA IP core; design effort therefore focuses on selecting fixed-point formats, pipeline depth/latency, and interface timing to meet throughput and numerical-accuracy requirements [14].

Proposed phase recovery architecture

3.1 System overview

As discussed in Chapter 2, coherent optical receivers are affected by both residual carrier frequency offset (CFO) and laser phase noise. A coarse frequency recovery stage removes the majority of the frequency offset, but practical estimators always leave a remaining residual error because of their finite frequency resolution. This residual CFO introduces a deterministic phase rotation across consecutive symbols, while the finite linewidth of the transmitter and local oscillator lasers adds a stochastic phase perturbation on top of this process.

In conventional feed-forward carrier recovery systems, a single carrier phase estimate is often calculated over a block of symbols. This significantly reduces computational complexity compared to per-symbol phase estimation, which is especially important in high-throughput coherent receivers where many symbols are processed in parallel. However, this also introduces an important limitation: the carrier phase is generally not constant over the duration of the processing block.

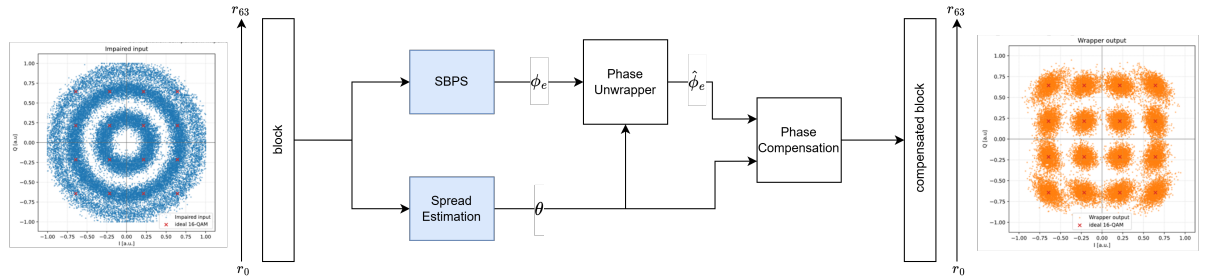


Figure 3.1: Top architecture

In the presence of residual CFO, the phase evolves approximately linearly with symbol index, while laser phase noise introduces additional random deviations. As a result, a single phase estimate can become insufficient for accurately compensating all symbols inside the block. Symbols located further away from the phase estimation point accumulate additional residual phase error.

The proposed architecture addresses this problem through a two-part carrier recovery structure consisting of:

1. a Serialized Blind Phase Search (SBPS) block,
2. and a Spread Estimation (SE) block.

The SBPS estimates the carrier phase near the middle of the processing block and acts as the central phase reference of the architecture. The SE block then estimates how the phase evolves around this mid-point estimate. Rather than estimating a separate phase for every symbol, the architecture reconstructs the intra-block phase evolution from:

- one centre-phase estimate,
- and one estimate of the total phase spread across the block.

This approach is based on the observation that, under practical coherent datacenter operating conditions, the residual CFO after coarse frequency recovery dominates the short-term phase evolution within one processing block. The accumulated linewidth-induced phase variation over a single block is typically much smaller than the deterministic phase drift caused by the residual frequency offset:

$$\sum_{i=0}^{B-1} \phi_i \ll 2\pi\Delta f T_s B \quad (3.1)$$

with B the block size, ϕ_i the linewidth-induced phase perturbation of symbol i , Δf the residual carrier frequency offset and T_s the symbol period.

Under this assumption, the phase evolution across the block may be approximated as locally linear. The carrier phase inside the block can therefore be reconstructed from only:

1. the carrier phase near the middle of the block,
2. and the total phase spread across the block.

Figure 3.4 illustrates the effect that motivates this additional spread estimate. Although the middle of the block can be compensated correctly by the SBPS estimate, symbols towards the block edges still exhibit residual constellation rotation when the intra-block phase evolution is ignored.

The estimated spread is subsequently used in two ways:

1. to improve phase unwrapping between neighbouring blocks,
2. and to reconstruct multiple phase corrections within the block during phase compensation.

As a result, the proposed architecture increases the effective frequency tracking range of the carrier recovery system while maintaining substantially lower complexity than conventional high-resolution BPS approaches.

3.2 Serialized binary phase search

In a traditional BPS implementation, many candidate phases are evaluated simultaneously over the full search interval. For each candidate phase, all symbols inside the processing window are rotated, sliced to the nearest constellation point and evaluated through a decision-directed metric. The candidate yielding the lowest metric is selected as the estimated carrier phase.

For square quadrature amplitude modulation (QAM) constellations, the search interval may be limited to:

$$\left[-\frac{\pi}{4}, \frac{\pi}{4}\right)$$

because the BPS metric is inherently $\pi/2$ -periodic due to the rotational symmetry of square constellations.

The key idea of SBPS is that the optimal phase estimate is approximated through successive refinement stages. Instead of evaluating many test phases simultaneously, only two candidate rotations are evaluated at each stage:

- one positive correction,
- and one negative correction,

relative to the previously selected estimate.

The candidate producing the lower metric is retained, after which a more refined correction is evaluated during the next stage.

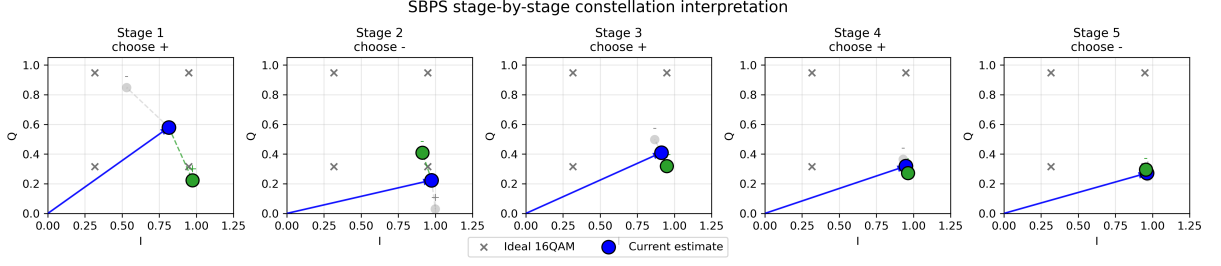


Figure 3.2: Conceptual representation of staged SBPS refinement

The proposed implementation adopts a binary refinement structure. The first stage evaluates the hypotheses:

$$\phi_1 \in \left\{ -\frac{\pi}{8}, +\frac{\pi}{8} \right\}$$

while stage m refines the estimate through:

$$\hat{\phi}_m = \hat{\phi}_{m-1} \pm \frac{\pi}{2^{m+2}} \quad (3.2)$$

Each stage therefore halves the remaining search uncertainty.

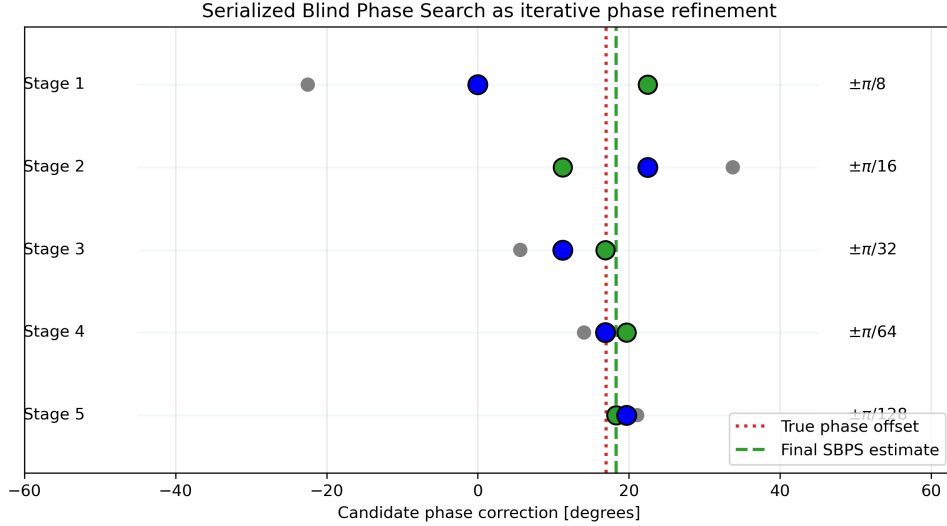


Figure 3.3: Phase representation of staged SBPS refinement

This procedure is mathematically equivalent to a binary search over the phase interval and yields an exponentially decreasing phase quantization error as the number of stages increases. The referenced work shows that the worst-case estimation error decreases exponentially with each added stage, while the total number of required candidate rotations only increases linearly with the number of stages [15].

For each candidate phase ϕ , a decision-directed metric is computed:

$$J(\phi) = \sum_{k \in \mathcal{W}} |D(r_k e^{-j\phi}) - r_k e^{-j\phi}|^2 \quad (3.3)$$

with:

- r_k the received symbol,
- $D(\cdot)$ the slicer operation,
- and $\mathcal{W}$ the metric accumulation window.

The candidate phase producing the smallest metric is assumed to correspond most closely to the true carrier phase. In this work, the resulting estimate is interpreted as the carrier phase near the middle of the processing block $\hat{\phi}_e$.

This estimate is subsequently forwarded to the phase unwrapping and phase compensation stages. Because only one phase estimate is produced per block, and because the BPS metric is $\pi/2$ -ambiguous for square QAM constellations, additional processing is required to track phase continuity between successive blocks. This task is handled by the phase unwrapping stage, while the SE block reconstructs the phase evolution inside each block.

3.2.1 Pattern effect

The decision metric of Equation 3.3 assumes that, after the correct test rotation, the slicer decision $D(\cdot)$ gives a useful reference for every symbol in the window. For square 16QAM this is not always true. The constellation contains inner symbols $(\pm s, \pm s)$, outer corner symbols $(\pm 3s, \pm 3s)$, and middle-ring symbols $(\pm s, \pm 3s)$ or $(\pm 3s, \pm s)$. The middle-ring symbols are close to both an inner/outer axis decision boundary and an angular decision boundary. When the test phase is still coarse, a middle-ring symbol can therefore be sliced to a neighbouring point in a way that produces a small Euclidean error for an incorrect candidate phase.

This behaviour is referred to as the pattern effect in the BPS literature. Li et al. explicitly identify it as a design constraint for reduced-complexity two-stage BPS estimators and propose parameter choices that reduce complexity while mitigating the pattern effect [25]. The problem is not that a single middle-ring symbol is always wrong. Rather, the accumulated metric can be biased when the block contains an unfavourable pattern of such symbols. In that case the metric curve can contain a false local minimum, so the binary stage may choose the wrong branch even though the correct phase is still inside the search interval.

The proposed SBPS implementation therefore uses radius-directed decisions in the first refinement stages. The purpose is to make the early metric less sensitive to ambiguous middle-ring decisions. Instead of allowing all 16QAM decision regions to contribute equally, the early stages use a simplified decision rule that keeps the reliable diagonal symbol classes: inner symbols remain mapped to $(\pm s, \pm s)$, while samples outside a central box are treated as corner-like symbols and mapped to $(\pm 3s, \pm 3s)$. This removes the explicit middle-ring decision from the coarse stages and makes the metric behave more like a quadrature phase shift keying (QPSK)-partitioned phase metric.

This radius-directed mode is only used for the first N_{RD} stages. These are the stages where the candidate phase spacing is largest and where the pattern effect is most likely to affect the branch decision. After the estimate has been refined, the remaining phase uncertainty is smaller, so the later stages return to the normal 16QAM slicer. The final stages can then use the full constellation geometry to reduce quantisation error without reintroducing the same level of coarse-stage false-minimum risk.

3.2.2 Block size consideration

This section will cover how the block size directly influences the effectiveness of the SBPS estimate. Coarse frequency recovery stage has a finite fast fourier-transform (FFT)-bin spacing, which determines the maximum residual CFO after frequency recovery. Using the half-bin error bound from Equation 2.3, the residual frequency error satisfies:

$$\Delta f_e \leq \frac{F_s}{2N_{FFT}} \quad (3.4)$$

This residual frequency error causes an accumulated phase drift across one SBPS processing block of B symbols:

$$\Delta \phi_{\text{block}} \approx 2\pi \Delta f_e B T_s \quad (3.5)$$

Since $T_s = 1/F_s$, substituting the worst-case residual CFO gives:

$$\Delta \phi_{\text{block,max}} = 2\pi \left(\frac{F_s}{2N_{FFT}} \right) B \left(\frac{1}{F_s} \right) = \pi \frac{B}{N_{FFT}} \quad (3.6)$$

The worst-case accumulated phase drift over one block therefore depends on the ratio between the processing block size and the FFT size, and not directly on the sampling frequency. For an unwrapping threshold of $\pi/4$, the phase drift over one block should satisfy:

$$\Delta\phi_{\text{block,max}} \leq \frac{\pi}{4} \quad (3.7)$$

which yields the design condition:

$$N_{\text{FFT}} > 4B \quad (3.8)$$

For example, a block size of $B = 64$ requires at least $N_{\text{FFT}} > 256$ to avoid additional cycle slips from the worst-case coarse frequency recovery error. This is only a theoretical minimum. In practice, a margin is required because linewidth-induced phase noise, additive noise and estimation errors also affect the phase trajectory.

The block size also creates a trade-off in estimation quality. A larger metric accumulation window improves robustness against additive noise because random perturbations are averaged over more samples [26]. At the same time, a larger block spans a longer time interval, so the carrier phase variation caused by residual CFO and laser phase noise becomes larger within the block. This trade-off is one of the main motivations for combining a middle-block SBPS estimate with a separate SE block.

3.3 Spread estimation algorithm

As discussed in Sections 2.2 and 2.3, the residual impairments after coarse frequency recovery consist of both stochastic laser phase noise and a remaining carrier frequency offset. While the linewidth-induced phase noise behaves as a random Wiener process, the remaining frequency offset introduces an approximately linear phase ramp across a processing block, as illustrated in Figure 2.1.

The proposed SBPS architecture estimates only a single carrier phase per block, namely the phase located near the middle of the block. This estimate is sufficiently accurate to serve as a robust phase reference for the block as a whole, but it does not describe how the phase evolves within the block itself. As a result, directly compensating all symbols using only the middle-block phase estimate still leaves a residual phase error on symbols located further away from the block centre.

This effect becomes increasingly important when the residual CFO after frequency recovery becomes larger, when the processing block size increases or when the laser linewidth introduces additional phase drift.

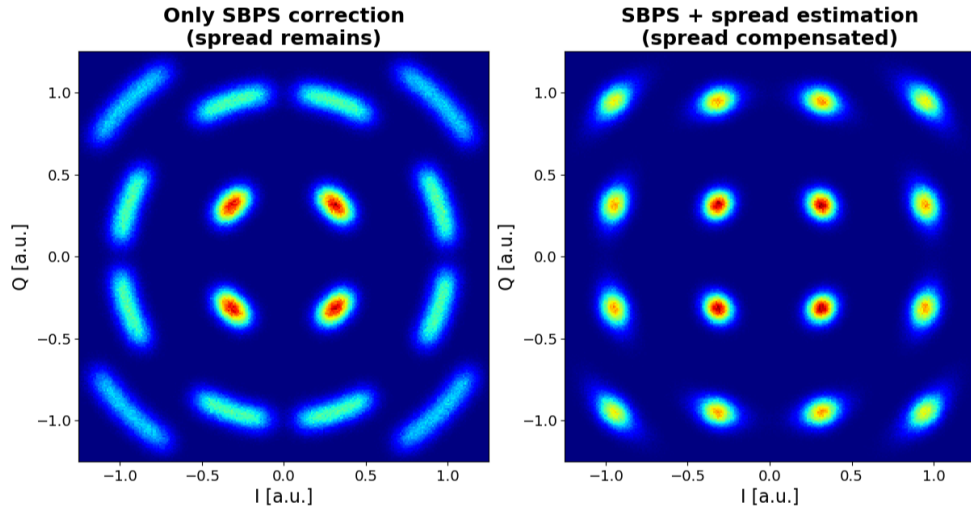


Figure 3.4: Residual phase correction with SBPS only and with combined SBPS+SE correction

The SE block is introduced to estimate this residual phase evolution across the block. Rather than estimating a separate carrier phase for every symbol, which would significantly increase computational complexity, the proposed method estimates the total phase spread over the block and reconstructs the intermediate phase trajectory from this information.

The working principle is easiest to see for a QPSK-like constellation. A received symbol can be written as the product of a transmitted constellation point and the unknown carrier phase rotation:

$$r_k = s_k e^{j\phi_k}. \quad (3.9)$$

For QPSK, the transmitted symbol has a constant magnitude and one of four phases separated by $\pi/2$:

$$s_k = A e^{j(\alpha + n_k \pi/2)}, \quad n_k \in \{0, 1, 2, 3\}, \quad (3.10)$$

where α is a fixed constellation rotation. Raising the received symbol to the fourth power gives:

$$r_k^4 = A^4 e^{j4\alpha} e^{j2\pi n_k} e^{j4\phi_k} = A^4 e^{j4\alpha} e^{j4\phi_k}. \quad (3.11)$$

The symbol-dependent phase term has disappeared because $e^{j2\pi n_k} = 1$. The fourth-power signal still contains a constant constellation factor $A^4 e^{j4\alpha}$, but this factor is the same for all QPSK symbols and therefore cancels in the normalized comparisons used below. The phase that remains between two transformed samples is four times the difference in carrier phase. In this ideal case, the fourth-power operation has removed the modulation phase ambiguity while preserving the carrier-related phase evolution.

The spread estimator uses this principle on selected 16QAM symbols. This requires an additional selection step because square 16QAM is not a constant-modulus constellation. The inner- and outer-ring symbols lie on the same diagonal phase set as QPSK; after the fourth-power operation they therefore share the same phase reference, apart from their amplitude. The middle-ring symbols do not satisfy this property: their phases are not separated only by multiples of $\pi/2$, so raising them to the fourth power leaves a symbol-dependent phase term. If such symbols were used directly, the estimator could confuse this residual modulation term with an actual carrier-phase spread.

Under the locally linear phase assumption introduced in Equation 3.1, the total phase spread across the block may be estimated from representative symbols located near:

- the beginning of the block,
- the middle of the block,
- and the end of the block.

The first and last symbols are used to estimate the magnitude of the total phase spread, while the middle symbol assists in determining the direction of the phase evolution.

Let R_0 and R_{B-1} denote the selected fourth-power symbols near the beginning and end of the block. The cosine of the total phase spread may then be estimated through the normalized dot product:

$$\frac{\langle R_0, R_{B-1} \rangle}{\|R_0\| \|R_{B-1}\|} = \cos(4|\theta|) \quad (3.12)$$

with θ the total phase spread across the block.

The spread magnitude can therefore be recovered through:

$$|\theta| = \frac{1}{4} \arccos \left(\frac{\langle R_0, R_{B-1} \rangle}{\|R_0\| \|R_{B-1}\|} \right) \quad (3.13)$$

The use of $\arccos(\cdot)$ instead of $\arctan(\cdot)$ is important because the total phase spread may exceed π . A direct arctangent-based angle estimation could therefore lead to sign ambiguities and incorrect unwrap decisions. The cosine-based formulation instead estimates only the absolute spread magnitude, after which the spread direction is estimated separately.

To determine the direction of the phase evolution, a second estimate is performed using symbols from the beginning and middle of the block. The sign of the phase evolution can be obtained through the sign of the complex cross product:

$$\text{sign}(\gamma) = -\text{sign}(\Re\{R_0\}\Im\{R_b\} - \Im\{R_0\}\Re\{R_b\}) \quad (3.14)$$

with R_b the selected middle-region symbol.

Combining both estimates yields the final spread estimate:

$$\hat{\theta} = \text{sign}(\gamma) \cdot |\theta| \quad (3.15)$$

The estimated spread is subsequently used to extrapolate leading and lagging phase estimates for the phase unwrapper and to reconstruct multiple phase corrections inside the block during phase compensation.

The fourth-power transformation of the 16QAM constellation is illustrated in Figure 3.5. Unlike QPSK, where all transformed symbols share one phase reference and one radius, transformed 16QAM symbols occupy multiple radii and, for the middle-ring points, multiple residual phase classes. The amplitude differences themselves are not a problem because the spread magnitude is extracted through a normalized dot product. The residual phase classes are the problem: they add a data-dependent phase offset that is unrelated to the carrier phase evolution.

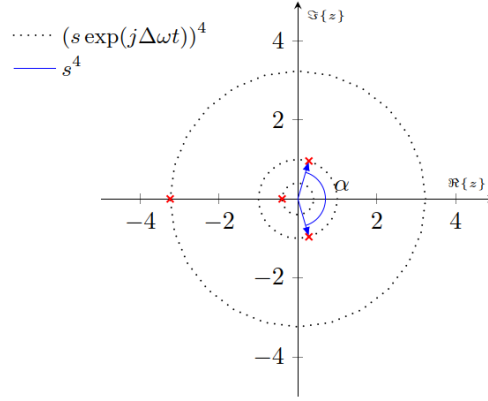


Figure 3.5: 16QAM symbols after fourth-power transformation

As a consequence, not every received symbol may be used directly for spread estimation. The architecture therefore includes an input-selection stage that identifies suitable inner- or outer-ring candidate symbols before the actual spread calculation is performed.

3.3.1 Input symbol consideration

Symbols originating from the middle ring of the 16QAM constellation can introduce large phase ambiguities after fourth-power transformation. Using these symbols directly in the normalized dot-product calculation of Equation 3.12 can therefore lead to incorrect spread estimates. To mitigate this problem, the spread estimator only considers symbols originating from the inner and outer constellation rings. These symbols retain the QPSK-like fourth-power phase relation and therefore yield more reliable spread estimates.

Rather than searching the complete processing block for suitable symbols, only a limited subset of symbols near the beginning, middle and end of the block is evaluated. This reduces hardware complexity while maintaining a sufficiently high probability of finding valid candidate symbols.

Let N denote the number of candidate symbols evaluated per region. Assuming uniformly distributed 16QAM symbols, the probability of finding at least one valid candidate pair may be approximated by:

$$P(\text{hit}) = 1 - \left[\left(\frac{1}{2^N} \right)^2 + 2 \left(\frac{1}{2^N} \left(1 - \frac{1}{2^N} \right) \right) \right] \quad (3.16)$$

A moderate value of N therefore already yields a high probability of obtaining usable symbols while avoiding the hardware cost associated with a full-block search. For example, $N = 3$ gives $P(\text{hit}) = 49/64 \approx 76.6\%$ under the uniform-symbol assumption, as illustrated in Figure 3.6. If no suitable candidate combination is found, the previously estimated spread value is retained. This behaviour is justified by the relatively slow variation of practical laser frequency offsets compared to the short duration of a processing block in baudrate-sampled coherent systems.

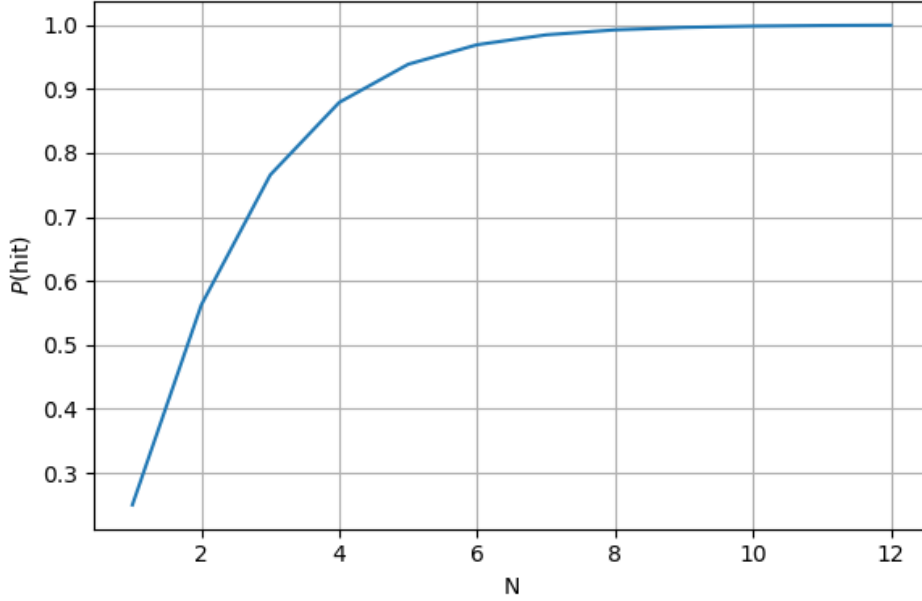


Figure 3.6: Hit probability versus #candidate symbols

The selected candidate symbols are subsequently forwarded to the hardware blocks responsible for fourth-power calculation, spread estimation and sign estimation.

3.4 Phase unwrapping

The SBPS stage produces one carrier phase estimate per block, but this estimate is still ambiguous by multiples of $\pi/2$ because the decision-directed metric is invariant under quadrant rotations for square QAM constellations. Therefore, the estimated centre phase $\hat{\phi}_{e,b}$ of block b must be unwrapped before it can be used as a continuous phase trajectory.

If only the centre phases of consecutive blocks were compared, a large intra-block phase spread could be interpreted incorrectly as a discontinuity between blocks. The proposed architecture therefore unwraps the phase by comparing the extrapolated edge phases of neighbouring blocks. The signed spread estimate $\hat{\theta}_b$ is used to estimate the phase at the left and right edge of block b :

$$\hat{\phi}_{L,b} = \hat{\phi}_{e,b} - \frac{\hat{\theta}_b}{2}, \quad (3.17)$$

$$\hat{\phi}_{R,b} = \hat{\phi}_{e,b} + \frac{\hat{\theta}_b}{2}. \quad (3.18)$$

Here, $\hat{\phi}_{L,b}$ represents the estimated carrier phase at the beginning of the block, while $\hat{\phi}_{R,b}$ represents the estimated carrier phase at the end of the block. This is also visualised in Figure 3.7. For a positive spread estimate, the phase therefore increases from the left edge to the right edge.

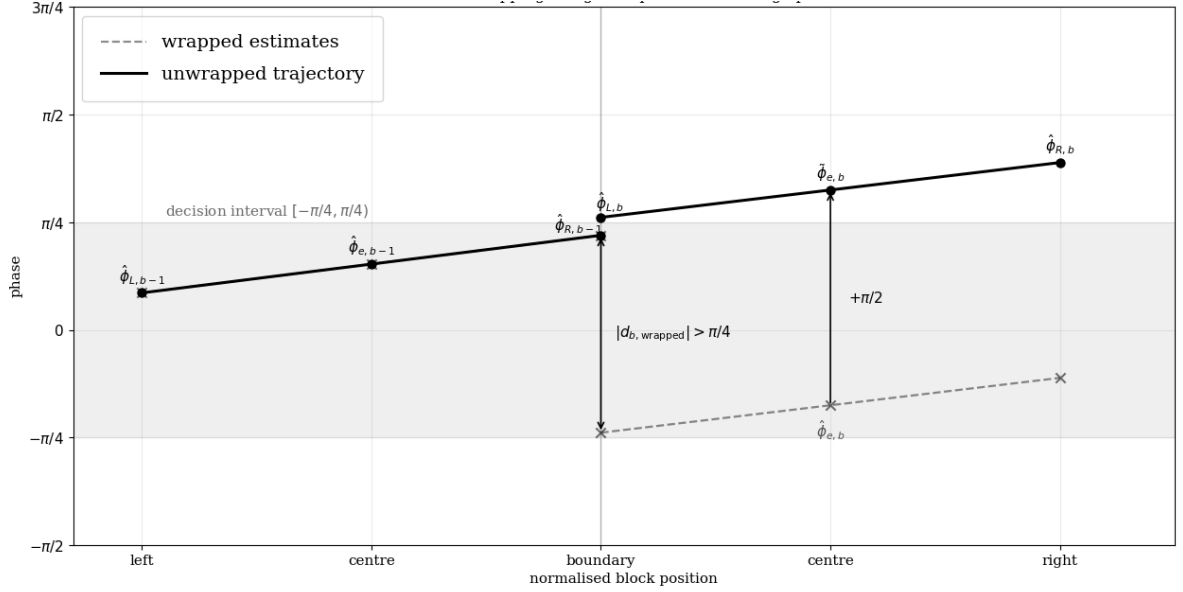


Figure 3.7: Example of a phase unwrapping

The edge discontinuity between two consecutive blocks is defined as:

$$d_b = \hat{\phi}_{L,b} - \hat{\phi}_{R,b-1}. \quad (3.19)$$

As long as the correct $\pi/2$ branch is maintained, this difference should remain small because the right edge of the previous block and the left edge of the current block describe adjacent points in the same phase trajectory. A discontinuity larger than the quadrant-decision threshold indicates that the wrapped SBPS estimate has crossed to a neighbouring $\pi/2$ branch.

The phase-unwrapping state is stored as an integer quadrant index q_b . This index determines which multiple of $\pi/2$ must be added to the wrapped SBPS phase estimate:

$$\tilde{\phi}_{e,b} = \hat{\phi}_{e,b} + q_b \frac{\pi}{2}, \quad (3.20)$$

where $\tilde{\phi}_{e,b}$ denotes the unwrapped centre phase. The quadrant index is updated from the edge discontinuity as:

$$q_b = \begin{cases} q_{b-1} - 1, & d_b \geq \frac{\pi}{4}, \\ q_{b-1} + 1, & d_b < -\frac{\pi}{4}, \\ q_{b-1}, & \text{otherwise.} \end{cases} \quad (3.21)$$

The first block establishes the initial branch and is therefore passed through without a correction. For all subsequent blocks, the update in Equation 3.21 tracks quadrant crossings caused by the wrapped SBPS estimate. In the hardware implementation this index may be stored modulo four, since adding 2π does not change the final compensated constellation.

The threshold of $\pi/4$ follows directly from the $\pi/2$ ambiguity of square QAM. If the edge-to-edge phase difference remains within $[-\pi/4, \pi/4]$, the current block is assumed to belong to the same quadrant branch. If this interval is exceeded, the estimate is corrected by one quadrant. This also defines the practical tracking condition of the unwrapper: the true phase change between the compared block edges, including residual CFO, phase noise and estimation error, should remain smaller than the threshold. Otherwise, a wrong quadrant update can occur and a cycle slip is introduced.

By using the spread estimate during unwrapping, the decision is made at the physical boundary between two blocks rather than at their centres. This improves robustness when a residual CFO creates a significant phase ramp inside each block. The phase unwrapper therefore links the blockwise SBPS estimates into a continuous phase trajectory that can be used by the final compensation stage.

3.5 Phase compensation

After phase unwrapping, the receiver has two phase-related quantities for each block:

- the unwrapped centre phase $\tilde{\phi}_{e,b}$,
- and the signed spread estimate $\hat{\theta}_b$.

Together, these quantities describe a locally linear approximation of the carrier phase across the block. The most direct compensation method would be to calculate an individual phase correction for every symbol. However, this would require a separate phase value and rotator input for each symbol, increasing routing and arithmetic complexity in the hardware implementation.

To reduce this complexity, the block is divided into N_{sec} compensation sections. Each section is compensated using one representative phase value located at the centre of that section. Let n denote the section index, with:

$$n = 0, 1, \dots, N_{\text{sec}} - 1. \quad (3.22)$$

The normalized position of the centre of section n , relative to the block centre, is:

$$c_n = \frac{2n + 1 - N_{\text{sec}}}{2N_{\text{sec}}}. \quad (3.23)$$

The phase correction applied to section n is then given by:

$$\hat{\phi}_{b,n} = \tilde{\phi}_{e,b} + c_n \hat{\theta}_b. \quad (3.24)$$

This expression reconstructs a piecewise-constant approximation of the linear phase trajectory, which is illustrated in Figure 3.8. The first section receives a correction close to the left-edge phase, the last section receives a correction close to the right-edge phase, and the middle sections are distributed symmetrically around the unwrapped SBPS centre phase.

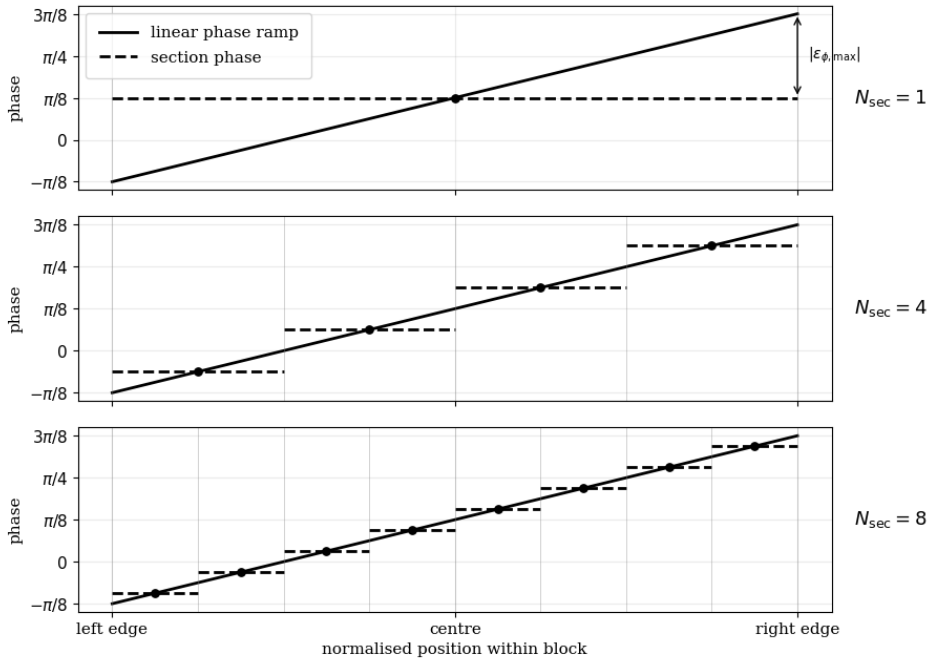


Figure 3.8: Section-based phase compensation

The set of symbol indices belonging to section n is defined as:

$$\mathcal{S}_n = \left\{ k \mid \left\lfloor \frac{nB}{N_{\text{sec}}} \right\rfloor \leq k < \left\lfloor \frac{(n+1)B}{N_{\text{sec}}} \right\rfloor \right\}. \quad (3.25)$$

All symbols belonging to section n are then rotated by the negative of the corresponding phase estimate:

$$\tilde{r}_{b,k} = r_{b,k} e^{-j\hat{\phi}_{b,n}}, \quad k \in \mathcal{S}_n. \quad (3.26)$$

This rotation removes both the common centre phase and the estimated intra-block phase ramp.

For $N_{\text{sec}} = 1$, the method reduces to conventional blockwise phase compensation using only the unwrapped SBPS estimate. Increasing N_{sec} improves the approximation of the phase ramp, but also increases the number of phase values that must be generated and routed to the rotators. For an ideal linear phase evolution, the maximum residual phase error inside one section is approximately:

$$|\epsilon_{\phi,\text{max}}| \approx \frac{|\hat{\theta}_b|}{2N_{\text{sec}}}. \quad (3.27)$$

This relation illustrates the main design trade-off. A larger number of compensation sections reduces the residual phase error after correction, while a smaller number of sections reduces hardware complexity. The implemented architecture therefore uses section-based compensation as a compromise between per-symbol phase correction and a single blockwise phase correction.

The hardware-oriented datapath, uses the piecewise-constant section model described above because the section coefficients of Equation 3.23 can be precomputed and each section phase can be generated with one multiplication and one addition. This makes the phase compensation stage well suited for a parallel block-based FPGA implementation.

This page is intentionally left blank.

Hardware implementation

4.1 Data representation and scaling

In this section, the data representation choices used throughout the hardware architecture are discussed and motivated. Because the proposed carrier recovery system is implemented entirely in digital hardware, the numerical representation of both the received samples and the phase-rotation coefficients has a direct impact on the achievable accuracy, hardware complexity and resource utilization.

sample representation

First, the representation of the received samples r is discussed. Afterwards, the representation of the phase-rotation coefficients used throughout the SBPS and phase compensation stages is explained.

The received symbols are represented in Cartesian form as complex in-phase/quadrature (I/Q) samples:

$$r = I + jQ \quad (4.1)$$

where both I and Q are stored using signed fixed-point formats. Cartesian I/Q representation is preferred over magnitude-phase representation because the majority of the DSP operations in the architecture consist of additions, multiplications and comparisons on the real and imaginary components. These operations map efficiently onto FPGA DSP slices and LUT-based arithmetic structures.

Furthermore, phase rotations in coherent DSP are naturally implemented using complex multiplication:

$$r' = r \cdot e^{-j\phi} \quad (4.2)$$

Expanding this into Cartesian form yields:

$$I' = I \cos(\phi) + Q \sin(\phi), \quad Q' = -I \sin(\phi) + Q \cos(\phi) \quad (4.3)$$

As a result, the phase-rotation coefficients are represented directly as fixed-point cosine and sine values rather than as angles themselves. While storing the phase as an angle would simplify interpretation, it would require costly trigonometric operations whenever a rotation has to be applied. By representing the phase coefficients directly as $\cos(\phi)$ and $\sin(\phi)$, the hardware implementation only requires multiplications and additions during rotation.

The received symbols correspond to a normalized 16QAM constellation. For a square 16QAM constellation, the possible in-phase and quadrature amplitudes are:

$$\pm s \quad \pm 3s \quad (4.4)$$

resulting in constellation points formed by all possible (I, Q) combinations of these values.

To normalize the average constellation power to unity, the scaling factor s is chosen as:

$$s = \frac{1}{\sqrt{10}} \quad (4.5)$$

This normalization ensures that the average symbol energy equals one:

$$\mathbb{E}[|r|^2] = 1 \quad (4.6)$$

which is convenient for both simulation and hardware verification, since impairments such as SNR, linewidth and residual frequency offset can then be expressed independently of the constellation scaling.

sample scaling

Because FPGA resources and datapaths are limited, all samples are represented using fixed-point formats. Floating-point arithmetic would significantly increase the hardware complexity, routing overhead and power consumption of the design. Fixed-point arithmetic, on the other hand, allows efficient mapping onto FPGA DSP resources while still providing sufficient numerical precision for the considered coherent optical communication scenarios.

In this work, the received I/Q samples are represented using signed Q1.15 fixed-point format. This corresponds to a 16-bit signed integer representation with one sign/integer bit and 15 fractional bits. The representable range is therefore:

$$-1 \leq x < 1 \quad (4.7)$$

Before mapping the constellation onto this range, the maximum possible signal excursion under phase rotation must first be considered.

The outer-most 16QAM constellation points are located at:

$$(\pm 3s, \pm 3s) \quad (4.8)$$

These points have the largest magnitude:

$$|r_{\max}| = \sqrt{(3s)^2 + (3s)^2} = 3\sqrt{2}s \quad (4.9)$$

Substituting $s = 1/\sqrt{10}$ yields:

$$|r_{\max}| \approx 1.34 \quad (4.10)$$

This means that a pure phase rotation of the outer-most constellation points can already produce instantaneous I or Q values larger than unity when the constellation aligns with one of the axes.

If this effect were not taken into account, clipping would occur during phase rotation. Such clipping is especially problematic for the SBPS metric calculations, since the Euclidean-distance calculations would no longer correctly represent the actual distance to the ideal constellation points. As a result, false local minima could appear in the error function, potentially causing incorrect phase estimation (as seen in Figure 4.1).

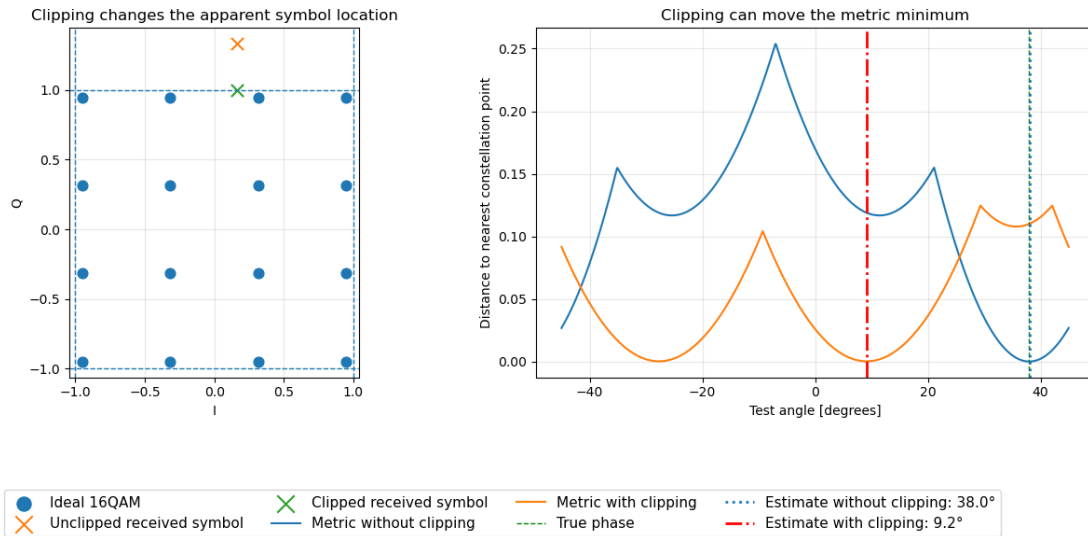


Figure 4.1: Clipping causing false minima for SBPS metric

Besides phase rotation, additive white Gaussian noise (AWGN) can further increase the instantaneous sample magnitude. Designing the fixed-point range only for ideal phase-rotated constellation points would therefore still result in clipping under low-SNR conditions.

Ideally, the constellation scaling would adapt dynamically to the received signal quality such that the available integer range is optimally utilized for every SNR. However, implementing such adaptive scaling falls outside the scope of this thesis. Instead, a fixed scaling factor is used throughout the architecture.

Taking into account both the maximum phase-rotated constellation magnitude and additional noise excursions, a static scaling factor of

$$\alpha = 0.678 \quad (4.11)$$

is selected.

Using this scaling factor, the normalized constellation points are mapped onto the Q1.15 integer range as:

$$\pm 7025 \quad \pm 3 \cdot 7025 \quad (4.12)$$

This scaling provides sufficient headroom for phase rotations and moderate noise excursions while still maintaining adequate fixed-point precision for the DSP 16-bit operations.

Figure 4.2 illustrates the effect of different SNR conditions on the fixed-point representation, including the scaling factor α . At sufficiently low SNR, the noisy constellation points exceed the representable range and clipping occurs. This clipping degrades the accuracy of both the SBPS metric calculations and the spread estimation calculations, ultimately reducing the phase recovery performance of the architecture.

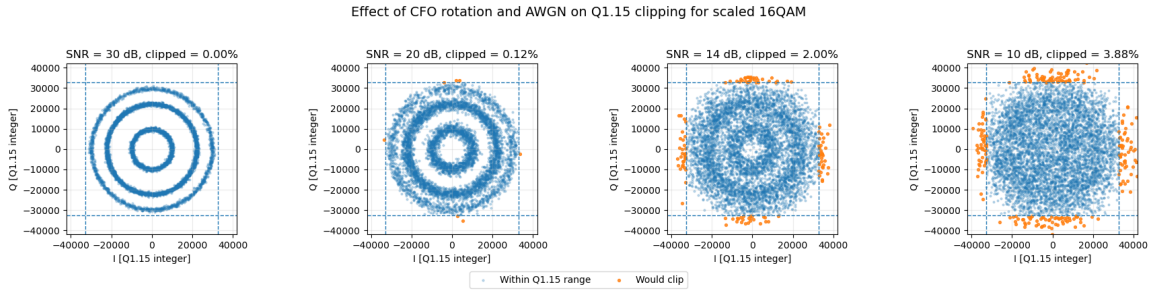


Figure 4.2: Clipping of 16QAM symbols under different SNR conditions

Phase representation

The phase-related quantities used throughout the architecture are represented separately from the received I/Q samples. Depending on the type of operation being performed, two different phase representations are used throughout the pipeline:

1. trigonometric representation using $\cos(\phi)$ and $\sin(\phi)$
2. direct phase-word representation using an angle ϕ

Both representations have advantages and disadvantages in terms of hardware complexity and ease of manipulation.

The trigonometric representation is inherently well-suited for applying phase rotations to complex samples. A rotation over phase ϕ can directly be implemented as seen in equation 4.3, meaning that only multiplications and additions are required. This maps efficiently onto FPGA DSP slices and avoids the need for expensive real-time trigonometric calculations.

For this reason, the SBPS stages internally use phase coefficients represented directly as fixed-point sine and cosine values. Since both $\sin(\phi)$ and $\cos(\phi)$ are bounded between -1 and 1 , these coefficients are represented using signed Q1.15 fixed-point format.

However, while trigonometric representation is efficient for applying rotations, it is considerably less convenient for manipulating the phase itself. Operations such as phase accumulation or phase interpolation become significantly more complex when the phase is stored as sine and cosine coefficients.

For example, adding two phase terms ϕ_e and θ in trigonometric form requires the angle-sum identities:

$$\cos(\phi_e + \theta) = \cos(\phi_e) \cos(\theta) - \sin(\phi_e) \sin(\theta) \quad (4.13)$$

$$\sin(\phi_e + \theta) = \sin(\phi_e) \cos(\theta) + \cos(\phi_e) \sin(\theta) \quad (4.14)$$

Rather than requiring a single addition, this operation now requires four multiplications and multiple additions. In contrast, when the phase is represented directly as a phase word, the same operation simply becomes:

$$\phi_{\text{new}} = \phi_e + \theta \quad (4.15)$$

which only requires an adder.

Within the SBPS module, this limitation is not problematic. The iterative test angles are static during runtime and are fully known at synthesis time. Therefore, all required sine and cosine coefficients can be precomputed and stored in small lookup tables. The different stage rotations, such as $\pm\pi/8$, $\pm\pi/16$ and subsequent refinements, are thus directly available in trigonometric form without requiring any runtime phase calculations.

The later pipeline stages, however, rely heavily on direct phase manipulation. The phase unwrapper calculates leading and lagging phase estimates, while the phase compensation stage interpolates phase values across the different sections of a block. These operations all naturally operate on phase words rather than trigonometric coefficients.

Furthermore, the CORDIC IP used in the phase compensation stage also accepts phase-word inputs directly. This makes phase-word representation considerably more efficient for the downstream processing stages.

The spread estimation (SE) module also inherently produces a phase-word representation. After the fourth-power operation removes the modulation, the phase spread is estimated using a normalized dot product (equation 3.12) followed by an arccosine operation (equation 3.13).

The output of this operation is fundamentally an angle estimate represented as a phase word. Representing this entire operation directly in trigonometric form would require repeated conversions between angular and Cartesian domains, significantly increasing the arithmetic complexity.

Taking into account the complete pipeline structure, a mixed representation strategy is therefore adopted. The SBPS stages use trigonometric phase representation because their primary purpose is efficient sample rotation using static test angles. The downstream pipeline stages, however, use direct phase-word representation because they require frequent phase manipulations and interpolations.

To bridge both domains, the final SBPS output coefficients are converted from trigonometric form into a phase word using a CORDIC-based vectoring operation.

The overall flow of phase representations between the different modules is illustrated in Figure 4.3.

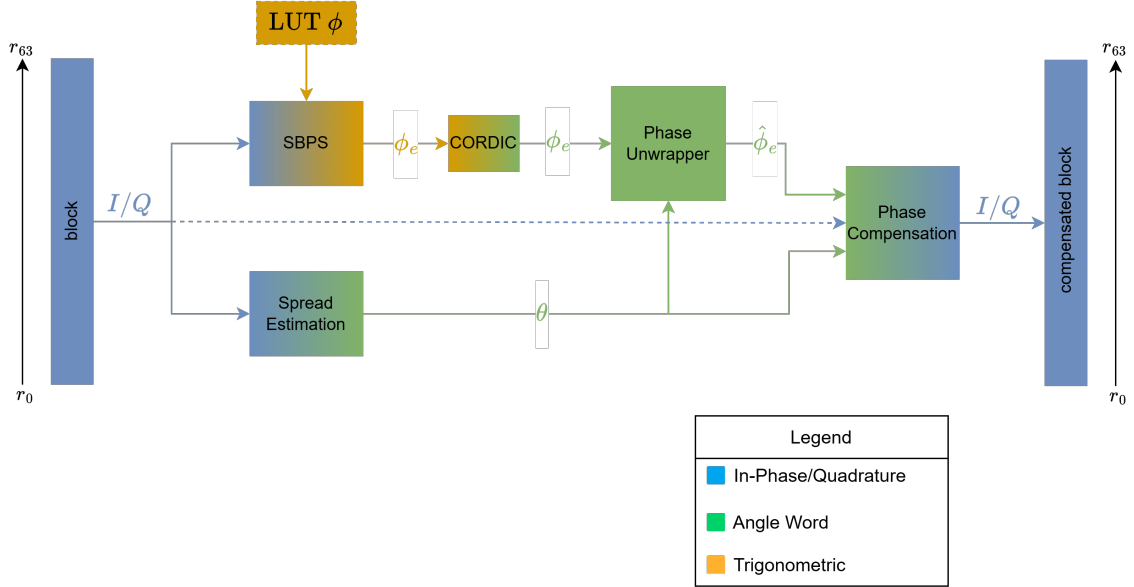


Figure 4.3: Data-representation throughout the top design

The use of separate fixed-point formats and phase representations therefore represents a trade-off between arithmetic precision, resource utilization and implementation complexity within the FPGA architecture.

4.2 Serialized binary phase search block

The hardware implementation follows a staged refinement architecture, shown in Fig. 4.4. The same received block

$$\bar{r} = (r_0, r_1, \dots, r_{W-1})$$

is presented to every stage, while only the accumulated phase estimate is updated from stage to stage. At stage m , two candidate phase updates are evaluated around the previously selected estimate ϕ_{m-1} , namely $\phi_{m-1} - \phi_t$ and $\phi_{m-1} + \phi_t$. The candidate that produces the smaller block error is selected as the stage output ϕ_m , which is then forwarded to the next stage. By cascading several stages with progressively smaller test angles, the architecture converges towards the phase estimate associated with the centre of the block.

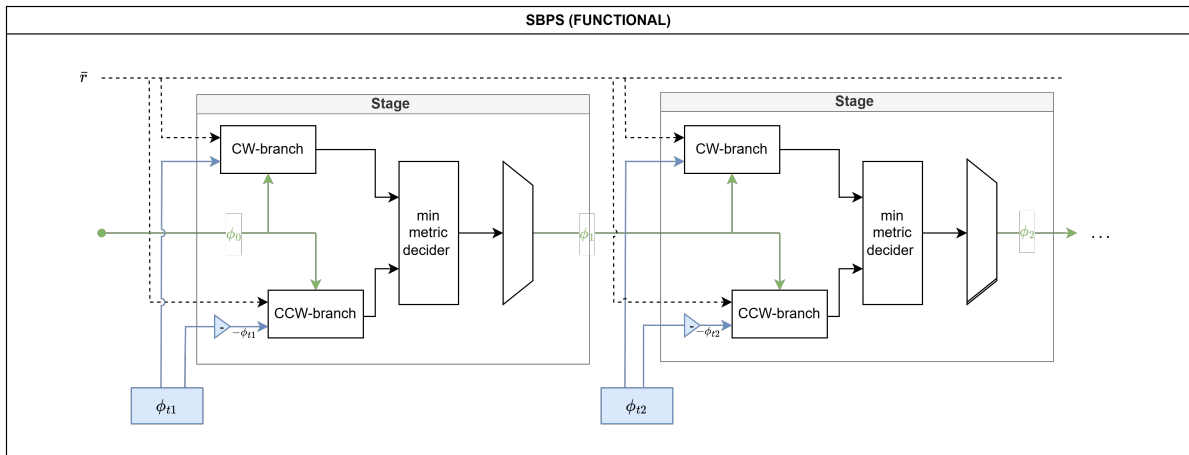


Figure 4.4: Functional diagram of the full SBPS architecture

4.2.1 Functional operation

Figure 4.5 shows the internal organisation of a single stage. The upper branch corresponds to the clockwise (CW) update and evaluates $\phi_{m-1} - \phi_t$, whereas the lower branch corresponds to the counter-clockwise (CCW) update and evaluates $\phi_{m-1} + \phi_t$. Each branch contains a *rotate metric slice* unit that processes the complete input block using its own candidate phase. The resulting block metrics are compared by the *min metric decoder*, and the associated candidate phase is selected by the final multiplexer (MUX). In this way, each stage performs a binary phase refinement around the current estimate.

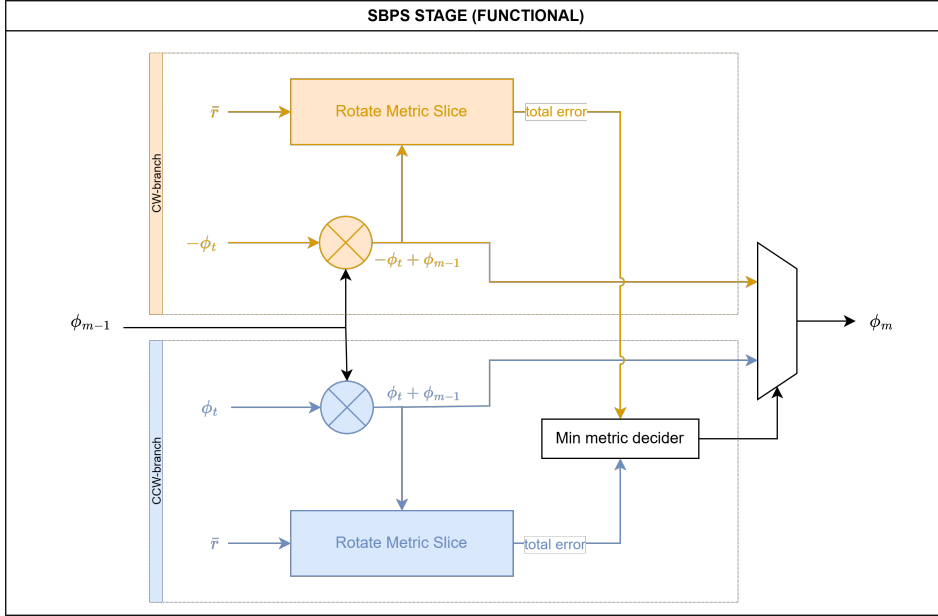


Figure 4.5: Functional diagram of the SBPS stage

The *rotate metric slice* unit is the core computational block of the design and is detailed in Fig. 4.6. For every symbol in the block, the candidate phase is used to rotate the received sample, the rotated sample is sliced to a reference constellation point, and the squared Euclidean distance between the rotated and sliced values is computed. In the normal stages this reference is the nearest 16QAM point. In the first radius-directed stages, the slicer uses the modified decision rule discussed in Subsection 3.2.1. The resulting per-symbol errors are accumulated over the full block window to obtain a single decision metric for the branch. All symbols in the block are therefore processed in parallel inside each branch, while the serial refinement occurs across the SBPS stages.

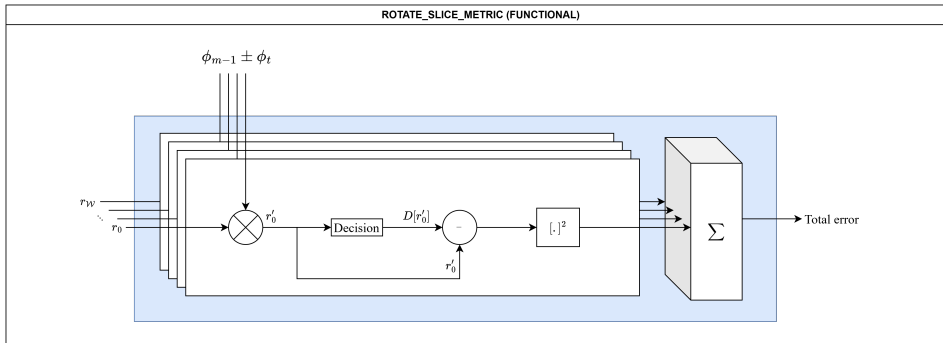


Figure 4.6: Functional diagram of the *rotate metric slice* unit

4.2.2 Data flow

The data-oriented view of the implementation is shown in Figs. 4.7, 4.8, and 4.9.

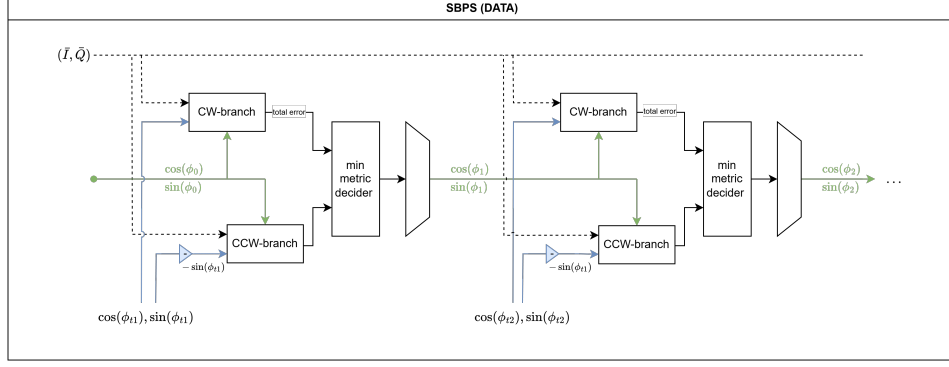


Figure 4.7: Datapath diagram of the full SBPS architecture

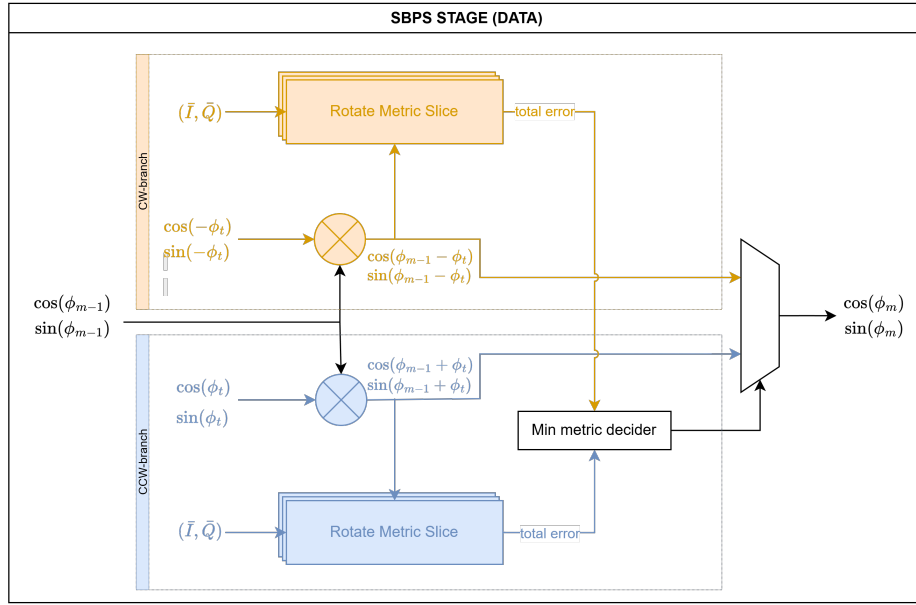


Figure 4.8: Datapath diagram of the SBPS stage

In the implemented datapath, phase is not represented as an explicit angle word. Instead, each phase value is carried by its trigonometric components ($\cos(\phi)$, $\sin(\phi)$) in fixed-point format. This allows the candidate phases to be generated directly by the *angle add* block from the previous estimate ($\cos(\phi_{m-1})$, $\sin(\phi_{m-1})$) and the stage test angle ($\cos(\phi_t)$, $\sin(\phi_t)$):

$$\cos(\phi_{m-1} \pm \phi_t) = \cos(\phi_{m-1}) \cos(\phi_t) \mp \sin(\phi_{m-1}) \sin(\phi_t), \quad (4.16)$$

$$\sin(\phi_{m-1} \pm \phi_t) = \sin(\phi_{m-1}) \cos(\phi_t) \pm \cos(\phi_{m-1}) \sin(\phi_t). \quad (4.17)$$

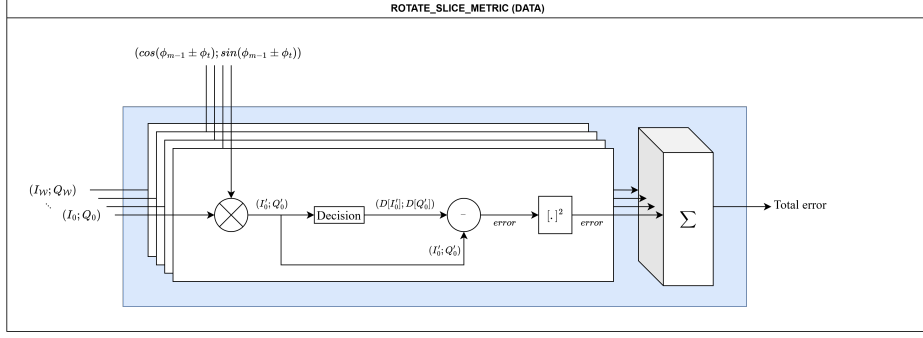
Once a candidate phase has been formed, the received samples are rotated in the I/Q plane. For a candidate angle

$$\theta \in \{\phi_{m-1} - \phi_t, \phi_{m-1} + \phi_t\},$$

the rotator computes

$$\begin{aligned} I'_k &= I_k \cos(\theta) + Q_k \sin(\theta), \\ Q'_k &= -I_k \sin(\theta) + Q_k \cos(\theta), \end{aligned} \quad (4.18)$$

for every symbol $r_k = I_k + jQ_k$ in the block. The rotated symbol is then denoted by $r'_k = I'_k + jQ'_k$.

Figure 4.9: Datapath diagram of the *rotate metric slice* unit

For standard 16QAM operation, the slicer maps each rotated sample to the nearest reference point $D[r'_k]$. The axis decisions are derived from the conventional inner-outer thresholds, with $\pm 2s$ defining the transition between the inner and outer levels of the constellation. The resulting single-symbol error is

$$e_k = |r'_k - D[r'_k]|^2 = (I'_k - D_I[r'_k])^2 + (Q'_k - D_Q[r'_k])^2, \quad (4.19)$$

and the branch metric is obtained by accumulation over the complete block:

$$E = \sum_{k=0}^{W-1} e_k. \quad (4.20)$$

This accumulated metric is the quantity used by the *min metric decider* to select the surviving branch.

4.2.3 Timing flow

The timing behaviour of the full chain, of one SBPS stage, and of the *rotate metric slice* unit is summarised in Figs. ??, 4.10, and 4.11, respectively. The control path uses single-cycle valid pulses rather than a ready/valid handshake, so temporal alignment is achieved entirely through fixed pipeline delays.

At the stage input, the two candidate phases are first generated while the input sample block is delayed in parallel so that both reach the branch datapaths in synchrony. The *angle add* operation introduces three elapsed clock cycles, and the sample block is delayed by the same amount before it enters the *rotate metric slice* unit.

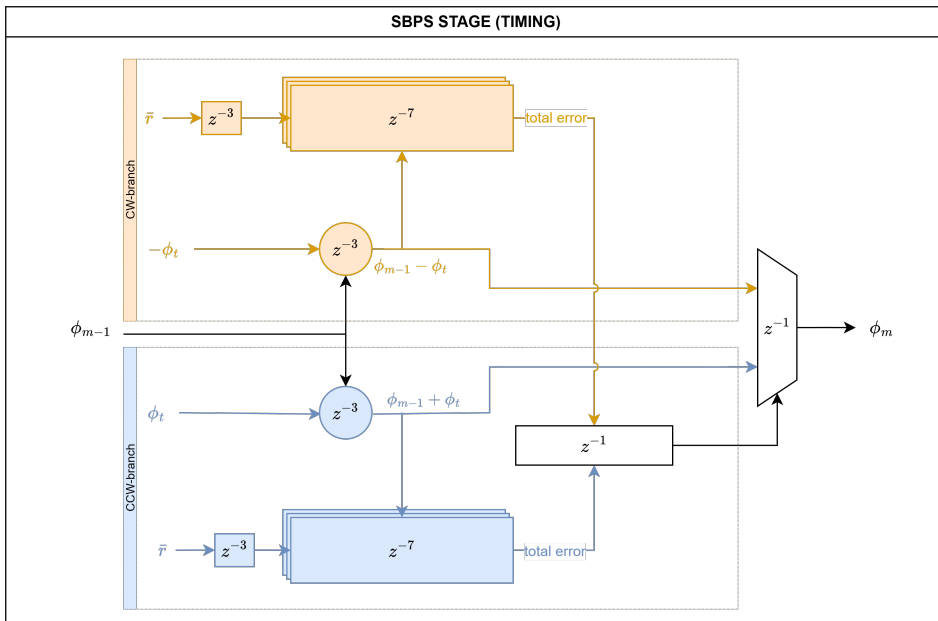


Figure 4.10: Timing diagram of the SBPS stage

Inside the *rotate metric slice* unit, the rotator contributes three cycles, the slicer and sample-alignment logic contribute one cycle, the single-symbol metric calculation contributes one cycle, and the summation plus output registration contribute two further cycles. The branch metric is therefore available seven cycles after entry into the *rotate metric slice* unit, or ten elapsed cycles after the stage launch when the initial angle-generation and input-alignment delay are included.

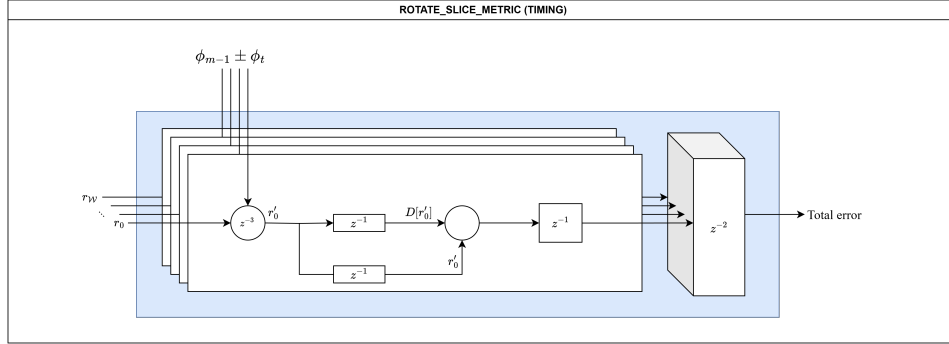


Figure 4.11: Timing diagram of the *rotate metric slice* unit

After both branch metrics have become available, the *min metric decider* requires one clock cycle to determine the winning branch, and the final stage output is registered one cycle later. Consequently, the phase-selection datapath of a stage spans eleven clock cycles, while the registered stage-valid output is asserted after twelve clock cycles. When latency is reported at the boundary of the complete SBPS chain rather than at a single stage boundary, one additional launch cycle must be included because the top-level chain registers the hand-off into the first stage.

4.2.4 Radius-directed stages

The motivation for the radius-directed stages is the pattern effect described in Subsection 3.2.1. During the first SBPS stages the candidate phases are still separated by relatively large angles. In this regime, middle-ring 16QAM symbols can produce misleading nearest-neighbour decisions and may create false minima in the accumulated branch metric. The hardware therefore supports a modified slicer mode for the first $G_{\text{RAD_DIRECTED_N}}$ stages of the chain.

In the normal 16QAM slicer, each axis is decided independently using the threshold $\pm 2s$:

$$D_{16\text{QAM}}(I', Q') \in \{\pm s, \pm 3s\} \times \{\pm s, \pm 3s\}. \quad (4.21)$$

This implements the usual nearest-neighbour decision regions. In the radius-directed mode, an additional comparator threshold $l/2$ is evaluated on the absolute values of the rotated components. If either component lies outside this central box, the symbol is forced to the corresponding outer corner:

$$D_{\text{RD}}(I', Q') = \begin{cases} (3s \operatorname{sgn}(I'), 3s \operatorname{sgn}(Q')), & |I'| \geq l/2 \vee |Q'| \geq l/2, \\ D_{16\text{QAM}}(I', Q'), & \text{otherwise.} \end{cases} \quad (4.22)$$

With l chosen such that the central box covers the inner constellation points but not the high-amplitude middle/outer points, this rule removes explicit middle-ring decisions from the coarse stages. The metric is then dominated by the inner and corner-like symbol classes, which have a more reliable angular relation to the carrier phase. This is cheaper than computing the Euclidean radius of every rotated sample, since the implementation only needs absolute-value operations and two comparisons per symbol.

The generic $G_{\text{RAD_DIRECTED_N}}$ controls how many early stages use this slicer mode. For stage index k , the generated chain sets the stage to radius-directed operation when $k < G_{\text{RAD_DIRECTED_N}}$. Once the phase estimate has been sufficiently refined, later stages revert to the conventional 16QAM slicer. This keeps the robustness benefit during coarse acquisition while preserving the final-stage resolution of the full 16QAM decision metric.

4.3 Spread estimation block

The hardware implementation is split up in the different operational sections of the algorithm, namely the input selector (1), fourth-power calculators (2), the spread estimator (3) & the sign estimator (4) and finally the filters (5). The layout of these modules and their relative position is illustrated on Fig 4.12. Each section will individually discuss the modules in terms of its functional operation, dataflow and timing flow in order to get a clearer picture of the RTL implementation.

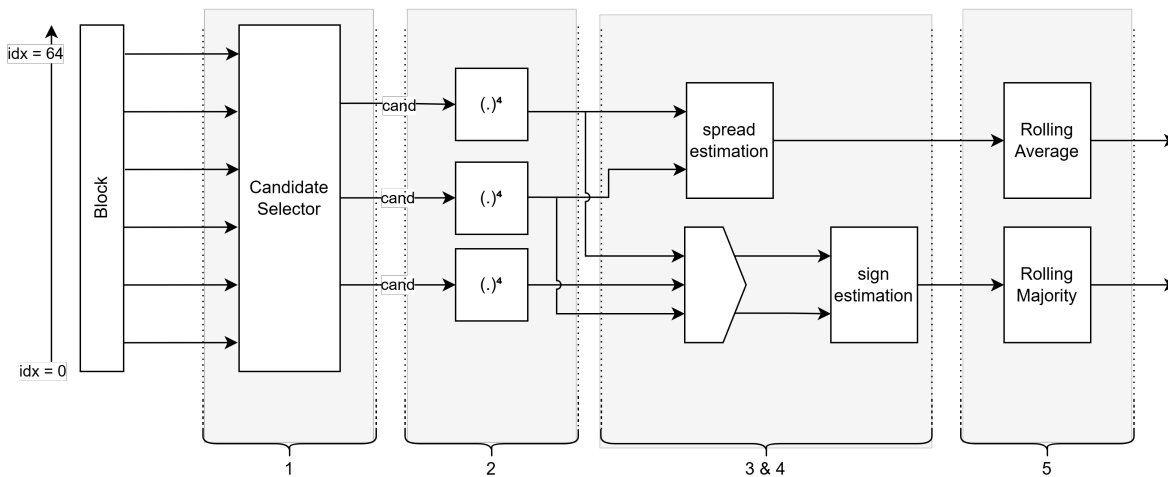


Figure 4.12: Top lay-out of SE block

4.3.1 Input selector

Functional operation

As introduced in Section 3.3.1, the spread and direction estimation blocks require a set of representative symbols from the input block. However, not all symbols are suitable for this purpose. Therefore, an input selection stage is required to extract valid candidate symbols from different regions of the block.

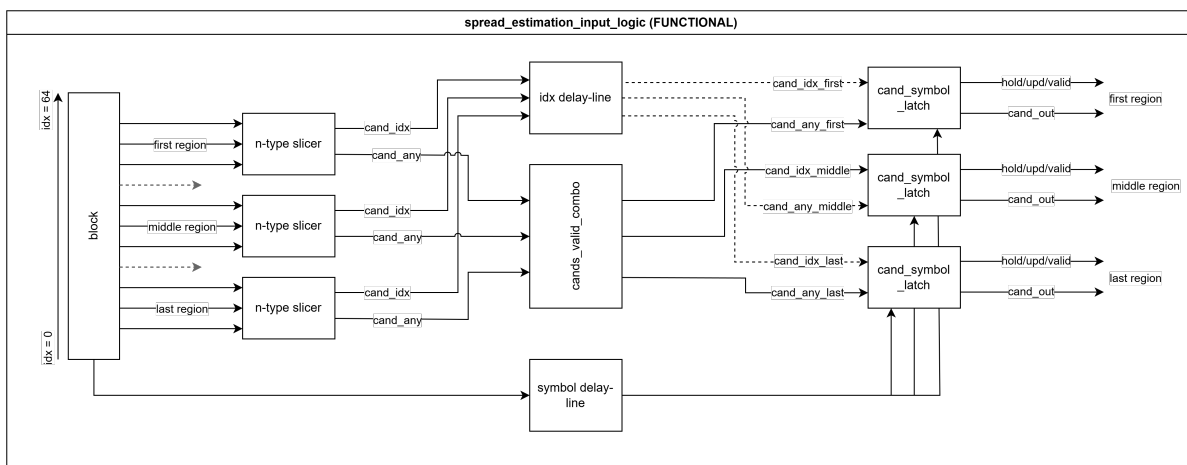


Figure 4.13: Functional diagram of the SE input selector

The input selector operates on the full block of received symbols $\bar{r}$, and outputs three candidate symbols x , y , and z , corresponding to the first, middle, and last regions of the block, respectively. These candidates are subsequently used for spread and direction estimation.

The selection process consists of three sequential steps:

1. Identification of valid candidate symbols within each region of the block.
2. Validation of the combination of candidates across regions.
3. Latching or updating of the selected symbols for further processing.

These steps are implemented by the *n-type slicer*, *cands_valid_combo*, and *cand_symbol_latch* modules, respectively.

Each region (first, middle, last) is processed by an independent *n-type slicer*. This module evaluates a small subset of symbols (three in this implementation) and determines whether a suitable candidate is present. A symbol is considered valid if it lies on the inner or outer ring of the constellation, which ensures reliable spread estimation (this criterion is further elaborated in the algorithmic description of the spread estimator). The index of the first valid symbol is returned via the *cand_idx* bus. If no valid candidate is found, the *cands_any* flag is deasserted.

The *cands_valid_combo* module evaluates the availability of candidates across the three regions. A valid combination is only accepted if at least two regions provide a candidate symbol. If this condition is not met, all candidate flags are suppressed. This constraint reduces the likelihood of unreliable spread or direction estimates, which would otherwise arise from insufficient spatial information within the block.

Finally, the *cand_symbol_latch* module selects and outputs the candidate symbols. For each region where the *cands_any* flag is asserted, the corresponding symbol is extracted from the input block using the provided *cand_idx*. If valid candidates are present, the outputs are updated and the *update* signal is asserted. Otherwise, the previous outputs are retained and the *hold* signal is asserted. In this way, consistency is maintained when no valid candidate combination is available.

Timing flow

The timing behaviour of the input selector is illustrated in Figure 4.14.

The *n-type slicer* requires two clock cycles to determine and register a valid candidate index. The subsequent *cands_valid_combo* stage introduces an additional cycle to produce stable validity signals.

To ensure correct operation of the *cand_symbol_latch*, all inputs must be temporally aligned. In particular, the candidate indices and their corresponding validity flags must be synchronized with the original input symbols. To achieve this, delay elements are inserted in the data path. The input symbol block $\bar{r}$ is delayed by three clock cycles, matching the combined latency of the slicer and combination stages. The *cand_idx* signals are delayed by one cycle to align with the output of the *cands_valid_combo* module.

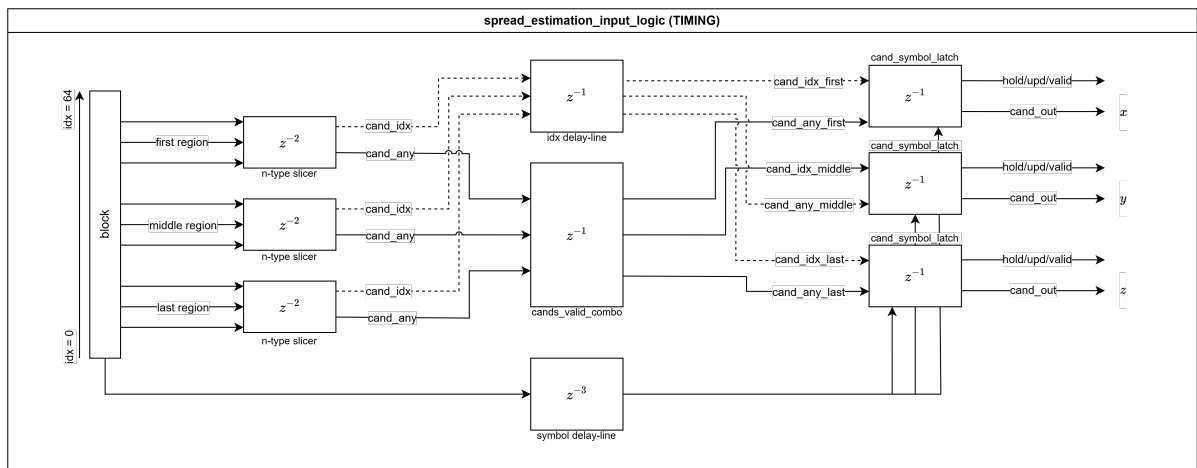


Figure 4.14: Timing flow diagram of the SE input selector

The *cand_symbol_latch* itself introduces one additional clock cycle to register the selected symbols. As a result, the total latency of the input selector amounts to four clock cycles.

4.3.2 Fourth-power I/Q calculators

Functional operation

The fourth-power calculators are an important part of the spread estimation algorithm. The purpose of the fourth-power operation is to suppress the rotational symmetry of the modulation and to make the carrier-induced phase evolution more directly observable.

For a complex input sample

$$r = I + jQ, \quad (4.23)$$

the block calculates

$$r^4 = (I + jQ)^4. \quad (4.24)$$

The output is again represented as a complex value, with a real and imaginary component. This operation is multiplication-heavy compared to most other operations in the low-complexity carrier recovery system. This is relevant because fourth-power based frequency or spread estimation is one of the computationally dominant operations in such systems: earlier work on this carrier recovery approach explicitly identifies the complex fourth-power operation as a large computation, requiring several real-valued multiplications per symbol [15].

In this implementation, the input samples are provided as signed fixed-point values in Q1.15 format. The fourth-power output is not rescaled back to Q1.15, but is kept at a much larger internal word length. This avoids unnecessary loss of precision in the following dot-product, sign and angle-estimation calculations without significantly effecting the compute needed for those calculations.

Data flow

Although the mathematical expression r^4 can be expanded directly, the hardware implementation avoids calculating all fourth-order terms independently. Instead, the calculation is structured as two consecutive squaring operations:

$$r^2 = (I + jQ)^2 = (I^2 - Q^2) + j(2IQ).$$

Defining

$$a = I^2 - Q^2, \quad b = 2IQ,$$

the fourth power becomes

$$r^4 = (r^2)^2 = (a + jb)^2.$$

This gives

$$r^4 = (a^2 - b^2) + j(2ab). \quad (4.25)$$

The real and imaginary fourth-power outputs are therefore calculated as

$$\Re\{r^4\} = a^2 - b^2, \quad (4.26)$$

$$\Im\{r^4\} = 2ab. \quad (4.27)$$

This decomposition maps naturally onto a pipelined multiplier structure. The first stage calculates the three second-order products I^2 , Q^2 and IQ . From these values, the second stage forms the intermediate real and imaginary components of r^2 : $a = I^2 - Q^2$ and $b = 2IQ$. The multiplication by two is implemented as a left shift, so no additional multiplier is required. The third stage then calculates a^2 , b^2 and ab . Finally, the fourth stage combines these terms into the real and imaginary components of the fourth-power output.

The resulting implementation requires six real multiplications: three multiplications in the first stage and three multiplications in the third stage. The remaining operations are additions, subtractions and shifts. This is more efficient than a direct polynomial expansion of $(I + jQ)^4$, because the intermediate square r^2 is reused.

The bit growth is also handled explicitly. Since the input samples are signed values with width W , the products I^2 , Q^2 and IQ require approximately $2W$ bits. The intermediate values a and b require one additional bit, because a is formed by a subtraction and b by a shift-left operation. Squaring a and b then doubles this width again. For this reason, the output width is chosen conservatively as

$$W_{\text{out}} \geq 4W + 3. \quad (4.28)$$

For the default input width of $W = 16$, this gives a required full-precision output width of at least 67 bits. The implementation uses a default output width of 70 bits, which provides additional margin and avoids overflow during the fourth-power calculation.

The use of a large fourth-power output width is acceptable here because the block is only applied to selected candidate symbols in the spread estimation path, rather than to every symbol in every blind phase search test angle. This fits the broader objective of the proposed architecture: reducing the number of expensive carrier recovery operations while preserving enough phase information to improve tracking of residual frequency offset.

Timing flow

The fourth-power calculator is implemented as a four-stage registered pipeline. Figure 4.15 shows the calculation flow through these stages.

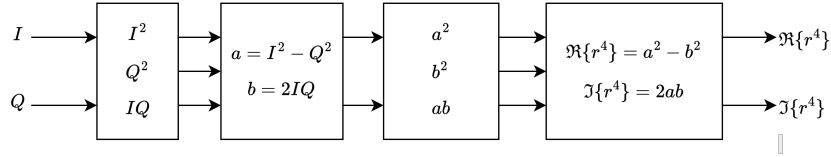


Figure 4.15: Calculation-flow of the fourth-power module

Each block in the figure corresponds to one clocked pipeline stage. The first stage performs the initial second-order products and registers the results internally. These intermediate values are then forwarded to the second stage, where the intermediate complex representation of the squared sample is formed.

The third stage contains the second multiplication layer of the architecture. Here, the intermediate terms generated in the previous stage are squared and multiplied again in preparation for the final fourth-power reconstruction. In the fourth and final stage, the real and imaginary outputs are assembled and driven to the output ports.

A valid signal is propagated alongside the datapath through the internal pipeline control signals `s1_valid`, `s2_valid`, `s3_valid` and `s4_valid`. This ensures that the output validity remains aligned with the corresponding fourth-power result throughout the pipeline.

The total latency of the module is therefore four clock cycles from assertion of `in_valid` to assertion of `out_valid`.

CORDIC-based alternative

A possible alternative implementation would be to use a CORDIC-based polar approach. In such a design, the input sample would first be converted from Cartesian form to magnitude and phase:

$$r = |r|e^{j\phi}. \quad (4.29)$$

The fourth power could then be calculated as

$$r^4 = |r|^4 e^{j4\phi}. \quad (4.30)$$

Conceptually, this is attractive because the phase multiplication by four is simple, and the CORDIC algorithm is well suited for angle extraction and vector rotation.

However, this approach is not necessarily beneficial in the context of this low-complexity architecture. A CORDIC implementation would require an angle extraction step, additional handling of the magnitude term, and a rotation or sine/cosine reconstruction step. This introduces iterative logic, latency and control complexity. The fourth-power block used here, by contrast, is a feed-forward arithmetic pipeline consisting only of multipliers, adders, subtractors and shifts. It also produces exactly the Cartesian components required by the following dot-product and sign-estimation logic.

The main advantage of a CORDIC-based approach would be multiplier reduction, especially in an FPGA design where DSP slices are scarce or already heavily used. Its main disadvantage is that the algorithm becomes less direct: the system would temporarily move to a polar representation, only to return to Cartesian components for the following processing stages. For this design, the multiplier-based fourth-power calculator is therefore the considered implementation. It keeps the data path simple, deterministic and easy to pipeline, while avoiding the additional latency and format conversions of a CORDIC-based solution.

4.3.3 Spread estimation

This subsection describes the part of the spread-estimation backend that calculates the absolute phase-spread estimate. The focus here is the computation from a pair of selected complex samples to the unsigned angle estimate $|\theta|$. The top overview is given in the figure below:

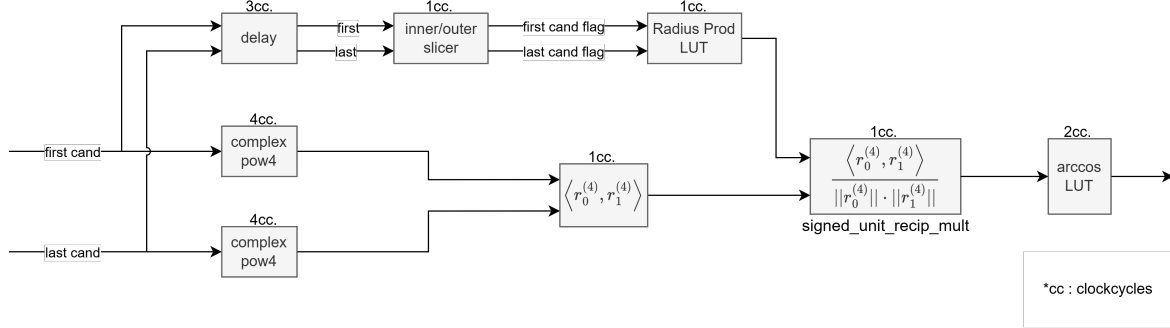


Figure 4.16: Top overview of the spread estimation module

Functional operation

After the selected samples have been raised to the fourth power, the spread-estimation block calculates the angle between the two resulting complex vectors. Let

$$r_0^{(4)} = I_0^{(4)} + jQ_0^{(4)}, \quad r_1^{(4)} = I_1^{(4)} + jQ_1^{(4)}.$$

where $I_x^{(4)}$ and $Q_x^{(4)}$ are the real and imaginary part respectively as also seen from equation 4.26. The numerator of the cosine estimate is the dot product of these two fourth-power vectors:

$$\langle r_0^{(4)}, r_1^{(4)} \rangle = I_0^{(4)} I_1^{(4)} + Q_0^{(4)} Q_1^{(4)}.$$

This dot product is proportional to

$$|r_0^{(4)}| |r_1^{(4)}| \cos(4|\theta|),$$

so the required normalization factor is the reciprocal of the product of the two fourth-power vector magnitudes.

For the 16-QAM constellation used here, the denominator is not computed with a run-time divider. Instead, each sample is sliced as belonging to the inner or outer diagonal radius of the constellation. The two inner/outer flags address a small radius-product LUT. This *radius_prod_LUT* directly returns a precomputed reciprocal multiplication factor for the possible radius products:

$$R_{II}^{-1}, \quad R_{IO}^{-1}, \quad R_{OO}^{-1}.$$

The reciprocal factors are scaled fixed-point constants, so the normalization is implemented as one signed multiplication followed by an arithmetic right shift by the *signed_unit_recip_mult* block. The result is saturated to the interval representable by Q1.31 and represents

$$x = \cos(4|\theta|).$$

The final conversion from x to an angle is performed by an arccosine LUT. Since the desired output is

$$|\theta| = \frac{1}{4} \arccos(x),$$

the LUT stores $\arccos(x)/4$ rather than $\arccos(x)$. Only the half-domain $x \in [0, 1]$ is stored, as marked in Figure 4.17. For negative inputs, the symmetry

$$\frac{\arccos(-x)}{4} = \frac{\pi}{4} - \frac{\arccos(x)}{4}$$

is used to reconstruct the correct value. The output of this stage is therefore the absolute phase-spread estimate $|\theta|$, limited to the range $0 \leq |\theta| \leq \pi/4$.

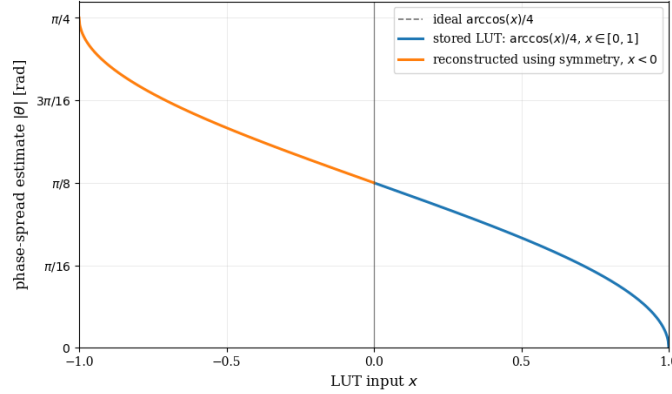


Figure 4.17: $\arccos(x)/4$ LUT symmetry

Data flow

The spread-estimation calculation is implemented as a fixed-point pipeline. The selected input samples are signed G_{sample} -bit values. In the implemented configuration, $G_{\text{sample}} = 16$, so these are 16-bit signed I/Q samples. The fourth-power stage produces signed real and imaginary components with width G_{pow4} . As discussed in the previous subsection, the implemented configuration uses $G_{\text{pow4}} = 70$. These components are kept at this extended precision because the subsequent dot product multiplies two fourth-power values.

The dot-product output has width $2G_{\text{pow4}} + 1$ and is signed, since the angle between the two fourth-power vectors can produce a negative cosine. For the implemented default $G_{\text{pow4}} = 70$, this gives a 141-bit signed dot-product result. In parallel, the delayed original I/Q samples are converted into inner/outer radius flags. The radius-product LUT outputs a signed 48-bit reciprocal factor. The constants are scaled such that multiplying the dot product by the LUT value and shifting right by $G_{\text{inv_frac}}$ fractional bits produces a normalized Q1.31 value. Unlike G_{sample} and G_{pow4} , the value of $G_{\text{inv_frac}}$ is tied to the chosen reciprocal-constant scaling and is therefore kept as a symbolic implementation parameter in this description.

The normalized value is interpreted as a signed Q1.31 representation of $\cos(4|\theta|)$, which corresponds to a 32-bit signed fixed-point word. Before addressing the arccosine LUT, the absolute value is taken. The special case $x = -1$ is saturated to the largest positive Q1.31 value before taking the absolute value, avoiding overflow in two's-complement representation. The LUT address is formed from the upper fractional bits of $|x|$, specifically bits 30 down to 19, giving a 12-bit address and therefore 4096 table entries.

The arccosine LUT output is also Q1.31. It stores $\arccos(|x|)/4$ for non-negative input magnitudes. A delayed sign bit from the original Q1.31 cosine value selects either the direct LUT output or the reconstructed value $\pi/4 - \text{LUT}(|x|)$. Thus the final output is a signed Q1.31 number, but it represents the non-negative angle $|\theta|$.

Timing flow

The block is fully pipelined. Once the pipeline is filled, a new valid sample pair can be accepted every clock cycle, provided both input-valid signals are asserted. The latency from the selected input pair to the $|\theta|$ output is eight clock cycles in the current implementation.

Because the architecture contains two parallel branches, a single cumulative-latency column can be misleading. Table 4.1 therefore reports the cumulative latency separately for each path up to the alignment point. After both branches have been aligned, the remaining stages are shown as a common path. In the table, “Cum.” is path-local before the merge and is measured from the original input pair after the merge.

The fourth-power dot-product path and the denominator-reciprocal path are explicitly balanced. The denominator path starts with a three-clock delay of the original I/Q samples so that its reciprocal factor becomes valid in the same clock cycle as the dot product. Both branches therefore align after five clock

Table 4.1: Pipeline latency of the absolute spread-estimation calculation.

Path	Block	+ clk	Cum.
Dot-product path	Fourth-power pipeline	4 clk	4 clk
Dot-product path	Dot product	1 clk	5 clk
Denominator path	I/Q delay for denominator path	3 clk	3 clk
Denominator path	Inner/outer flag generation	1 clk	4 clk
Denominator path	Radius-product reciprocal LUT	1 clk	5 clk
Common after merge	Reciprocal multiplication and saturation	1 clk	6 clk
Common after merge	Arccos LUT address, BRAM read and sign alignment	2 clk	8 clk

cycles. The reciprocal multiplication stage only asserts its output-valid signal when both the dot product and the reciprocal factor are valid. The arccosine stage then adds two clocks, matching the LUT read latency and the delayed sign/valid signals used for half-domain reconstruction.

4.3.4 Sign estimation

The absolute spread-estimation path determines the magnitude $|\theta|$. A separate sign-estimation path determines whether this magnitude must be applied as a positive or negative phase estimate. This subsection describes the blocks that generate this final sign decision and align it with the absolute-angle output. An overview of this topology can be seen in Figure 4.18.

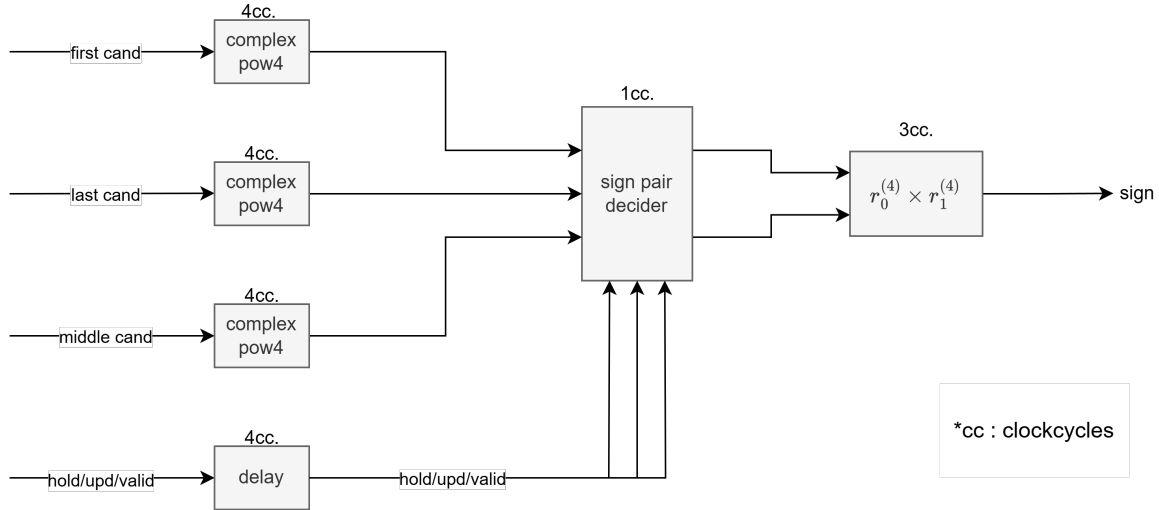


Figure 4.18: Top overview of the sign estimation module

Functional operation

The sign estimator uses the orientation of two fourth-power complex vectors. For two selected fourth-power samples

$$r_0^{(4)} = I_0^{(4)} + jQ_0^{(4)}, \quad r_1^{(4)} = I_1^{(4)} + jQ_1^{(4)},$$

the sign is determined from the two-dimensional cross product

$$r_0^{(4)} \times r_1^{(4)} = I_0^{(4)}Q_1^{(4)} - Q_0^{(4)}I_1^{(4)}.$$

This quantity is proportional to

$$|r_0^{(4)}| |r_1^{(4)}| \sin(\phi_1 - \phi_0),$$

so its sign gives the rotation direction between the two fourth-power vectors. A positive cross product corresponds to one rotation direction, while a negative cross product corresponds to the opposite direction. Only the sign of $r_0^{(4)} \times r_1^{(4)}$ is needed; the magnitude of the cross product is discarded.

The sign path receives three candidate symbols: first, middle and last. The sign-pair decision block selects which adjacent pair should be used for the sign estimate. If the first and middle candidates are both marked as updated, the sign is calculated from the first–middle pair. Otherwise, if the middle and last candidates are both marked as updated, the sign is calculated from the middle–last pair. If neither update combination is present, the previously selected pair is held.

Once the pair has been selected, the cross-product sign block computes the two products $I_0^{(4)}Q_1^{(4)}$ and $Q_0^{(4)}I_1^{(4)}$, subtracts them, and encodes the sign as a two-bit value. In the final sign-application block, the code "11" is interpreted as a positive sign and "01" as a negative sign. The signed angle is then generated as

$$\theta = \begin{cases} -|\theta|, & \text{if the sign code is "01",} \\ |\theta|, & \text{otherwise.} \end{cases}$$

When the optional sign filter is enabled, the raw sign decisions are passed through a rolling majority filter. This filter stores the last $2^{G_{\log 2_signwin}}$ sign decisions and outputs the majority sign. In case of an exact tie, the previous output sign is retained. This avoids rapid sign toggling when consecutive estimates are close to the decision boundary.

Data flow

The sign estimator operates on the same fourth-power representation used by the absolute-angle calculation. The first and last fourth-power values are exposed by the absolute spread-estimation backend, while the middle symbol is passed through an additional fourth-power block. Each fourth-power vector consists of signed real and imaginary components with width G_{pow4} . In the implemented configuration, $G_{\text{pow4}} = 70$, so both the real and imaginary fourth-power components are 70-bit signed values.

The update and hold control signals from the candidate-selection logic are Boolean control signals. Since the fourth-power pipeline has a latency of four clock cycles, these control signals are delayed by the same amount before they are used by the sign-pair decision block. This ensures that the delayed update decisions refer to the fourth-power samples currently present at the sign-pair selector inputs.

The selected sign pair consists of two signed complex vectors:

$$(I_0^{(4)}, Q_0^{(4)}) \quad \text{and} \quad (I_1^{(4)}, Q_1^{(4)}).$$

The cross-product block multiplies two G_{pow4} -bit signed values, producing two signed products of width $2G_{\text{pow4}}$. With $G_{\text{pow4}} = 70$, each product is therefore 140 bits wide. The subtraction is performed with one additional bit, giving a signed difference width of $2G_{\text{pow4}} + 1$. In the implemented configuration this corresponds to a 141-bit signed difference. Only the sign of this difference is encoded at the output.

The raw sign output is a two-bit code. The final theta-signing block interprets "01" as a negative sign and negates the Q1.31 absolute-angle value in that case. All other sign codes produce a positive output. The resulting signed angle remains in the same fixed-point format as the absolute-angle input.

Timing flow

The sign path is timed to match the absolute-angle pipeline. The fourth-power outputs are available after four clock cycles. The update and hold control signals are therefore delayed by four clock cycles before pair selection. The pair selector registers the selected pair, the cross-product sign block uses two pipeline stages, and the sign output is delayed by one additional clock. This gives an eight-clock latency from the candidate-symbol inputs to the raw sign output, matching the eight-clock latency of the $|\theta|$ path.

Here too, two branches exist before alignment: the fourth-power data path and the delayed update/hold control path. Table 4.2 therefore shows separate cumulative latencies up to the pair-selection stage, followed by the common post-alignment stages. As above, "Cum." is path-local before the merge and is measured from the original input pair after the merge.

The sign-estimation backend contains an assertion that checks whether the raw sign-valid signal and the absolute-angle valid signal are aligned. The data path and control path are aligned after four clock cycles, after which the remaining four stages are shared. After optional filtering, the sign majority filter produces one output for each valid input sign. The final sign-application block then waits until both the absolute-angle valid signal and the sign-valid signal are asserted in the same clock cycle before producing the signed θ output.

Table 4.2: Pipeline latency of the sign-estimation calculation.

Path	Block	+ clk	Cum.
Data path	Fourth-power pipeline	4 clk	4 clk
Control path	Hold/update control delay	4 clk	4 clk
Common after merge	Sign-pair selection register	1 clk	5 clk
Common after merge	Cross-product products	1 clk	6 clk
Common after merge	Cross-product subtraction and sign encoding	1 clk	7 clk
Common after merge	Sign alignment delay	1 clk	8 clk

4.3.5 Filters

Two optional filters are placed after the raw spread-estimation backend. The first filter smooths the absolute angle estimate, while the second filter smooths the sign decision. They are implemented separately because the two signals have different data types: the angle magnitude is a fixed-point numerical value, whereas the sign is a two-bit decision.

Magnitude moving-average filter

The magnitude filter is a rolling moving average over the Q-format angle estimate. Its window length is constrained to a power of two:

$$N_{\text{avg}} = 2^{G_{\log 2_{\text{avg}}}}.$$

This makes the final division by the window length a right shift instead of a general divider to reduce computational complexity. The filter stores the last N_{avg} input samples in a circular buffer and maintains a running sum. For every new valid input sample $x[n]$, the oldest sample in the buffer is subtracted and the new sample is added:

$$S[n] = S[n - 1] + x[n] - x[n - N_{\text{avg}}].$$

The filtered output is then

$$y[n] = \frac{S[n]}{N_{\text{avg}}},$$

implemented as an arithmetic right shift by $G_{\log 2_{\text{avg}}}$ bits.

The accumulator width is increased by $G_{\log 2_{\text{avg}}}$ bits compared to the input width. This provides enough range to hold the sum of N_{avg} signed input samples without overflow under normal fixed-point assumptions. If $G_{\log 2_{\text{avg}}} = 0$, the window length is one and the filter becomes a direct pass-through.

At start-up, the first valid sample is replicated across the entire logical window. This means the first filtered output is equal to the first input sample, instead of requiring a long fill period before a valid average can be produced. After this initialization, every valid input produces one valid filtered output.

Sign rolling-majority filter

The sign filter performs a rolling majority vote over the raw sign decisions, a graphical overview of its functional principles are given in Figure 4.19. Its window length is also a power of two:

$$N_{\text{sign}} = 2^{G_{\log 2_{\text{signwin}}}}.$$

The incoming sign is encoded as a two-bit value, where "11" represents a positive sign and "01" represents a negative sign. Internally, the filter stores only one bit per sign decision: positive signs are stored as 1 and negative signs as 0.

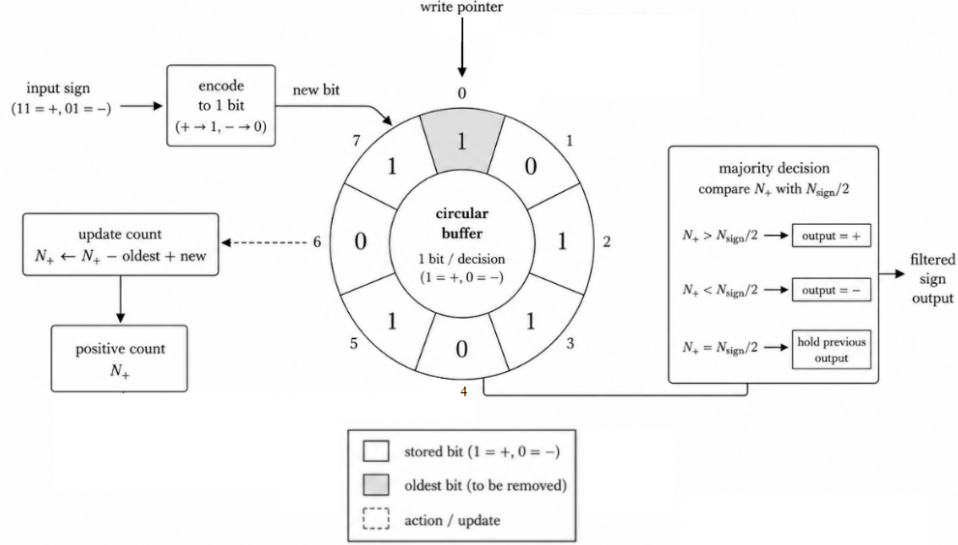


Figure 4.19: Sign rolling-majority filter with circular buffer

As with the moving-average filter, the sign filter uses a circular buffer. Instead of storing a numerical sum of signed values, it keeps a count of the number of positive signs currently present in the window. For every valid input sign, the oldest stored bit is removed from the count and the new sign bit is added. The output is then decided by comparing the positive count with half the window length:

$$\text{sign}_{\text{out}} = \begin{cases} +, & \text{if } N_+ > N_{\text{sign}}/2, \\ -, & \text{if } N_+ < N_{\text{sign}}/2, \\ \text{previous sign}, & \text{if } N_+ = N_{\text{sign}}/2. \end{cases}$$

The tie case is handled by holding the previous output sign. This prevents the filtered sign from toggling when the window contains an equal number of positive and negative decisions.

At reset, the sign-memory contents and output are initialized to the positive sign. On the first valid input, the whole logical window is initialized with that first sign, so the first output follows the first real sign decision. If $G_{\log 2_ \text{signwin}} = 0$, the window length is one and the filter becomes a direct sign pass-through.

Timing and alignment

Both filters are streaming filters: after initialization, each valid input produces one valid output. The moving-average filter and the sign-majority filter each register their output, so they add one clock cycle of processing latency relative to their valid input. Because both filtered paths are driven from already aligned raw backend outputs, using the same valid-driven structure keeps the filtered magnitude and filtered sign aligned for the final signed- θ generation.

4.3.6 Magnitude and Sign integration

After filtering, the angle estimate is reconstructed by combining the filtered magnitude with the filtered sign decision. The magnitude filter provides the absolute value $|\theta_{\text{flt}}|$, while the sign filter determines whether this magnitude should be interpreted as a positive or negative angle. The final signed estimate is therefore formed as

$$\theta_{\text{out}} = \begin{cases} |\theta_{\text{flt}}|, & \text{if } \text{sign}_{\text{flt}} = +, \\ -|\theta_{\text{flt}}|, & \text{if } \text{sign}_{\text{flt}} = -. \end{cases}$$

At last, the output valid signal is asserted only when both filtered components are valid, producing one aligned signed- θ value for each accepted raw estimate.

4.4 Phase unwrapper

The phase unwrapper receives the wrapped centre-phase estimate from the SBPS chain and the signed phase-spread estimate from the SE block. In the implemented system this logic is part of the *PUPC* block, but it is described separately here because it forms the stateful boundary between the blockwise estimator and the final phase compensator. The top overview of the implemented hardware with annotated timing is illustrated below.

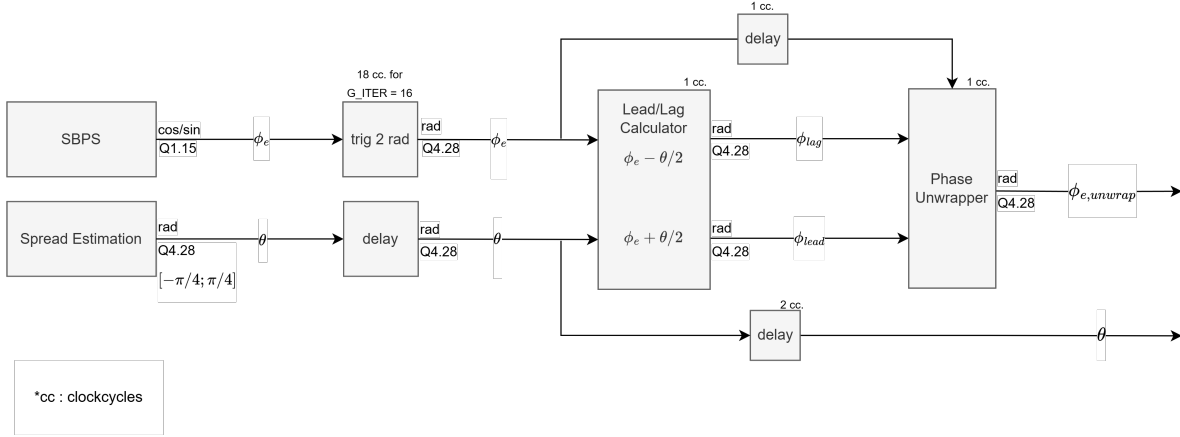


Figure 4.20: Top overview of phase unwrapper block

4.4.1 Functional operation

The RTL hierarchy is split into two modules. The *phase_unwrapper_top* module first reconstructs the leading and lagging edge phases of the current block, while the *phase_unwrapper* module performs the actual branch tracking. The edge reconstruction is implemented by the *lead_lag_calculator*:

$$\phi_{\text{lead}} = \phi_e + \frac{\theta}{2}, \quad (4.31)$$

$$\phi_{\text{lag}} = \phi_e - \frac{\theta}{2}. \quad (4.32)$$

Here, ϕ_e is the wrapped centre phase in Q4.28 and θ is the signed spread estimate in the same format. The division by two is implemented as an arithmetic right shift, so the Q-format is preserved.

For every valid block after the first one, the unwrapper compares the current lagging edge with the stored leading edge of the previous block:

$$d_b = \phi_{\text{lag},b} - \phi_{\text{lead},b-1}. \quad (4.33)$$

If this difference crosses the $\pm\pi/4$ decision threshold, the two-bit unwrap counter is updated by one quadrant. A difference larger than or equal to $+\pi/4$ decrements the counter, while a difference below $-\pi/4$ increments it. The counter is stored modulo four, because the final constellation is unchanged by an additional 2π rotation. The counter value selects one of four Q4.28 offsets:

$$0, \quad \frac{\pi}{2}, \quad \pi, \quad \frac{3\pi}{2}.$$

This offset is added to the delayed centre phase to produce the unwrapped centre phase. The first valid block is passed through without a correction, because no previous leading edge is available yet. The current leading edge is then stored for the next block.

4.4.2 Data flow

All phase quantities inside the unwrapper are signed 32-bit Q4.28 words. The constants $\pi/4$, $\pi/2$, π , and $3\pi/2$ are stored directly in the same representation. The branch state itself is only a two-bit unsigned value, exposed at the PUPC boundary as *dbg_pu_idx* for simulation and debugging.

The *phase_unwrapper_top* module delays the centre phase by one clock cycle so that it is aligned with the lead and lag outputs of the *lead_lag_calculator*. The unwrapping decision therefore receives three aligned values: the current leading edge, the current lagging edge and the current centre phase. The output remains a Q4.28 centre-phase word, so no conversion is required before the section-coefficient generator.

4.4.3 Timing flow

The lead/lag calculation is registered and contributes one clock cycle. The centre phase is delayed by the same amount inside *phase_unwrapper_top*. The *phase_unwrapper* then performs the edge-difference comparison, counter update, offset selection and output registration in the following cycle. The unwrapped phase is therefore available two clock cycles after a valid wrapped phase and spread estimate enter *phase_unwrapper_top*.

The surrounding *phase_unwrap_comp_top* module delays θ by the same two clock cycles before passing it to the section-phase generator. This keeps the spread estimate aligned with the unwrapped centre phase used for the final compensation coefficients.

4.5 Phase compensation block

The phase compensation block forms the final stage of the carrier-recovery datapath. It combines the unwrapped centre phase, the signed spread estimate and the delayed received symbol block to generate one CORDIC rotation phase per symbol. In the RTL this functionality is implemented by the *PUPC* hierarchy. The top-level *PUPC* module (see Figure 4.21) first converts and aligns the incoming signals, after which the *phase_correction_rotate_top* module generates the per-symbol phases and rotates the complete block.

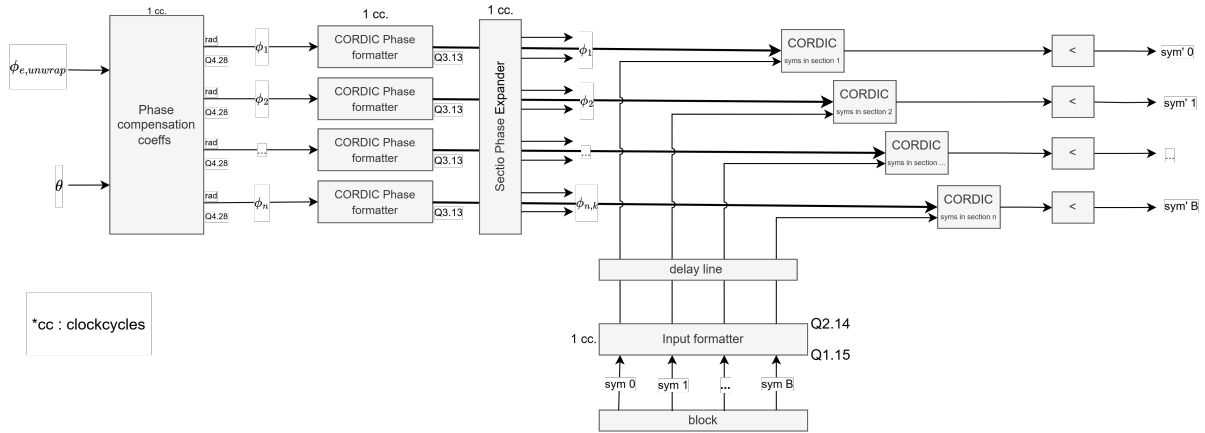


Figure 4.21: Top overview of phase compensation block

4.5.1 Functional operation

The SBPS chain represents its selected centre phase as Q1.15 cosine and sine values. Since the phase unwrapper and compensation logic operate on explicit phase words, the *PUPC* top level converts these two trigonometric components to a signed Q4.28 angle using the *trig_2_rad* module, as seen in Figure 4.20. The signed spread estimate θ already arrives as Q4.28 and is therefore only delayed to match the *trig_2_rad* latency. The input I/Q block is delayed by the same amount so that samples, centre phase and spread estimate enter the correction stage in the same clock cycle.

After phase unwrapping, the *phase_compensation_coeffs* module generates one representative phase per compensation section. The section coefficients are computed at elaboration time from

$$c_n = \frac{2n + 1 - N_{\text{sec}}}{2N_{\text{sec}}}, \quad n = 0, \dots, N_{\text{sec}} - 1, \quad (4.34)$$

and stored as signed fixed-point constants with $G_{\text{COEFF_FRAC}}$ fractional bits. For each valid block, every section phase is then computed as

$$\phi_n = \tilde{\phi}_e + c_n\theta. \quad (4.35)$$

The multiplication $c_n\theta$ is followed by an arithmetic right shift by $G_{\text{COEFF_FRAC}}$ bits, returning the result to the Q4.28 phase format before it is added to the unwrapped centre phase.

The generated section phases are converted to the CORDIC phase format and expanded to per-symbol phases. The *section_phase_expander* assumes that $G_{\text{NUM_SYM}}$ is evenly divisible by G_{NSEC} , so each section contains

$$G_{\text{SYM_PER_SEC}} = \frac{G_{\text{NUM_SYM}}}{G_{\text{NSEC}}}$$

symbols. For symbol index k , the applied phase is selected from section $k/G_{\text{SYM_PER_SEC}}$. The resulting per-symbol phases drive a parallel *cordic_rotate_bank*, which instantiates one Xilinx CORDIC rotator per symbol.

4.5.2 Data flow

The PUPC datapath uses three phase representations. The SBPS output enters as Q1.15 sine and cosine. The internal unwrapping and section-generation path uses signed Q4.28 phase words. Finally, the phases passed to the CORDIC IP are converted to Q3.13. During this conversion, the Q4.28 section phase is first wrapped into the interval $[-\pi, \pi]$, then rounded by adding or subtracting 2^{14} , and finally shifted right by 15 bits.

The I/Q samples enter the block as separate packed Q1.15 buses. Before the CORDIC bank, the *input_reformatter* converts every symbol to the CORDIC Cartesian format by shifting Q1.15 to Q2.14. The per-symbol Cartesian word is packed as $\{X, Y\}$. After rotation, the CORDIC outputs are interpreted as Q2.14 and shifted left by one bit to return to Q1.15. The output block is again presented as separate packed I and Q buses.

The implemented Vivado IP instance is parameterized with $G_{\text{NUM_SYM}} = 32$, $G_{\text{NSEC}} = 16$, $G_{\text{SAMPLE_WIDTH}} = 16$, $G_{\text{PHASE_WIDTH}} = 32$, $G_{\text{COEFF_WIDTH}} = 16$, $G_{\text{COEFF_FRAC}} = 14$, $G_{\text{PHASE_OUT_WIDTH}} = 16$, and $G_{\text{DELAY}} = 18$. These generics keep the block-size, section count and fixed-point formats explicit, while allowing the same hierarchy to be regenerated for other sweep configurations.

4.5.3 Timing flow

At the PUPC top level, the *atan2* conversion defines the initial alignment point. The RTL uses an 18-cycle delay for both the Q4.28 spread estimate and the packed I/Q symbol block, matching the *atan2* output latency. The valid signal into the phase-correction backend is asserted only when the *atan2* output, delayed theta and delayed symbol block are all valid.

Inside *phase_correction_input_top*, the phase unwrapping and section-phase generation require three clock cycles: two cycles for *phase_unwrapper_top* and one cycle for *phase_compensation_coeffs*. The I/Q block is delayed by three cycles in parallel. The *input_reformatter* then adds one registered conversion cycle, and the *section_phase_expander* adds one further cycle. The Cartesian path is delayed by one cycle after reformatting so that it remains aligned with the expanded phase vector.

Table 4.3: Pipeline latency before the CORDIC rotation bank in the PUPC backend.

Stage	+ clk	Cum. after PUPC alignment
Phase unwrapping	2 clk	2 clk
Section-phase coefficient generation	1 clk	3 clk
I/Q and phase reformatting	1 clk	4 clk
Section-to-symbol phase expansion and Cartesian alignment	1 clk	5 clk

After this point, the latency is determined by the configured Xilinx CORDIC IP. All CORDIC instances in the bank receive the same valid pulse and use the same configuration, so the bank output-valid signal is taken from the first CORDIC output valid. The rotated samples are registered when the CORDIC output becomes valid and are then exposed at the PUPC output as the corrected block.

4.6 Top-design overview

The previous sections described the main processing blocks individually. The top design combines these blocks into a single blockwise carrier-recovery datapath. Its purpose is not to introduce a new estimation algorithm, but to connect the already discussed SBPS, SE, phase unwrapping and phase-compensation blocks while preserving the correct timing relation between their outputs.

The implemented wrapper is built around one received block interface. The in-phase and quadrature samples enter as two packed buses, each containing $G_{\text{NUM.SYM}} = 32$ signed Q1.15 samples. From this common input, the datapath splits into three parallel branches:

1. the SBPS chain estimates the wrapped centre phase of the block,
2. the SE block estimates the signed phase spread over the block,
3. an explicit I/Q delay line preserves the original input block until the phase estimates are available.

This structure is important because the spread estimator in the implemented top design operates on the original received block, not on a block that has already been derotated by the SBPS result. The SBPS and SE branches therefore both observe the same noisy, phase-impaired input block. Their outputs describe complementary aspects of that same block: ϕ_e is the wrapped centre-phase estimate, while θ is the signed phase-spread estimate. An overview is given in the following figure:

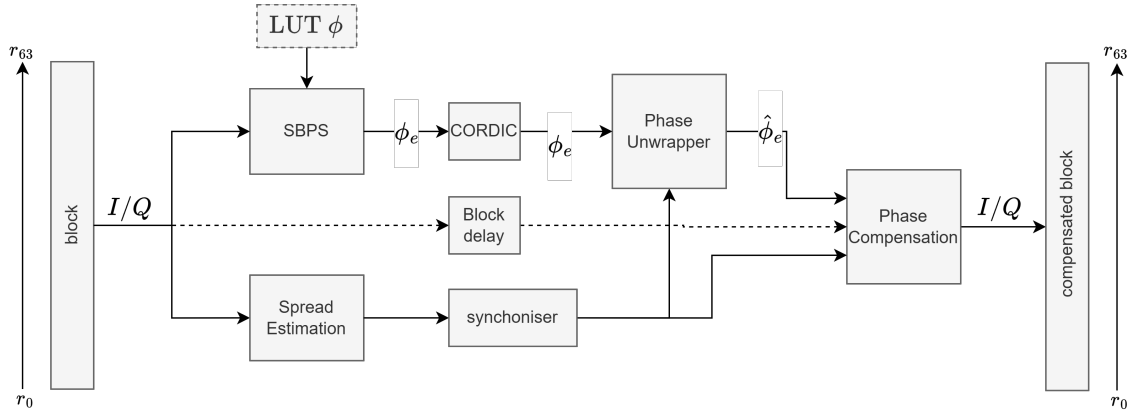


Figure 4.22: Top overview implemented algorithm

4.6.1 Functional dataflow

At the top-design boundary, `in_valid` is distributed to both the SBPS chain and the SE block. The SBPS chain processes the full block through its serial refinement stages and outputs the selected phase estimate as trigonometric coefficients. Internally these coefficients leave the SBPS IP as Q1.31 cosine and sine values. Before they enter the *PUPC* block, the *trigQ31.to.Q15* adapter truncates them to Q1.15 by dropping the lower fractional bits (Figure 4.3). This adapter is purely combinational, so it does not alter the valid timing of the SBPS output.

In parallel, the SE block outputs the signed spread estimate θ directly as a 32-bit Q4.28 phase word. This representation matches the downstream phase-unwrapping and phase-compensation logic, so no numerical format conversion is needed at the top-design boundary. Only a timing-alignment delay is inserted before θ is passed to *PUPC*.

The third branch contains the raw I/Q block delay. This branch carries the original Q1.15 samples, still packed as separate I and Q buses. The delayed block is the block that *PUPC* finally rotates. Consequently, the final compensation stage applies the centre phase and phase-spread information to the same sample block from which these estimates were derived.

Table 4.4: Main data streams at the top-design boundary.

Stream	Representation	Top-design role
<i>I/Q</i> input block	32×16 -bit Q1.15, packed per component	Common input to SBPS, SE and the sample-delay branch
SBPS phase	Q1.31 $\cos(\phi_e)$, $\sin(\phi_e)$, truncated to Q1.15	Wrapped centre-phase estimate for <i>PUPC</i>
SE spread	Signed 32-bit Q4.28 phase word	Signed spread estimate θ , delayed to match the SBPS branch
Delayed <i>I/Q</i> block	32×16 -bit Q1.15, packed per component	Original sample block supplied to the final rotation stage
Corrected <i>I/Q</i> block	32×16 -bit Q1.15, packed per component	Output of the <i>PUPC</i> correction path

After these top-level alignment operations, *PUPC* performs the remaining phase-domain processing. First, the Q1.15 trigonometric SBPS result is converted to a Q4.28 phase word using the `atan2` path described in the phase-compensation section. The delayed θ value and delayed *I/Q* block are aligned to this conversion latency inside *PUPC*. The phase unwrapper then combines the wrapped centre phase and signed spread estimate to maintain the correct $\pi/2$ branch across successive blocks. Finally, the compensation-coefficient generator expands the block-level phase description into section phases and then to per-symbol CORDIC rotation phases.

4.6.2 Timing and alignment

The top design uses fixed-latency valid pulses rather than a back-pressure handshake. The `in_valid` pulse launches the SBPS and SE branches at the same clock edge, and every following alignment decision is expressed as a deterministic delay. This is why the wrapper contains two explicit alignment blocks in addition to the algorithmic modules.

The first alignment block is the *synchroniser*. It delays the signed Q4.28 spread estimate until it is temporally aligned with the SBPS centre-phase estimate. Its delay is parameterised as

$$D_{\text{sync}} = G_{\text{SBPS_STAGES}} G_{\text{SBPS_STAGEDELAY}} - G_{\text{SE_DELAY}}. \quad (4.36)$$

The assertion inside this block checks that the configured SBPS latency model is not shorter than the configured SE latency. If both branches are already aligned, the block becomes a pass-through; otherwise, both the valid pulse and θ data word are shifted through the same delay line. In the generated block design, the SBPS valid pulse is used as the master valid into *PUPC*, while the synchronised θ data is routed to the *PUPC* theta input. This makes the top-level timing contract explicit: θ must be delayed until it is valid on the same cycle as the SBPS output coefficients.

The second alignment block is the *iq_block_delay*. It delays the full packed *I/Q* block by the same configured SBPS latency model:

$$D_{\text{IQ}} = G_{\text{SBPS_STAGES}} G_{\text{SBPS_STAGEDELAY}}. \quad (4.37)$$

This delay is needed because the samples themselves do not pass through the SBPS or SE estimation pipelines before correction. Without the delay, *PUPC* would receive a phase estimate belonging to an older block while rotating the newest input block. By delaying the raw block at the top level, the final rotation stage receives three mutually aligned quantities: the wrapped centre-phase estimate, the signed spread estimate and the corresponding received samples.

There is one additional alignment boundary inside *PUPC*. The SBPS phase enters *PUPC* as Q1.15 sine and cosine, whereas the unwrapper and compensation logic operate on Q4.28 phase words. The `atan2` conversion therefore defines a local timing point inside *PUPC*; the spread estimate and the *I/Q* block are delayed by the same conversion latency before entering the phase-unwrapping and section-generation backend. This second alignment step is local to the compensation block, while the *synchroniser* and *iq_block_delay* solve the inter-block timing at the top-design level.

Functionally, the top design can therefore be viewed as a three-input merge into *PUPC*: a wrapped centre phase from SBPS, a signed phase spread from SE, and the delayed original sample block. The additional wrapper blocks do not change the phase-recovery algorithm itself; they ensure that the different numerical representations and pipeline latencies meet at the same block boundary.

Verification and simulation model

5.1 Overview

The verification flow is built around a Python golden model and a set of vector-driven VHDL testbenches. The purpose of this flow is not only to check whether the final architecture produces corrected 16QAM symbols, but also to isolate errors in the individual processing blocks before they are integrated into the complete phase recovery chain.

The general simulation flow is shown conceptually in Figure 5.1 with:

1. Python generates deterministic input vectors and the corresponding reference data.
2. The generated vectors are quantized to the fixed-point formats used by the RTL.
3. A VHDL testbench reads the vectors from text files and applies them to the design under test (DUT).
4. The testbench either checks the result internally or writes an output vector file.
5. Python post-processing scripts compare the RTL output with the golden model or clean reference symbols.

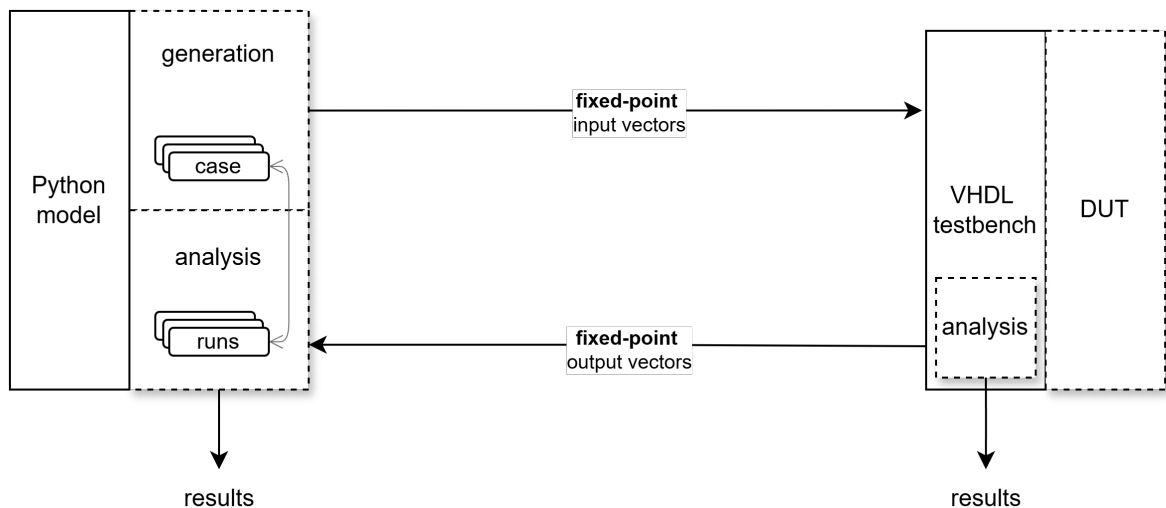


Figure 5.1: Verification pipeline

This separation between vector generation, RTL simulation and output analysis makes the verification reproducible. Each input file fixes the random seeds, channel impairments and block size used for a test

case. The same vector can therefore be rerun after changes to the RTL, and the resulting output can be compared with the same Python reference.

Two verification levels are used:

- block-level verification, where individual datapath modules are checked against simple analytical or Python-generated references,
- and system-level verification, where the complete phase-recovery wrapper is simulated using impaired 16QAM symbol streams.

The block-level tests are mainly used to verify arithmetic behaviour, fixed-point scaling, valid-signal alignment and corner cases. The system-level tests are used to verify that all blocks remain correctly aligned after integration and that the recovered output stream can be decoded against the transmitted bit sequence. The quantitative results of these simulations are discussed in Chapter 6; this chapter describes the simulation model and verification procedure.

5.2 Test vector generation

The Python side of the verification environment is contained in the `phase_recovery` folder. The scripts are grouped by purpose into generation, analysis, plotting and utility modules. The generation scripts create deterministic input files for the RTL testbenches, while the analysis scripts interpret the output files produced by simulation.

5.2.1 Golden model structure

The Python golden model follows the same block structure as the RTL implementation. The most important algorithmic modules are:

- `bps_serialised.py`, which models the serialized binary phase search,
- `spread_16qam.py`, which models the spread-estimation calculation,
- `phase_unwrap.py`, which models the edge-continuity phase unwrapper,
- and `compensate.py`, which models section-based phase compensation.

These modules operate in floating point and are used to generate expected behaviour before the RTL fixed-point effects are introduced. For vector generation, the floating-point values are quantized to the fixed-point formats used by the hardware. This makes the generated reference vectors suitable for direct comparison with the RTL datapaths.

5.2.2 Block-level vectors

Separate vector generators are used for the major processing blocks. For example, the SBPS stage vectors contain the input symbol block, the previous phase estimate, the stage test angle and the expected selected output phase. The phase-unwrapper vectors contain Q4.28 centre-phase and spread estimates together with the expected unwrapped phase. The spread-estimation vectors contain selected symbols and reference values for fourth-power, dot-product, arccosine and sign-related subblocks.

The generated files are text based. Comment lines start with `#`, and data lines contain integer fixed-point values. This keeps the interface simple for VHDL `textio` testbenches and makes the files easy to inspect manually. A typical block-based input line has the form:

```
block_id I0 Q0 I1 Q1 ... I(B-1) Q(B-1)
```

Additional reference fields are appended for module-specific tests. For example, an SBPS stage vector also contains the expected sine and cosine values of the selected phase, while a phase-unwrapper vector contains the expected Q4.28 unwrapped phase. Including these additional reference field allow for simple output analysis without leaving the VHDL testbench environment.

5.2.3 System-level cases

For full-chain verification, complete impaired 16QAM cases are generated by the input-generation script. This script first generates a random Gray-coded bit sequence, maps the bits to scaled 16QAM symbols, applies channel impairments and writes the resulting vectors to a case directory. The general lay-out can be seen in Figure 5.2.

Each case directory contains:

- `impaired_16qam_q15.txt`, the Q1.15 input stream for RTL simulation,
- `golden_symbols_q15.txt`, the clean transmitted reference symbols,
- `golden_bits_gray.txt`, the transmitted Gray-coded bit sequence,
- and `metadata.json`, which records the generation parameters.

The metadata file stores the block length, number of symbols, baud rate, SNR, residual CFO, linewidth-related parameters, random seeds and constellation scaling. This is important because the verification results must be traceable to the exact channel condition and symbol sequence that produced them.

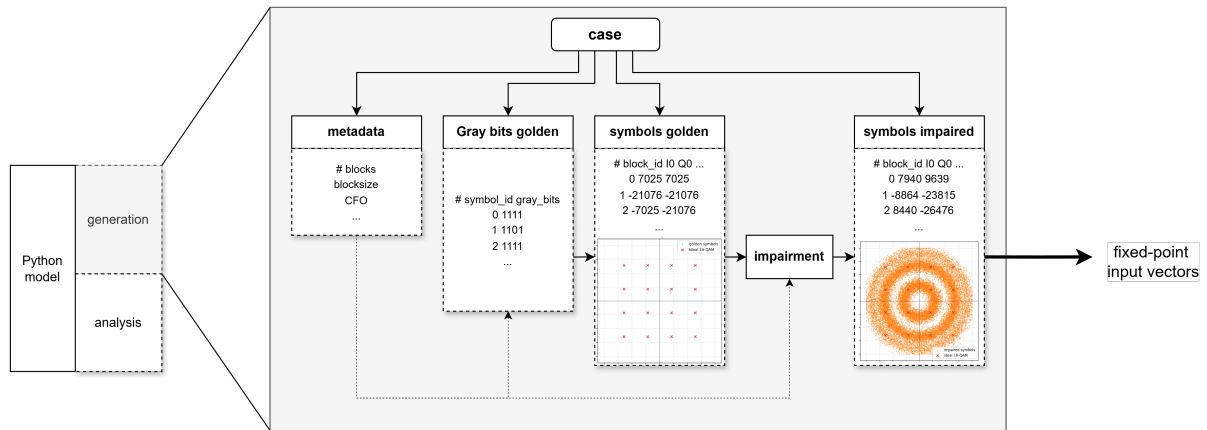


Figure 5.2: Case lay-out

5.3 VHDL testbench setup

The VHDL testbenches are split in the same way as the RTL projects. The SBPS, SE, phase unwrapper, phase correction and top-level wrapper each have dedicated simulation files. Some of these testbenches are self-checking and stop on assertion failures, while others write output vectors that are analysed by Python afterwards.

5.3.1 Block-level testbenches

The block-level testbenches are used during development to verify one module or one small group of modules at a time. Examples include:

- `tb_sbps_chain_generic_vectors`, which applies generated SBPS vectors to the serialized search chain,
- `tb_spread_estimation_backend`, which verifies the absolute spread-estimation backend,
- `tb_phase_unwrapper_top_vectors`, which checks the phase unwrapper against Q4.28 reference vectors,
- `tb_phase_compensation_coeffs`, which verifies the generated section phase words,
- and `tb_cordic_rotate_bank_from_vectors`, which checks the parallel CORDIC rotation output.

These testbenches generally follow the same structure. After reset, they read one vector line, drive the corresponding input signals for one valid cycle, wait for the expected output-valid pulse and then compare the result against the reference. For deterministic arithmetic blocks, exact integer comparison is used. For CORDIC-based rotations, a small integer tolerance is allowed because the Xilinx CORDIC IP introduces its own rounding behaviour.

Valid-signal timing is checked explicitly in several testbenches. This is especially important for the spread-estimation and phase-correction paths, where data and control signals travel through different latency branches before they are merged again. A mismatch in valid alignment can produce numerically plausible but block-shifted results, so the testbenches verify both the output values and the cycle in which they appear.

5.3.2 Top-level wrapper testbench

The system-level simulation uses `tb_wrapper_from_vectors`. This testbench reads the impaired Q1.15 input stream generated by Python and applies one input block to the complete Vivado block-design wrapper for each `in_valid` pulse. The testbench writes the corrected output stream to a text file using the same block format as the input:

```
block_id I0 Q0 ... I(B-1) Q(B-1)
```

The input and output file paths are passed to the testbench through generics. This allows the same compiled testbench to be reused for multiple simulation cases without editing the VHDL source. The wrapper currently emits an initial all-zero output block; this first output-valid pulse is ignored by the testbench so that the written output file contains only the meaningful corrected blocks.

Simulation batches are automated by the wrapper-run script. It scans the case directory for inputs, compiles the Vivado simulation sources when required, elaborates a separate snapshot for each case with the correct generic file paths and runs the simulation in batch mode. Each run receives its own output directory under the phase-results folder, containing the RTL output vector and associated logs.

5.4 Error analysis

After simulation, Python analysis scripts are used to compare the RTL output against the expected behaviour. The exact error metric depends on the verification level.

5.4.1 Block-level analysis

For individual arithmetic blocks, the analysis is mostly based on fixed-point error. The Python reference produces the expected integer output in the same Q-format as the RTL. The output error can then be expressed either directly in integer LSBs or converted back to radians for phase-related blocks.

For SBPS verification, the relevant checks are the selected branch, the selected effective phase and the decision-directed metric behaviour. The generated vectors make it possible to verify whether a stage selected the same candidate as the golden model. Additional analysis scripts can reconstruct the residual phase error and decision-directed EVM from the vector files.

For spread estimation, the analysis checks the fixed-point fourth-power path, the normalized dot product, the arccosine LUT output, the sign decision and the final signed spread estimate. The absolute magnitude and sign are deliberately analysed separately, because an error in the sign path can produce a correct magnitude with the wrong compensation direction.

For the phase unwrapper, the comparison is performed in Q4.28. The reference sequence generated by Python includes the expected quadrant branch, so the output phase and the debug unwrap index can both be inspected. This makes it possible to distinguish between a numerical phase-word error and an incorrect quadrant update.

For the phase compensator, the analysis compares the generated section phases and the rotated I/Q samples. When a clean reference block is available, the residual phase per symbol can be estimated from the phase difference between the RTL output and the transmitted symbol.

5.4.2 System-level analysis

For full-wrapper runs, the first analysis step is to compare the corrected RTL output with the impaired input and the clean reference constellation. This is used to produce constellation overlays and heatmaps,

which are useful for detecting phase-tracking failures, persistent quadrant slips or residual intra-block rotation.

The second analysis step computes residual phase metrics. The corrected RTL samples are compared with the clean reference symbols block by block. For each block, the residual phase trajectory is estimated and fitted with a centre residual phase and a residual phase spread. This separates a remaining common phase offset from a remaining intra-block slope.

These metrics form the basis for the system-level results in Chapter 6. The overall lay-out of the result analysis structure is given in Figure 5.3.

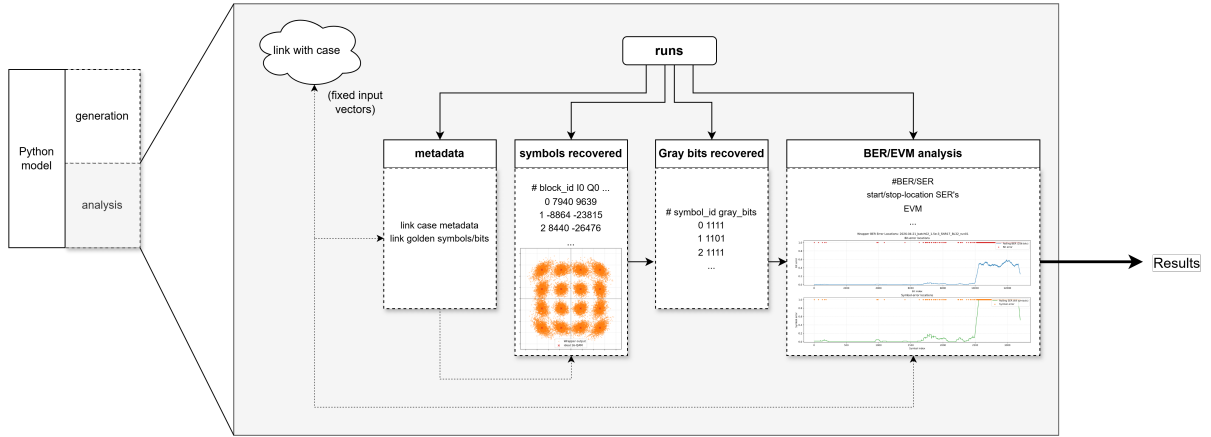


Figure 5.3: Result lay-out

This analysis structure keeps the verification and the final result interpretation separate. Chapter 6 uses the generated reports and plots to discuss phase error, EVM, latency and resource usage, while the present chapter defines how those reports are produced.

This page is intentionally left blank.

6.1 Evaluation scope

The verification methodology, vector format and post-processing flow were described in Chapter 5. This chapter uses the outputs of that flow to evaluate the implemented carrier-recovery design. The evaluation is organised around three questions:

- whether the RTL implementation follows the Python reference behaviour closely enough for system-level interpretation,
- whether the recovered symbols improve under the residual-CFO, linewidth and SNR conditions for which the architecture was designed,
- and how the selected algorithmic parameters relate to hardware cost, latency and timing.

The first part of the chapter therefore checks the main processing blocks against controlled reference cases. The SBPS and SE blocks are tested with deterministic phase sweeps, while the complete receiver is compared against the floating-point Python model for representative channel cases. The second part evaluates correction quality using constellation diagrams, EVM metrics and parameter sweeps.

The main design parameters considered in the sweeps are the number of SBPS refinement stages and the number of phase-compensation sections. The residual CFO and linewidth are varied to test whether the same hardware settings remain useful as the intra-block phase evolution becomes more demanding. Unless stated otherwise, the same transmitted bit sequence and channel seed are used when comparing parameter settings. This makes the curves directly comparable: differences in the plotted metrics are then caused by the receiver configuration rather than by a different noise realisation.

The default simulation settings are given in Table 6.1. The normalized residual-CFO value $\Delta f T_s = 2 \times 10^{-3}$ corresponds to a residual frequency offset of 64 MHz for the assumed 32 GBd symbol rate. Similarly, the normalized linewidth values used in the sweeps can be converted to physical linewidths by dividing the plotted product $\Delta \nu T_s$ by $T_s = 31.25$ ps. These normalized values are used in the plots because they directly determine the accumulated phase variation per symbol.

Table 6.1: Overview of the simulation defaults used for the system-level results

Parameter	Default value
Modulation format	16QAM, Gray-coded
Block size B	32
SBPS stages	6
Radius-directed stages	2
Phase-compensation sections	8
SNR	28 dB
Residual CFO	$\Delta f T_s = 2 \times 10^{-3}$
Linewidth setting	$\Delta \nu T_s = 3 \times 10^{-5}$
Number of compared symbols	6400 per case

6.2 Agreement with the reference model

6.2.1 SBPS

The SBPS block was first evaluated in isolation using a VHDL testbench that exposes the raw fixed-point cosine and sine outputs of the final refinement stage. The stimulus is a deterministic phase sweep: each symbol in the block is rotated by a known phase offset, and the offset is stepped over the test range. No AWGN, linewidth noise or residual-CFO variation is added in this block-level test, so the observed error is dominated by the search procedure and fixed-point arithmetic rather than by channel randomness.

Only inner- and outer-ring 16QAM symbols are used in this idealised test case. This avoids the pattern effect discussed in Section 3.2, where middle-ring symbols can create misleading local minima for the decision-directed metric. As a consequence, this isolated test mainly validates the binary phase-search and fixed-point trigonometric output path. It does not by itself validate the radius-directed early stages, because the selected stimulus deliberately avoids the symbols for which that logic is most relevant. The radius-directed path is instead exercised in the top-level comparison of Subsection 6.2.3.

For this block-level check the testbench uses 12 SBPS refinement stages. From Equation 3.2, the final correction step is approximately 0.011° . This is finer than the stage count used in the final receiver, but it is useful for validation because it separates the intended search behaviour from coarse phase-quantisation effects.

Figures 6.1 and 6.2 compare the RTL trigonometric outputs with the golden reference. The largest relative deviations occur around points where one of the reference components approaches zero, for example the cosine channel near $\pi/2$. In these regions the relative-error metric is poorly conditioned: a small absolute fixed-point difference is divided by a very small reference value and therefore appears as a visible spike. Away from these zero crossings, the relative error remains below approximately 0.25%, indicating that the implemented SBPS output path follows the expected phase estimate.

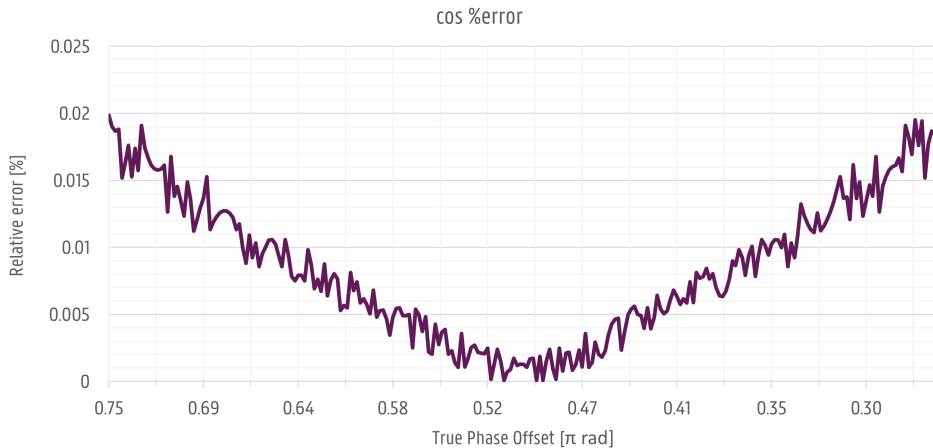


Figure 6.1: Relative RTL-cosine error for phase slope

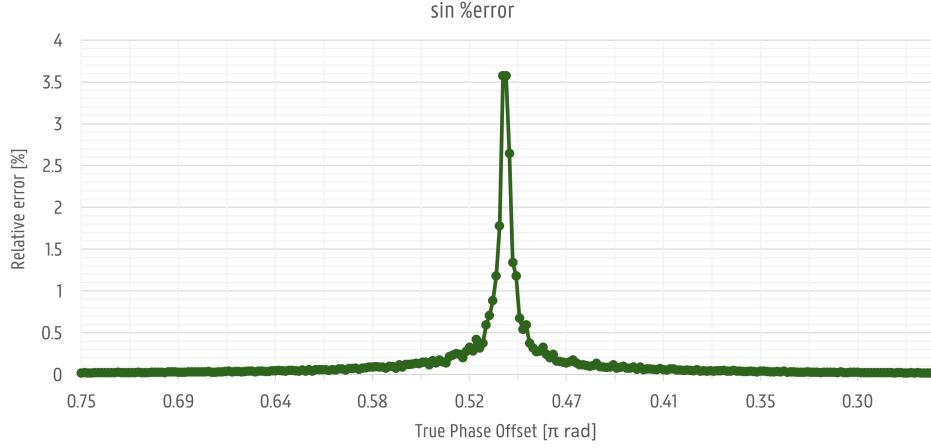


Figure 6.2: Relative RTL-sine error for phase slope

The isolated SBPS test therefore supports the functional correctness of the staged phase-search implementation. The remaining deviations are consistent with fixed-point quantisation and with the limitations of a relative-error plot near zero-valued trigonometric components.

6.2.2 SE

For the spread-estimation block, the final `signed_theta_Q428` output is compared against a known intra-block phase sweep over the interval $[-\pi/4, +\pi/4]$. As for the SBPS block, the plotted quantity is the relative error with respect to the reference sweep.

This test case is also constructed from inner- and outer-ring symbols only. The spread estimator selects representative symbols near the beginning, middle and end of the block. If a region contains only middle-ring symbols, the selector can legitimately hold the previous valid estimate instead of issuing a new candidate. That behaviour is part of the complete input-selection logic, but it would obscure the numerical accuracy of the spread-estimation datapath in this isolated sweep. The deterministic input selector was therefore verified separately, while Figure 6.3 focuses on the arithmetic path from selected symbols to the signed spread estimate.

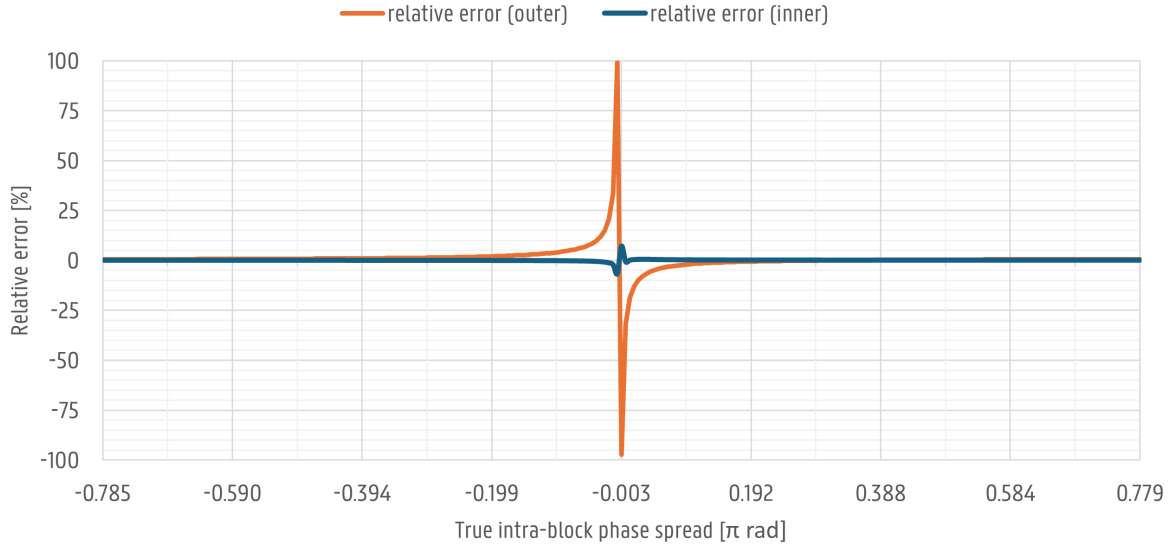


Figure 6.3: Relative RTL error for intra-block phase slope

Figure 6.3 shows that the RTL follows the reference over most of the tested interval, with the largest

relative deviation concentrated around zero phase spread. This is the least favourable point for a relative-error metric, because the reference angle itself approaches zero. In addition, the normalized dot product approaches $x = \cos(4|\theta|) = 1$, where the $\arccos(\cdot)$ mapping is highly sensitive to small fixed-point and LUT-address quantisation errors. A small absolute error in the normalized cosine can therefore become a large relative error in the recovered angle near zero.

The outer-ring-only case shows the most pronounced zero-spread deviation. This is consistent with the use of a separate outer-outer reciprocal factor in the radius-product normalization: near $x = 1$, even a small fixed-point scaling difference becomes visible after the arccosine conversion. Since the effect is localised around zero spread and does not persist over the full range, it is interpreted as endpoint sensitivity of the fixed-point implementation rather than as a wrong sign decision or a systematic spread-estimation bias.

6.2.3 Top-level agreement

Before interpreting the system-level EVM curves, the complete fixed-point RTL output is compared against the Python reference model. The comparison uses the default receiver configuration from Table 6.1: a block size of 32, six SBPS stages, two radius-directed stages and eight compensation sections. Four representative channel cases are evaluated by combining two residual-CFO values with two SNR values, while the linewidth is kept at the default setting.

The purpose is not to prove that every intermediate integer value is identical, because the two implementations do not use exactly the same arithmetic. Instead, the comparison checks whether the RTL produces the same recovered phase behaviour as the reference model within the expected fixed-point error. This also gives an indirect validation of the radius-directed stages. If the radius-directed slicer selected wrong local minima in the early SBPS stages, the resulting centre-phase estimate would differ strongly from the Python model and would appear as a large phase-error difference after compensation.

For each model, the phase error is calculated against the clean reference trajectory. The model difference is then computed as:

$$\Delta\text{ERR} = \text{ERR}_{\text{py}} - \text{ERR}_{\text{RTL}} \quad (6.1)$$

where

$$\begin{aligned} \text{ERR}_m &= \frac{1}{N} \sum_{k=0}^{N-1} e_{m,k}, \\ e_{m,k} &= \phi_{k,\text{ref}} - \hat{\phi}_{k,m}, \quad m \in \{\text{py}, \text{RTL}\}. \end{aligned} \quad (6.2)$$

Figure 6.4 shows this signed model difference as a function of block index for the four residual-CFO/SNR combinations. The traces fluctuate around zero and do not show a persistent drift over the simulated blocks. This is important, because a systematic offset between the Python model and the RTL would indicate either a scaling mismatch, a sign convention error or a fixed delay error in one of the phase paths.

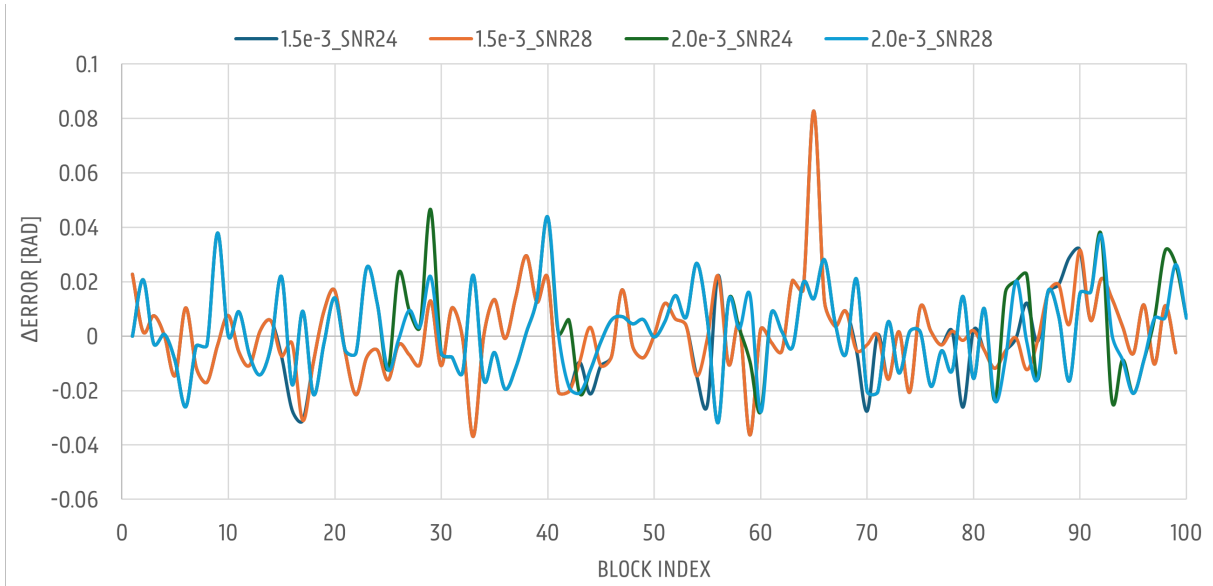


Figure 6.4: Signed phase-error difference between Python reference model and RTL output over block index

The visual observation is confirmed by Table 6.2. For all four cases, the 95% confidence interval of the mean model difference includes zero. The largest mean offset is 2.251 mrad, which is small compared with the phase drift that must be compensated inside one block.

Table 6.2: Python-to-RTL phase-error differences for the top-level receiver, $n = 99$ blocks

CFO ($\Delta f T_s$)	SNR (dB)	Mean Error [mrad]	std dev [mrad]	95% CI [mrad]
$2e - 3$	24	+2.251	16.590	[-1.058, +5.559]
$2e - 3$	28	+1.254	16.590	[-1.823, + 4.332]
$1.5e - 3$	24	-0.070	16.722	[-3.405, +3.265]
$1.5e - 3$	28	+0.672	15.419	[-2.403, +3.747]

The remaining difference is consistent with the implementation differences between both models. The Python reference uses floating-point arithmetic, while the RTL uses fixed-point quantisation throughout the datapath. In addition, the Python model evaluates the arccosine operation with the NumPy implementation, whereas the RTL uses the LUT-based approximation described in Subsection 4.3.3. The final compensation rotation is also not identical: the RTL uses a CORDIC-based rotator, while the Python model uses floating-point complex exponentials. Finally, the phase-compensation section coefficients are quantised in the hardware implementation. In the RTL, the phase applied to section n follows from Equation 4.35, while the Python model uses the exact floating-point coefficients.

The relative size of the observed model difference can be estimated from the most demanding comparison case, with $\Delta f T_s = 2 \times 10^{-3}$ and SNR = 24 dB. The deterministic phase increment per symbol due to residual CFO is

$$\Delta\Phi = 2\pi\Delta f T_s = 12.57 \text{ mrad/symbol}. \quad (6.3)$$

For a block of 32 symbols, this corresponds to an intra-block phase change of approximately 0.39 rad, or 22.3 degrees. The largest mean model difference in Table 6.2 is therefore below 1% of the deterministic phase drift across the block. This does not remove the need to consider fixed-point deviations, as the standard deviations are larger than the means, but it does show that the implemented RTL does not introduce a measurable average phase bias relative to the reference model.

This agreement validates the use of the RTL output in the following sections. The subsequent performance trends can therefore be interpreted as consequences of the algorithmic parameters and channel conditions, rather than as artefacts of a gross mismatch between the reference model and the hardware implementation.

6.3 Correction behaviour

This section shows how the recovered signal evolves through the complete carrier-recovery chain. The purpose is to make the separate contributions of the centre-phase estimate and the phase-spread compensation visible before reducing the result to a single EVM number.

Figure 6.5 compares the impaired input, the output after SBPS-only correction and the output after combined SBPS+SE correction for the default case. The left constellation shows the impaired symbols before carrier recovery. The three visible rings correspond to the three possible magnitudes of the normalized 16QAM constellation. Because the residual CFO rotates the constellation over time, and the linewidth adds random phase perturbations, these magnitude classes appear as circular traces rather than as compact decision regions.

The middle constellation shows the effect of applying only the SBPS centre-phase estimate. In the RTL configuration this corresponds to using one compensation section, so every symbol in the block is rotated by the same unwrapped centre phase. The SBPS estimate already removes the dominant common phase rotation: the sixteen decision regions become visible again, and the received symbols are grouped around the ideal constellation points. However, the clusters remain elongated along a rotational direction. This residual spread is expected, because a single centre-phase estimate cannot compensate the phase slope that accumulates between the centre and the edges of the block.

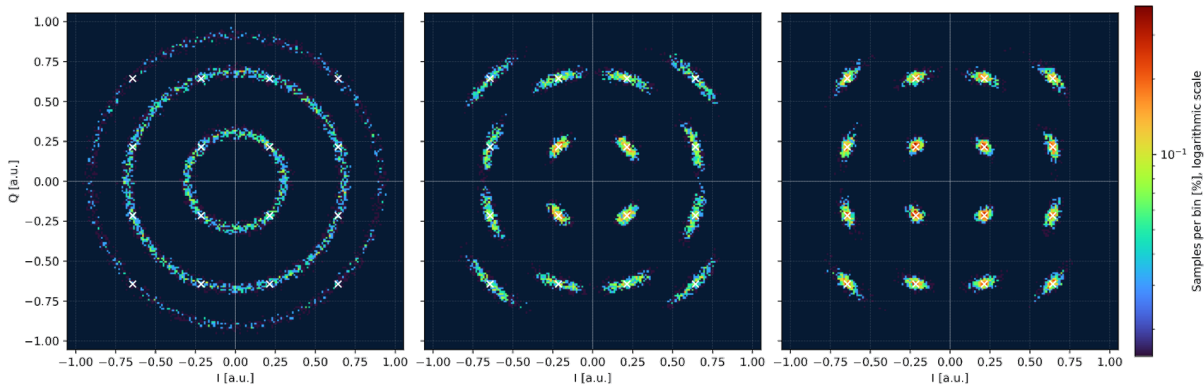


Figure 6.5: Constellation diagram throughout the recovery pipeline

The right constellation shows the default configuration, where the spread estimate is applied over eight compensation sections across the 32-symbol block. The remaining clusters are visibly more compact than in the SBPS-only case, especially for the outer constellation points where a given phase error produces a larger absolute I/Q displacement.

The EVM values quoted for this comparison are calculated by comparing each observed symbol stream with the corresponding clean transmitted 16QAM symbols:

$$\text{EVM} [\%] = 100 \sqrt{\frac{\sum_{k=0}^{N-1} |r_k - s_k|^2}{\sum_{k=0}^{N-1} |s_k|^2}}, \quad (6.4)$$

where r_k is either the impaired input, the SBPS-only output or the SBPS+SE output, and s_k is the corresponding clean reference symbol. The denominator normalizes the error to the average reference-symbol power, so the result can be reported as a percentage. With this definition, the impaired input has an EVM of 142%, the SBPS-only correction reduces it to 12.69%, and the combined SBPS+SE correction reduces it further to 7.41%.

The constellation plots are useful for detecting gross correction behaviour, but they do not fully characterize cycle slips. For square QAM, a wrong quadrant branch can rotate the constellation by a multiple of $\pi/2$, which may still look visually plausible in a scatter plot. The remaining sections therefore use EVM and symbol-index localisation to distinguish ordinary residual phase error from edge-related compensation limits.

6.4 Performance sweeps

The performance sweeps evaluate the two main complexity parameters of the receiver: the number of SBPS refinement stages and the number of compensation sections. The primary metric is the relative EVM reduction, denoted here as $\Delta\text{EVM}_{\text{dB}}$. This metric is used because it is more stable than bit error rate (BER) for the short simulation runs used in the parameter sweeps, while still reflecting how much closer the recovered symbols are to the clean reference constellation.

For each case, the input EVM is calculated against the clean transmitted symbols before carrier recovery, and the output EVM is calculated against the same reference after recovery. The plotted metric is

$$\Delta\text{EVM}_{\text{dB}} = 20 \log_{10} \left(\frac{\text{EVM}_{\text{out}}}{\text{EVM}_{\text{in}}} \right). \quad (6.5)$$

With this definition, a more negative value means a larger EVM reduction. The shaded multiplier count in the sweep figures gives a first-order indication of the corresponding hardware cost. It is not a complete post-synthesis resource report, but it shows the expected scaling trend as stages or sections are added.

6.4.1 Influence of SBPS refinement stages

The number of SBPS stages determines both the phase-search resolution and the amount of hardware required by the serialized search chain. Figure 6.6 shows the effect of the refinement depth on $\Delta\text{EVM}_{\text{dB}}$ for several linewidth settings. In this sweep, the residual CFO is kept at the default value while the linewidth is varied.

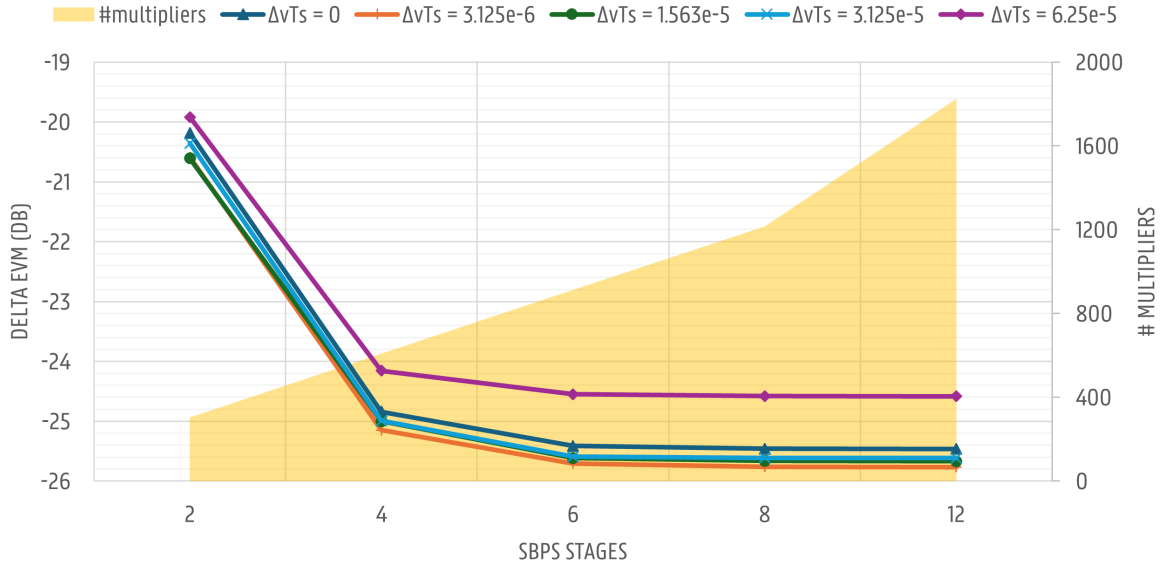


Figure 6.6: Effect of SBPS refinement depth on recovery performance

The strongest improvement occurs when increasing the search depth from two to four stages. With only two stages, the phase estimate is still too coarse, so the recovered EVM remains limited by phase quantisation in the SBPS estimate. Four stages already move the operating point close to the final performance region. Increasing the depth from four to six stages still gives a measurable improvement, but the gain is smaller. Beyond six stages, the curves are almost flat for all linewidths in the sweep.

This saturation is consistent with the role of the SBPS block in the complete architecture. Once the centre-phase estimate is sufficiently accurate, the dominant remaining errors no longer come from the SBPS search resolution. They are instead caused by additive noise, linewidth-induced phase variation, spread-estimation error and the finite granularity of the compensation sections. Adding more SBPS stages then increases the number of multipliers in the serialized chain without producing a proportional improvement in $\Delta\text{EVM}_{\text{dB}}$.

The linewidth dependence follows the same interpretation. The largest linewidth case remains worse than the lower-linewidth cases over the full stage sweep, because the local phase evolution becomes less well described by the deterministic centre-phase estimate plus a linear spread estimate. This effect cannot be removed by making the centre-phase search indefinitely fine.

6.4.2 Influence of compensation sections

The number of phase-compensation sections controls how finely the estimated spread is applied across one block. Figure 6.7 shows $\Delta\text{EVM}_{\text{dB}}$ versus the number of sections for a fixed residual CFO of $\Delta f T_s = 2 \times 10^{-3}$ and several linewidth settings. The one-section case represents pure blockwise compensation: the whole block is corrected with the unwrapped SBPS centre phase, and the spread estimate does not create an intra-block phase profile.

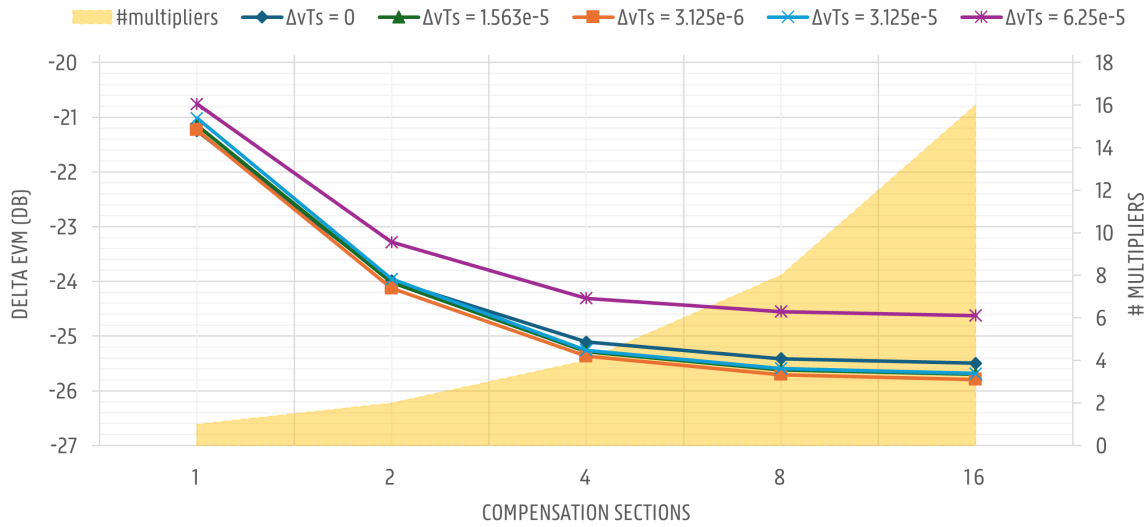


Figure 6.7: Effect of phase-compensation granularity on the recovery performance

Increasing the number of sections improves the recovery performance over the full linewidth range. This follows directly from the section-based compensation model in Equation 3.27: for an approximately linear phase ramp, smaller sections reduce the maximum residual phase error left inside each section. The largest gain is obtained when moving from one section to two and then to four sections. After this point, the curves begin to saturate. The eight- and sixteen-section configurations still improve the result slightly, but the improvement is much smaller than the cost increase suggested by the multiplier-count trend.

The linewidth trend is also visible in this sweep. The $\Delta\nu T_s = 0$ case, where only the deterministic residual CFO is present, obtains the best compensation result. As the linewidth increases, the recovered EVM worsens for every section count. This is expected from Equation 3.1: the spread estimator and section compensator assume that the phase trajectory inside one block is well approximated by a local linear model. A larger linewidth adds more stochastic, non-linear phase variation inside the block, so increasing the number of deterministic compensation sections cannot remove all of the remaining error.

6.4.3 Error localisation

The previous sweeps report one aggregate EVM value per configuration. To understand where the remaining error originates, the residual EVM is also inspected as a function of the symbol position inside the block. This is especially useful for distinguishing centre-phase errors from intra-block phase-slope errors. A centre-phase error would affect all symbol positions similarly, while insufficient spread compensation should mainly affect the block edges.

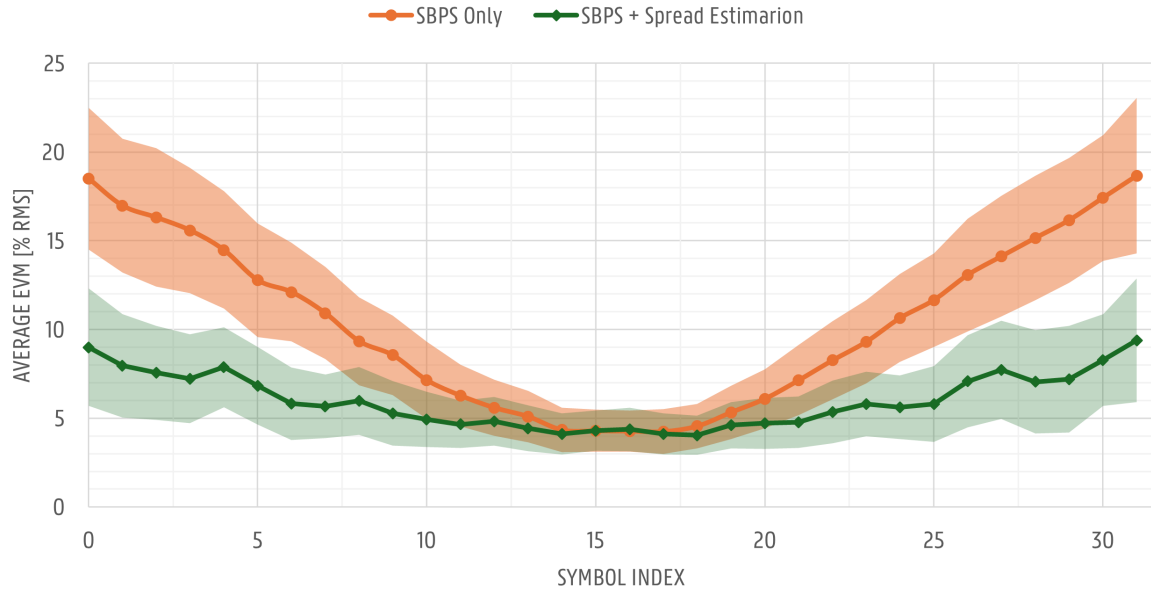


Figure 6.8: Residual EVM after SBPS correction and SBPS + SE correction

Figure 6.8 shows this localisation for the default case, with shaded regions representing the 95% CI. With SBPS-only correction, the residual EVM has a clear U-shaped profile: it is lowest near the centre of the block and increases towards both edges. This matches the expected behaviour of a blockwise centre-phase estimate. The centre of the block is corrected well, while symbols further away from the centre retain more of the uncompensated phase ramp.

After adding spread estimation and section-based compensation, the residual EVM becomes both lower and flatter over the block. The largest improvement occurs near the first and last symbols, where the SBPS-only correction leaves the largest phase error. Near the centre, the difference between both curves is smaller because the centre-phase estimate is already a good correction there. This confirms the intended division of work between both blocks: SBPS establishes the centre phase, while SE reduces the edge error caused by intra-block phase evolution.

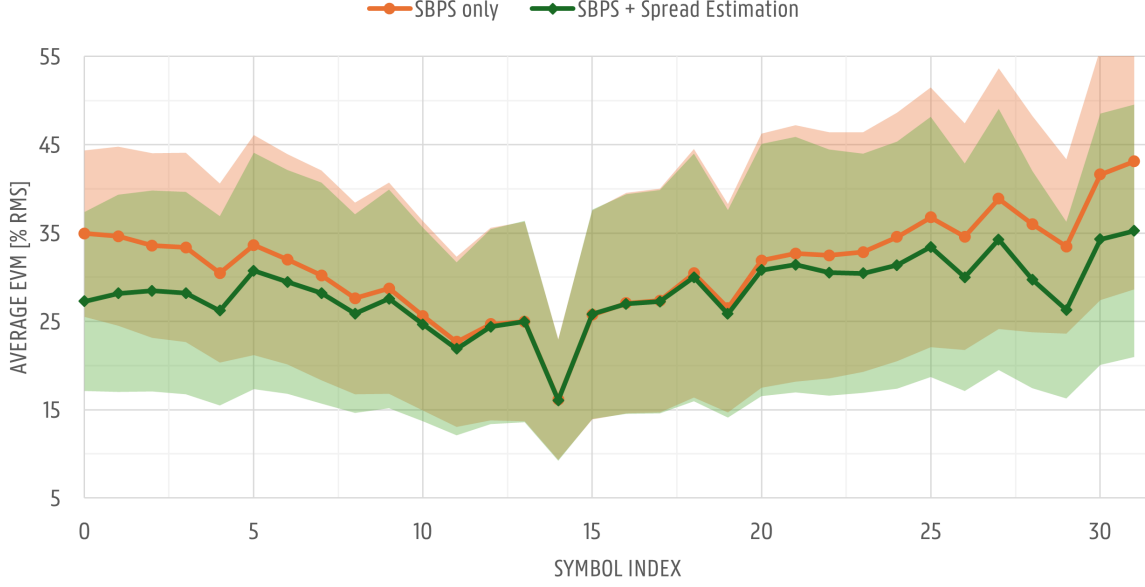


Figure 6.9: Residual EVM after SBPS correction and SBPS + SE correction, 96 MHz CFO

Figure 6.9 repeats the localisation for a more demanding 96 MHz residual-CFO case. At 32 GBd, this corresponds to $\Delta f T_s = 3 \times 10^{-3}$, so the deterministic phase slope inside one block is larger than in the default case. The residual EVM is substantially higher for both correction methods, and the spread between blocks is also larger. The SBPS+SE curve still improves the average result compared with SBPS only, but the improvement is less clean than in the default case.

This behaviour indicates that the receiver is closer to its practical tracking boundary. The centre of the block still benefits from the SBPS estimate, but the larger phase evolution makes the linear spread estimate and the section approximation more sensitive to estimation error and linewidth perturbations. The fact that the EVM remains position-dependent even with sixteen sections suggests that the limitation is not only the number of compensation sections; spread-estimation accuracy, unwrap robustness and the validity of the local-linear phase assumption also become important.

The hardware implementation was described in Chapter 4. The goal here is only to connect the measured performance to resource usage and latency. For the hardware implementation, the XCU-200-fsgd2104-2-e chip from the Virtex UltraScale+ series was used, with Vivado 2025.1 using the default implementation parameters.

6.4.4 Resource usage

The sweep results show where additional hardware stops giving a significant performance improvement for the default setup. The SBPS stage sweep points to six refinement stages as the useful knee of the curve, while the compensation-section sweep shows that eight sections capture most of the section-based improvement. The implementation results in this section therefore focus on this operating region and on the clock target that could be reached after routing.

Two sets of implementation results are relevant. The archived $B = 32$ clock sweep was run with six SBPS stages and eight compensation sections. It shows that the design closes timing at 10 ns, but that the same pipeline structure is just too slow for an 8 ns target. A revisited timing-critical version of the design contains the additional register inserted in the spread-estimation reciprocal-multiplication path, the most critical path.

Table 6.3: FPGA resource usage for the evaluated receiver configurations

Configuration	LUT	FF	DSP	BRAM	T_{clk} [ns]	Result
$B = 32$, baseline pipeline	33331	36006	229	4	10.0	pass
$B = 32$, baseline pipeline	33340	36006	229	4	8.0	fail
$B = 32$, timing-revised pipeline	33342	36007	229	4	8.0	pass

The table shows that the 8 ns failure in the archived $B = 32$ sweep is not caused by a large increase in area. The passing 10 ns point and the failing 8 ns point use practically the same resources: about 33.3k LUTs, 36.0k flip-flops, 229 DSP blocks and four BRAM tiles. The revisited design also has the same resource profile. Relative to the failing 8 ns sweep point, the routed registered design adds one flip-flop and only two LUTs, while the DSP and BRAM counts remain unchanged. On the XCU200 device this corresponds to approximately 2.82% of the LUTs, 1.52% of the registers, 3.35% of the DSP blocks and 0.19% of the BRAM tiles. This also supports the algorithmic choices from Figures 6.6 and 6.7. Adding SBPS stages replicates the two-branch rotate-metric-slice structure for each refinement step, so its cost increases approximately linearly with the number of stages. Adding compensation sections increases the number of section phase values and the amount of parallel rotation hardware required by the phase-correction block. Because the measured performance already plateaus around six stages and eight sections, the selected operating point gives a favourable balance between phase-recovery performance, area and timing margin.

6.4.5 Latency

The latency and timing results complete the hardware evaluation by showing whether the selected architecture can be clocked at the intended rate after implementation. Two quantities are considered separately. The first is the pipeline latency, expressed in clock cycles from the assertion of `in.valid` for one input block to the corresponding output-valid pulse. This value is fixed by the register structure described in Chapter 4. The second is the post-implementation timing margin reported by Vivado for a selected clock period. This value depends on placement, routing and the selected receiver parameters.

The clock-sweep flow is used to evaluate the design over a small set of candidate clock periods. For each point in the sweep, Vivado is run through implementation and timing analysis using the same RTL configuration and device target. The main timing quantities extracted from the implementation reports are the worst negative slack (WNS), total negative slack (TNS), worst hold slack (WHS), total hold slack (THS) and the number of failing setup and hold endpoints. The setup timing condition is considered closed when WNS is non-negative and TNS is zero. Similarly, hold timing is considered closed when WHS is non-negative and THS is zero.

$$\text{timing closed} = (\text{WNS} \geq 0) \wedge (\text{TNS} = 0) \wedge (\text{WHS} \geq 0) \wedge (\text{THS} = 0). \quad (6.6)$$

For a passing implementation point, the clock frequency is obtained from the constrained clock period:

$$f_{\text{clk}} = \frac{1}{T_{\text{clk}}}. \quad (6.7)$$

Table 6.4: Post-implementation timing-closure summary for the clock sweep

Configuration	B	T_{clk} [ns]	Stages	Sections	WNS [ns]	TNS [ns]	WHS [ns]	THS [ns]	Fail EP	Status
$B = 32$, baseline pipeline	32	10.0	6	8	+1.199	0.000	+0.030	0.000	0/0	pass
$B = 32$, baseline pipeline	32	8.0	6	8	-0.257	-6.797	+0.039	0.000	32/0	fail
$B = 32$, timing-revised pipeline	256	8.0	6	8	+0.064	0.000	+0.040	0.000	0/0	pass

The unrevisited pipeline confirms that the unregistered reciprocal-multiplication path is close to the 8 ns target but does not meet it after routing. The 10 ns point closes with 1.199 ns of setup slack, while the 8 ns point fails with -0.257 ns WNS and 32 failing setup endpoints. Hold timing is not the limiting factor in either case: the worst hold slack remains positive and THS is zero.

The routed report of the current registered top-level design shows the effect of inserting the additional register in the critical spread-estimation path. The 8 ns constraint is met with 0.064 ns WNS and 0.040 ns WHS, corresponding to a measured post-route operating point of 125 MHz. The worst setup path is still located inside the spread-estimation block, in the signed reciprocal-multiply logic.

But, the added register splits the arithmetic path enough to recover timing closure with almost no resource penalty.

The pipeline latency is reported in both cycles and time. If the implemented receiver latency is L cycles, the absolute latency at a selected clock period is

$$t_{\text{lat}} = LT_{\text{clk}}. \quad (6.8)$$

This latency does not affect the steady-state throughput when the design accepts one block per input transaction, but it determines the delay between an input block and the corresponding corrected output block.

Table 6.5: Pipeline-latency summary for the implemented receiver

Path or block	Latency [cycles]	Latency at 8 ns [ns]	Comment
SBPS valid alignment	$G_{\text{stages}}G_{\text{stage}}$	configuration dependent	Knee-point for $B=32$ uses $6 \cdot 14$ alignment cycles.
Spread-estimation synchroni- sation delay	15	120	Includes the additional register in the reciprocal-multiply path.
Phase-correction phase/data alignment	18	144	Delay used to align the corrected phase information with the block samples.
Additional timing-fix cost	+1	+8	Extra latency relative to the fail- ing 8 ns baseline path.

The final interpretation combines these timing results with the performance sweeps from Section 6.4. A configuration that improves $\Delta\text{EVM}_{\text{dB}}$ only slightly but reduces WNS or lowers the fastest passing clock frequency is not attractive for the final architecture. Conversely, the selected six-stage, eight-section $B = 32$ configuration is already close to 125 MHz, and the registered top-level implementation demonstrates that the remaining limit can be removed with a one-cycle local pipeline change. The throughput remains one block per accepted input transaction after pipeline fill, while the absolute latency increases by only 8 ns at the 125 MHz operating point.

Discussion

7.1 Limitations

The implemented architecture was evaluated as a hardware realization of the proposed SBPS+SE carrier-recovery concept. The most important configurable parameters in this evaluation are the number of SBPS refinement stages, the number of radius-directed stages, the processing block size and the number of phase-compensation sections. These parameters correspond directly to the algorithmic assumptions introduced in Chapter 3: SBPS is responsible for a sufficiently accurate centre-phase estimate, while SE and section-based compensation assume that the remaining intra-block phase evolution can be represented by a signed spread value around this centre phase.

The first limitation is therefore the locally linear phase assumption from Equation 3.1. This assumption is introduced in the system overview of Chapter 3 and then used explicitly in the spread-estimation derivation in Section 3.3.1 and in the compensation model of Equations 3.24–3.27. The results confirm both the usefulness and the boundary of this assumption. Figure 6.7 shows that increasing the number of compensation sections improves the result when the residual phase evolution is mainly a deterministic slope. However, Figure 6.9 shows that under a more demanding residual-CFO case, the remaining error stays position-dependent even with more compensation granularity. This indicates that spread-estimation accuracy, unwrap robustness and non-linear linewidth-induced phase variation also become limiting factors.

The second limitation is related to phase unwrapping. The unwrapper assumes that the leading and lagging edge estimates of neighbouring blocks provide a reliable continuity test, as formalized in Equation 3.19. Its quadrant update in Equation 3.21 is therefore reliable only as long as the estimated edge discontinuity remains within the $\pm\pi/4$ decision margin. A poor spread estimate can affect the system twice: it first causes an inaccurate intra-block compensation profile, and it can then move the edge-continuity decision closer to a wrong branch. In mild cases this appears as increased edge EVM; in severe cases it can become a cycle slip.

A third limitation follows from the fixed input scaling. Chapter 4 explains the Q1.15 sample representation and the static constellation scaling used to preserve headroom during rotations and moderate noise. Figures 4.1 and 4.2 show why this is necessary: clipping can distort the decision-directed SBPS metric and create false minima. The same scaling choice also affects the fourth-power path, because saturated samples would bias the selected-symbol and dot-product calculations. The implemented receiver is therefore not an adaptive front-end; changing the input dynamic range or operating at substantially different SNR conditions would require renewed fixed-point validation.

The fourth limitation is the dependence of SE on suitable 16QAM symbols. The input-symbol considerations in Section 3.3.1 explain why middle-ring symbols are less useful after the fourth-power operation and why the estimator selects representative candidates near the beginning, middle and end of a block. The hardware version of this selector is described in the SE input-selector section of Chapter 4. This keeps the design compact, but it makes the spread estimate sequence-dependent. Under unfavourable symbol patterns the estimator may have to hold previous candidates or use a less reliable pair, which can reduce the quality of the signed spread estimate.

Finally, the RTL contains several generics, but the complete verification flow is centred around the selected Q1.15 sample representation, Q4.28 phase representation and implemented lookup-table widths. These representations are summarized in Figure 4.3 and in Table 4.4. Changing these widths affects clipping behaviour, accumulator growth, LUT addressing, CORDIC conversion and valid-signal alignment. The current results should therefore be interpreted as validation of the selected implementation point,

not as proof that every parameterized variant is already verified.

7.2 Implementation Trade-Offs

The implementation deliberately favours explicit timing alignment over a more flexible streaming interface. The top-design overview in Chapter 4 shows that the raw I/Q block is sent in parallel to SBPS, SE and an I/Q delay branch. The *synchroniser* delays the signed spread estimate to match the SBPS centre-phase path, while *iq_block_delay* delays the raw samples until the corresponding phase estimates are available. This makes the timing contract easy to verify: when *PUPC* receives a valid phase estimate, it also receives the spread estimate and sample block belonging to the same input block.

The cost of this explicit alignment is that the wrapper is not yet a fully elastic receiver interface. Most internal datapaths are pipelined, and the spread-estimation backend can accept aligned valid inputs once filled, as shown in Tables 4.1 and 4.2. However, the complete design still relies on fixed valid pulses and deterministic delay lines rather than on a ready/valid protocol with backpressure. This is appropriate for the verification flow of Chapter 5, but a production receiver would require a more general streaming integration.

The mixed phase representation is another important trade-off. The SBPS stages use sine and cosine coefficients because the dominant operation is Cartesian rotation followed by slicing and metric accumulation, as described by Equation 4.18. This is efficient for the SBPS metric, but inconvenient for phase addition and interpolation. The downstream PU and PC blocks therefore use Q4.28 phase words, which make edge reconstruction, branch unwrapping and section-phase generation simple. The conversion cost at the boundary is justified because each representation is used where it best matches the required operation.

The resource results also show a practical timing trade-off. Table 6.3 and Table 6.4 show that the evaluated design is not limited by total area on the selected XCU200 device. Instead, the critical issue was a specific arithmetic path in the spread-estimation reciprocal-multiplication logic. Adding one local pipeline register allowed the design to meet the 8 ns timing target with negligible area increase. This illustrates that the main hardware challenge is not only reducing operation count, but also placing registers where wide fixed-point arithmetic would otherwise dominate the clock period.

7.3 Possible Improvements

One direct improvement is a more systematic fixed-point exploration. The present design validates one main numerical configuration. Sweeping the sample width, phase-word width, fourth-power output width, arccosine-LUT address width and section-coefficient precision would show which parts of the design are over-provisioned and which parts are accuracy-limiting. The spread-estimation result in Figure 6.3 makes this especially relevant near zero spread, where small normalized-dot-product errors are amplified by the nonlinear $\arccos(\cdot)$ mapping.

The input selector could also be improved. The current design searches a limited number of symbols near the beginning, middle and end of the block, then relies on the candidate-valid logic and hold behaviour described in Chapter 4. A more robust version could search a larger window, rank candidates by distance from ambiguous decision regions, or use reliability information from the slicer. Such changes would mainly target cases where wrong spread signs or stale spread values create edge-localized residual error.

Another possible improvement is adaptive parameter selection. The results in Figures 6.6 and 6.7 show that the useful number of SBPS stages and compensation sections depends on the channel condition. A receiver that estimates whether it is operating in a benign or demanding phase-noise regime could reduce switching activity by disabling unnecessary refinement stages or compensation sections when the full correction depth is not needed.

The SBPS rotation branches are also candidates for alternative implementation. The current design uses multiplier-based rotations to keep the branch latency short and predictable. A pipelined CORDIC-based rotator could reduce DSP usage, but it would increase latency and register usage and would require new top-level alignment settings. This option is therefore not automatically better, but it becomes relevant if DSP resources are more constrained than in the evaluated FPGA.

Finally, the architecture could be extended beyond the current 16QAM setting. Higher-order QAM formats would require different candidate-selection rules, different radius normalization and possibly

different modulation-removal logic. This extension would determine whether the same low-complexity centre-phase-plus-spread principle remains useful for coherent links with higher spectral efficiency.

7.4 Societal Responsibility

The societal relevance of this work comes from its intended application context: optical interconnects inside datacenters. As discussed in the introduction, datacenter traffic continues to grow because of cloud services, streaming platforms and AI workloads. Improving the efficiency of coherent receiver DSP can therefore contribute to communication systems that provide higher throughput without increasing hardware complexity proportionally.

The direct stakeholders are the designers and operators of optical transceivers and datacenter networks, who must balance bandwidth, latency, power consumption and cost. Users of datacenter services are indirect stakeholders, because the scalability and reliability of these services depend on the interconnect fabric. The environment is also a stakeholder. Datacenters consume substantial electrical power, and optical transceivers contribute to this demand. A lower-complexity carrier-recovery backend can reduce part of the digital processing cost, but it should not be presented as a complete sustainability solution. Efficiency improvements reduce the energy per processed bit, while total environmental impact still depends on deployment scale, energy source and growth in demand.

For that reason, the value of this thesis lies in making one receiver function more hardware-efficient and more transparent. The design choices, fixed-point assumptions, verification flow and limitations are explicitly documented, allowing future work to compare accuracy, resources and timing instead of treating the receiver backend as a black box.

The same transparency also prevents overclaiming. The architecture is evaluated for a specific modulation format, block size and impairment range. It supports more efficient coherent interconnect research, but practical deployment would still require integration into a complete receiver, power measurements on the target technology and evaluation under broader operating conditions. Within this scope, the thesis contributes to responsible system design by reducing unnecessary processing complexity while clearly identifying the remaining technical and environmental caveats.

This page is intentionally left blank.

Conclusion

8.1 Summary

This thesis investigated the hardware implementation of a low-complexity carrier-recovery backend for baudrate-sampled 16QAM coherent optical links. The motivation was the increasing complexity of carrier recovery in high-throughput coherent receivers, where conventional blind phase search provides robust phase estimation but requires many candidate rotations and metric evaluations. The proposed architecture reduces this cost by using a serialised binary phase-search structure and augments it with spread estimation so that residual intra-block phase evolution can still be compensated.

The work first described the phase-recovery algorithm from a system perspective. The SBPS block estimates the wrapped centre phase of a processing block, while the SE block estimates the signed phase spread across that block. The phase unwrapper then maintains the correct quadrant branch between successive blocks, and the phase compensator applies a section-based correction across the delayed input block. This division of work is central to the architecture: SBPS provides a robust phase anchor, while SE reduces the residual edge error caused by residual CFO.

The main contribution of this thesis is the translation of this algorithmic structure into a complete fixed-point FPGA implementation. The implementation uses Q1.15 packed I/Q samples, trigonometric phase coefficients in the SBPS path, Q4.28 phase words in the unwrapping and compensation path, and CORDIC-based rotation in the final correction stage. Within this implementation, the spread-estimation branch was developed as a hardware datapath with candidate-symbol selection, fourth-power processing, normalized dot-product evaluation, sign estimation, filtering and LUT-based angle recovery. These blocks are integrated in a top design where the raw input block is processed in parallel by SBPS and SE, while an explicit I/Q delay line preserves the corresponding sample block for the final *PUPC* correction stage.

A second contribution is the verification and evaluation framework around this implementation. Python reference models, fixed-point vector generation, VHDL testbenches and post-processing scripts were used to validate both individual arithmetic paths and the complete wrapper. The top-level comparison showed no persistent phase drift between the Python model and RTL; the largest measured mean model difference was 2.251 mrad over the evaluated cases, which is small compared with the intra-block phase drift that must be compensated.

The system-level results confirmed the purpose of the spread-estimation path. In the default test case, the impaired signal had an EVM of 142%. Applying only the blockwise SBPS correction reduced this to 12.69%, while the combined SBPS+SE correction reduced it further to 7.41%. Error-localisation plots showed that the additional spread compensation mainly improves the first and last symbols of the block, as expected from a correction method that targets intra-block phase slope.

8.2 Achieved Objectives

The first objective was to implement the proposed low-complexity carrier-recovery architecture on an FPGA. This objective was achieved by implementing the SBPS, SE, phase-unwrapping and phase-compensation paths as a complete fixed-point RTL design. The resulting top-level wrapper connects the estimators and correction stage through explicit latency-alignment blocks, making the full processing chain available for simulation and implementation.

The second objective was to preserve the intended algorithmic behaviour under fixed-point hardware constraints. The block-level and top-level verification results show that the RTL follows the refer-

ence behaviour closely enough for system-level interpretation. The remaining differences are consistent with fixed-point quantisation, LUT-based arccosine approximation, coefficient quantisation and CORDIC rounding. No systematic phase bias was observed in the evaluated top-level cases.

The third objective was to evaluate the trade-off between receiver performance and hardware cost. The stage-count sweep showed that increasing the SBPS depth gives the largest benefit up to approximately six stages for the evaluated configuration. Beyond this point, the remaining error is dominated by spread-estimation error, linewidth-induced phase variation and compensation granularity rather than by centre-phase search resolution. The compensation-section sweep showed a similar saturation: moving from one section to multiple sections gives a clear improvement, but the gain becomes smaller beyond eight sections.

The fourth objective was to assess whether the design can be implemented with realistic timing. The evaluated $B = 32$, six-stage, eight-section configuration used about 33.3k LUTs, 36.0k flip-flops, 229 DSP blocks and four BRAM tiles on the XCU200 device. A local timing fix in the spread-estimation reciprocal-multiplication path allowed the design to close timing at an 8 ns clock period, corresponding to 125 MHz, with negligible additional area.

Overall, the thesis demonstrates that the proposed SBPS+SE architecture is implementable as a complete fixed-point FPGA datapath and that the spread-estimation extension provides a measurable performance benefit over SBPS-only blockwise correction under the evaluated residual-CFO conditions.

8.3 Main Contributions

The contributions of this thesis can be summarized as a coherent implementation path from algorithm to measured hardware design. Starting from the proposed SBPS+SE concept, the work develops a fixed-point RTL receiver backend in which the centre-phase estimator, spread estimator, phase unwrapper and phase compensator are implemented as interacting hardware blocks rather than as isolated models.

The spread-estimation path forms the most specific hardware contribution. It combines candidate-symbol selection, fourth-power arithmetic, normalized dot-product processing, sign estimation, filtering and LUT-based angle recovery into a pipeline that produces a signed Q4.28 spread estimate. This estimate is then integrated with the SBPS centre-phase estimate and the delayed I/Q samples through the top-level alignment structure described in Chapter 4.

The final contribution is the validation and interpretation of the resulting design. The thesis establishes a reproducible Python-to-VHDL verification flow and uses it to compare the fixed-point RTL against reference behaviour. The performance and implementation results then connect algorithmic parameters, such as SBPS stage count and compensation-section count, to EVM improvement, latency, timing closure and resource usage.

8.4 Future Work

Several directions remain open for future work. A first extension is a more systematic fixed-point exploration. Although the design contains several generics, the verification flow in this thesis focused on one main Q1.15/Q4.28 configuration. Sweeping sample width, phase-word width, accumulator width and LUT resolution would show which parts of the datapath are accuracy-limiting and which parts could be reduced to save area or power.

The SBPS rotation branches are another candidate for optimization. The current implementation uses multiplier-based rotations to keep stage latency predictable. A pipelined CORDIC-based implementation could reduce DSP usage, but it would need a careful comparison of latency, register cost, numerical accuracy and top-level timing alignment.

Future work should also consider a more streaming-oriented receiver integration. The present wrapper uses fixed valid pulses and deterministic delay lines, which is useful for verification, but a production receiver would require integration with the preceding equalization and frequency-recovery stages, continuous scheduling and possibly ready/valid backpressure.

Finally, the architecture should be evaluated beyond the current 16QAM setting. Higher-order QAM formats would require new candidate-selection rules, different normalization constants and possibly different modulation-removal logic. Extending the method in that direction would determine whether the same low-complexity carrier-recovery principle remains useful for future coherent datacenter links with higher spectral efficiency.

Bibliography

- [1] D. J. Blumenthal, H. Ballani, R. O. Behunin, J. E. Bowers, P. Costa, D. Lenoski, P. A. Morton, S. B. Papp, and P. T. Rakich, “Frequency-stabilized links for coherent wdm fiber interconnects in the datacenter,” *Journal of Lightwave Technology*, vol. 38, no. 13, pp. 3376–3386, 2020.
- [2] C. Zhang, F. Chen, L. Wang, L. Wang, and C. Yue, “Recent advances of high-speed short-reach optical interconnects for data centers,” *IEEE Open Journal of the Solid-State Circuits Society*, vol. 5, pp. 86–100, 2025.
- [3] X. Zhou, C. F. Lam, R. Urata, and H. Liu, “State-of-the-art 800g/1.6t datacom interconnects and outlook for 3.2t,” in *Optical Fiber Communication Conference (OFC) 2023*, p. W3D.1, 2023.
- [4] X. Zhou, R. Urata, and H. Liu, “Beyond 1 tb/s intra-data center interconnect technology: Im-dd or coherent?,” *Journal of Lightwave Technology*, 2020.
- [5] P. M. Seiler, G. Georgieva, G. Winzer, A. Peczek, K. Voigt, S. Lischke, A. Fatemi, and L. Zimmermann, “Toward coherent o-band data center interconnects,” *Frontiers of Optoelectronics*, vol. 14, no. 4, pp. 414–425, 2021.
- [6] J. M. Kahn and K.-P. Ho, “Spectral efficiency limits and modulation/detection techniques for dwdm systems,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 10, no. 2, pp. 259–272, 2004.
- [7] S. J. Savory, “Digital coherent optical receivers: Algorithms and subsystems,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 16, no. 5, pp. 1164–1179, 2010.
- [8] M. Kuschnerov, F. N. Hauske, K. Piyawanno, B. Spinnler, M. S. Alfiad, A. Napoli, and B. Lankl, “Dsp for coherent single-carrier receivers,” *Journal of Lightwave Technology*, vol. 27, no. 16, pp. 3614–3622, 2009.
- [9] K. Kikuchi, “Fundamentals of coherent optical fiber communications,” *Journal of Lightwave Technology*, 2016.
- [10] J. Cheng, C. Xie, Y. Chen, X. Chen, M. Tang, and S. Fu, “Comparison of coherent and imdd transceivers for intra datacenter optical interconnects,” in *Optical Fiber Communication Conference (OFC) 2019*, p. W1F.2, 2019.
- [11] A. Leven, N. Kaneda, U.-V. Koc, and Y.-K. Chen, “Frequency estimation in intradyne reception,” *IEEE Photonics Technology Letters*, vol. 19, no. 6, pp. 366–368, 2007.
- [12] E. Ip and J. M. Kahn, “Feedforward carrier recovery for coherent optical communications,” *Journal of Lightwave Technology*, vol. 25, no. 9, pp. 2675–2692, 2007.
- [13] T. Pfau, S. Hoffmann, and R. Noé, “Hardware-efficient coherent digital receiver concept with feed-forward carrier recovery for m-qam constellations,” *Journal of Lightwave Technology*, vol. 27, no. 8, pp. 989–999, 2009.
- [14] A. (Xilinx), “Cordic v6.0 logiccore ip product guide (pg105),” 2021. Release date 2021-08-06.
- [15] e. a. Jonas De Busscher, Axl Bomhals, “Low complexity carrier recovery system design for baudrate sampled coherent datacenter interconnects,” *IEEE Communications Letters*, 2025.
- [16] M. S. Wartak, *Computational Photonics: An Introduction with MATLAB*. Cambridge University Press, 2013. Chapter 10: Optical receivers.
- [17] I. Roudas, “Coherent optical communication systems,” in *Optical Networks: WDM Systems and Networks*, Springer, 2011.

- [18] M. Selmi, Y. Jaouen, and P. Ciblat, “Accurate digital frequency offset estimator for coherent polmux qam transmission systems,” in *Proceedings of the European Conference on Optical Communication (ECOC)*, 2009. Paper P3.08, Vienna.
- [19] J. C. Rasmussen, “Advances in coherent detection algorithms,” in *Proceedings of SPIE*, vol. 7960, 2011.
- [20] A. Devices, “Mixed-signal and dsp design techniques: The dft/fft (bin spacing and frequency resolution).” PDF technical handbook chapter.
- [21] H.-P. . A. Technologies, “Spectrum and signal analyzer measurements and noise.” Technical measurement guide (FFT/bin-based resolution and estimation effects).
- [22] T. Yang *et al.*, “Linewidth-tolerant and multi-format carrier phase estimation based on blind phase search,” *Optics Express*, 2018.
- [23] J. E. Volder, “The cordic trigonometric computing technique,” *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [24] P. K. Meher, J. Valls, T.-B. Juang, K. Sridharan, and K. Maharatna, “50 years of cordic: Algorithms, architectures, and applications,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 56, no. 9, pp. 1893–1907, 2009.
- [25] J. Li, L. Li, Z. Tao, T. Hoshida, and J. C. Rasmussen, “Laser-linewidth-tolerant feed-forward carrier phase estimator with reduced complexity for qam,” *Journal of Lightwave Technology*, vol. 29, no. 16, pp. 2358–2364, 2011.
- [26] D. A. A. Mello, F. A. Barbosa, and J. D. Reis, “Interplay of probabilistic shaping and the blind phase search algorithm,” *Journal of Lightwave Technology*, vol. 36, p. 5096–5105, Nov. 2018.

This page is intentionally left blank.

This page is intentionally left blank.

